

Sistemas Operacionais

Roteiros dos Trabalhos Práticos

Teoria, roteiro passo a passo e trechos de código em C e Python

- 1 Shell Simplificado (Mini-Shell)
- 2 Simulador de Escalonador de CPU
- 3 Simulador de Paginação
- 4 Alocador de Memória Customizado
- 5 Problemas Clássicos de Sincronização

Sumário

Orientações Gerais	1
Como este material está organizado	1
Sobre os trechos de código	2
Ambiente de desenvolvimento	3
Método de trabalho recomendado	4
Relatório	5
O que deve ser entregue	5
Integridade acadêmica	6
Visão geral dos cinco trabalhos	6
1 Shell Simplificado (Mini-Shell)	7
1.1 Visão geral	7
1.2 Fundamentação teórica	8
1.3 Especificação do trabalho	18
1.4 Roteiro de implementação	20
1.5 Exemplos de entrada e saída	31
1.6 Experimentos e validação funcional	33
1.7 Relatório	34
1.8 Critérios de avaliação e checklist	35
1.9 Leituras recomendadas	35
2 Simulador de Escalonador de CPU	37
2.1 Visão geral	37
2.2 Fundamentação teórica	37
2.3 Especificação do trabalho	47
2.4 Roteiro de implementação	49
2.5 Exemplos de entrada e saída	56
2.6 Experimentos	58
2.7 Perguntas para o relatório	60
2.8 Critérios de avaliação e checklist	60
2.9 Leituras recomendadas	61

3	Simulador de Paginação	62
3.1	Visão geral	62
3.2	Fundamentação teórica	63
3.3	Especificação do trabalho	72
3.4	Roteiro de implementação	73
3.5	Exemplos de entrada e saída	82
3.6	Experimentos	85
3.7	Perguntas para o relatório	87
3.8	CrITÉrios de avaliação e checklist	87
3.9	Leituras recomendadas	88
4	Alocador de Memória Customizado	90
4.1	Visão geral	90
4.2	Fundamentação teórica	91
4.3	Especificação do trabalho	97
4.4	Como simular memória em Python	99
4.5	Roteiro de implementação	100
4.6	Exemplos de entrada e saída	107
4.7	Experimentos	110
4.8	Perguntas para o relatório	112
4.9	CrITÉrios de avaliação	112
4.10	Leituras recomendadas	113
5	Problemas Clássicos de Sincronização	114
5.1	Visão geral	114
5.2	Fundamentação teórica	115
5.3	Especificação do trabalho	127
5.4	Roteiro de implementação	128
5.5	Exemplos de saída esperada	137
5.6	Experimentos	139
5.7	Perguntas para o relatório	140
5.8	CrITÉrios de avaliação e checklist	141
5.9	Leituras recomendadas	142
A	Equivalências entre C e Python	143
A.1	Processos, arquivos e sinais (Capítulo 1)	143
A.2	Threads e sincronização (Capítulo 5)	144
A.3	Estruturas de dados (Capítulos 2, 3 e 4)	144
B	Bibliografia geral	145

Orientações Gerais

Este material reúne, em um único documento, os cinco trabalhos práticos da disciplina de Sistemas Operacionais. Cada capítulo corresponde a um trabalho diferente e foi escrito supondo que **você ainda não estudou a teoria relacionada** a ele. Por isso, antes do roteiro de implementação, cada capítulo traz uma fundamentação teórica que apresenta os conceitos necessários “do zero”, com definições, exemplos resolvidos e diagramas.

Como este material está organizado

Todos os capítulos seguem a mesma estrutura, para que você saiba sempre onde procurar cada tipo de informação:

1. **Visão geral e objetivos** — o que será construído e o que se espera que você aprenda.
2. **Fundamentação teórica** — os conceitos de Sistemas Operacionais que o trabalho exercita.
Leia esta parte com calma antes de começar a programar.
3. **Especificação do trabalho** — requisitos obrigatórios e desejáveis, formato de entrada e saída, estrutura de entrega.
4. **Roteiro de implementação** — uma sequência de passos incrementais. Cada passo diz *o que fazer, por que fazer* naquela ordem, mostra trechos de código em C e em Python e termina com uma lista do que verificar antes de seguir.
5. **Exemplos de entrada e saída** — casos de teste com o resultado esperado.
6. **Experimentos** — o que medir e comparar para o relatório.
7. **Relatório, critérios de avaliação e checklist de entrega.**
8. **Leituras recomendadas.**

Ao longo do texto aparecem caixas coloridas com papéis específicos:

Definição: termo

Definição formal de um conceito importante. Vale a pena memorizar.

Ideia-chave

Explicação intuitiva, analogia ou observação que ajuda a entender um conceito.

Atenção

Erros comuns, armadilhas e pontos em que muitos alunos perdem tempo (ou nota).

Dica

Sugestões práticas de implementação, depuração ou organização.

Verifique antes de seguir

Testes que o seu programa deve passar ao final de um passo. Não avance para o próximo passo enquanto estes testes não funcionarem.

Sobre os trechos de código

Os trabalhos podem ser feitos em **C** ou em **Python**. Por isso, cada passo do roteiro traz trechos nas duas linguagens, identificados assim:

</> C | exemplo de trecho em C

```
/* Estruturas, assinaturas e fragmentos de lógica */  
int exemplo(int x) {  
    /* TODO: a implementação é sua */  
}
```

Python | exemplo de trecho em Python

```
def exemplo(x: int) -> int:  
    # TODO: a implementação é sua  
    ...
```

Atenção

Os trechos são propositalmente incompletos. Eles mostram a *forma* de uma estrutura de dados, a assinatura de uma função, o uso correto de uma chamada de sistema ou um fragmento de lógica — nunca a solução inteira. As lacunas estão marcadas com **TODO** e reticências (...). Copiar os trechos e “preencher o mínimo” não é suficiente: você precisa entender a teoria e o roteiro para completar cada função.

Escolhendo a linguagem

Escolha **uma** linguagem por trabalho (e mantenha-a do início ao fim). As duas opções são válidas, mas exercitam o sistema operacional de maneiras um pouco diferentes:

Trabalho	Em C	Em Python
Mini-Shell	<code>fork</code> , <code>execvp</code> , <code>pipe</code> , <code>dup2</code> , <code>waitpid</code> , <code>sigaction</code>	As mesmas chamadas via módulo <code>os</code> (<code>os.fork</code> , <code>os.execvp</code> , <code>os.pipe</code> , <code>os.dup2</code> , <code>os.waitpid</code>) e <code>signal</code> . Proibido usar <code>subprocess</code> ou <code>os.system</code> .
Escalonador	<code>struct</code> , vetores, filas implementadas à mão	<code>dataclass</code> , listas, <code>collections.deque</code>
Paginação	vetores de quadros, contadores	listas, <code>dict</code> , <code>deque</code>
Alocador	heap real (um único <code>malloc/mmap</code> ou vetor estático), ponteiros e aritmética de ponteiros	heap simulado em um único <code>bytearray</code> ; cabeçalhos gravados com <code>struct.pack_into</code> ; “ponteiros” viram deslocamentos inteiros
Sincronização	<code>pthread</code> , <code>pthread_mutex_t</code> , <code>sem_t</code> , <code>pthread_cond_t</code>	<code>threading.Thread</code> , <code>Lock</code> , <code>Semaphore</code> , <code>Condition</code> . Proibido usar <code>queue.Queue</code> como buffer.

💡 Por que C é a linguagem “natural” de SO?

Praticamente todos os sistemas operacionais modernos (Linux, os núcleos do Windows e do macOS) são escritos em C. As chamadas de sistema do Linux são expostas como funções C (biblioteca `libc`). Em Python, o módulo `os` é apenas uma camada fina sobre essas mesmas funções: `os.fork()` chama o `fork()` da `libc`. Por isso, quem programar em Python estará usando exatamente os mesmos mecanismos do sistema — só com uma sintaxe mais confortável.

Ambiente de desenvolvimento

O uso de **Linux (nativo ou WSL no Windows)** é obrigatório. Trabalhos que não compilem ou não executem em Linux serão desconsiderados. Em uma distribuição baseada em Debian/Ubuntu, instale as ferramentas com:

>_ Instalação das ferramentas (Ubuntu/Debian/WSL)

```
$ sudo apt update
$ sudo apt install build-essential gdb valgrind make python3 python3-pip
$ gcc --version          # confirme que o compilador está instalado
$ python3 --version      # Python 3.10 ou superior
```

Compilação e Makefile

Todos os projetos devem incluir um **Makefile** funcional: o professor deve conseguir compilar (ou executar, no caso de Python) o seu trabalho digitando apenas **make**. Um esqueleto mínimo para C é mostrado abaixo — ajuste nomes de arquivos e alvos ao seu projeto.

📄 Makefile (esqueleto para C)

```
CC      = gcc
CFLAGS = -Wall -Wextra -g -std=c11
```

```

SRC      = src/main.c src/modulo1.c   # TODO: liste seus arquivos
OBJ      = $(SRC:.c=.o)
EXEC     = programa                   # TODO: nome do executável

all: $(EXEC)

$(EXEC): $(OBJ)
    $(CC) $(CFLAGS) -o $@ $^

clean:
    rm -f $(OBJ) $(EXEC)

```

Para projetos em Python, o **Makefile** pode simplesmente oferecer alvos de execução e de teste:

Makefile (esqueleto para Python)

```

PY = python3

run:
    $(PY) src/main.py      # TODO: argumentos padrão

test:
    $(PY) -m pytest testes/ # ou o seu script de testes

```

Dica

Compile sempre com **-Wall -Wextra** e trate *todos* os avisos (*warnings*). A maior parte dos bugs difíceis de encontrar em C (variável não inicializada, comparação entre tipos diferentes, retorno esquecido) aparece antes como um aviso do compilador.

Ferramentas de depuração

Ferramenta	Para que serve
gdb	Depurador interativo para C: executar passo a passo, inspecionar variáveis, ver a pilha de chamadas (bt) de um programa travado.
valgrind	Detecta vazamentos de memória, acessos inválidos e uso de memória não inicializada (valgrind --leak-check=full ./prog).
-fsanitize=address	<i>AddressSanitizer</i> : detecta estouro de vetores e uso de memória já liberada, com mensagens muito claras.
-fsanitize=thread	<i>ThreadSanitizer</i> : detecta condições de corrida (Cap. 5).
strace	Mostra cada chamada de sistema feita por um processo — excelente para o Mini-Shell (strace -f ./mysh).
ps, pstree	Listam processos; úteis para detectar processos zumbis.
pdb	Depurador do Python (python3 -m pdb main.py).

Método de trabalho recomendado

1. **Leia a teoria antes de programar.** Tente explicar com suas palavras, para um colega, cada definição do capítulo.

2. **Resolva os exemplos à mão.** Antes de implementar um algoritmo, resolva no papel os exemplos da seção de entrada e saída. Se você não consegue resolver à mão, não conseguirá programar.
3. **Desenvolva de forma incremental.** Siga os passos do roteiro na ordem; cada passo produz um programa que compila e funciona, ainda que incompleto.
4. **Teste a cada passo.** Use as caixas “Verifique antes de seguir”.
5. **Use controle de versão (git).** Faça um *commit* ao final de cada passo: além de proteger seu trabalho, o histórico evidencia a autoria.
6. **Anote decisões e dificuldades** enquanto trabalha — elas serão o material bruto da seção de implementação e da discussão do relatório.

Relatório

- O relatório deve ser escrito no **Overleaf**, utilizando o **modelo oficial da SBC** (Sociedade Brasileira de Computação) para artigos.
- Estrutura mínima: Introdução, Fundamentação/Conceitos, Metodologia/Implementação, Resultados Experimentais, Discussão (respostas às perguntas de cada trabalho) e Conclusão.
- Extensão recomendada: 5 a 8 páginas no formato SBC.

⚠ Atenção

Relatórios entregues apenas em PDF, sem os fontes `.tex`, ou que não sigam o modelo SBC, serão desconsiderados.

O que deve ser entregue

A entrega de cada trabalho deve conter obrigatoriamente:

1. **Código-fonte completo**, organizado conforme a estrutura sugerida no capítulo do trabalho, com `Makefile`.
2. **Fontes `.tex` do relatório** (projeto Overleaf exportado, incluindo imagens e bibliografia).
3. **PDF final do relatório**, gerado a partir do mesmo projeto Overleaf.

Prazos

Data	Entrega
08/09/2026	Projeto: implementação completa, descrição da implementação, validação e experimentos
20/09/2026	Trabalho completo: código + relatório (fontes <code>.tex</code> e PDF, modelo SBC)

Integridade acadêmica

⚠ Atenção

- Todas as entregas serão submetidas a **verificação de plágio** (comparação de código entre grupos e turmas, por exemplo com MOSS, e comparação textual dos relatórios).
- **Plágio anula o trabalho**: cópia total ou parcial de código ou relatório de outro grupo, de turmas anteriores ou de fontes externas sem citação resulta em nota zero para todos os envolvidos.
- **O uso de Inteligência Artificial (ChatGPT, Copilot, Claude e similares) para gerar código ou relatório é proibido** e também anula o trabalho. A IA pode ser usada *apenas* para tirar dúvidas conceituais gerais.
- Grupos poderão ser convocados para uma **arguição oral** sobre o código e o relatório. Você deve ser capaz de explicar cada linha que entregou.

Visão geral dos cinco trabalhos

Cap.	Trabalho	Principais conceitos exercitados
1	Mini-Shell	processos, fork/exec/wait , descritores de arquivo, redirecionamento, pipes, sinais
2	Escalonador de CPU	estados de processo, troca de contexto, FCFS, SJF, SRTF, Round Robin, prioridade, <i>aging</i> , métricas de desempenho
3	Simulador de Paginação	memória virtual, páginas e quadros, <i>page fault</i> , FIFO, LRU, Ótimo, Segunda Chance, anomalia de Belady
4	Alocador de Memória	heap, malloc/free , fragmentação interna e externa, First/Best/Worst-Fit, divisão e coalescência de blocos
5	Sincronização	threads, condição de corrida, mutex, semáforos, variáveis de condição, deadlock, <i>starvation</i>

Capítulo 1

Shell Simplificado (Mini-Shell)

1.1 Visão geral

Toda vez que você abre um terminal no Linux e digita `ls -l`, um programa chamado *shell* (`bash`, `zsh`, `sh`...) lê o que você escreveu, descobre qual programa deve ser executado, pede ao sistema operacional que crie um novo processo para executá-lo, espera esse processo terminar e mostra o prompt novamente. Parece simples, mas por trás desse ciclo estão alguns dos conceitos mais importantes de Sistemas Operacionais: **processos**, **chamadas de sistema**, **descritores de arquivo**, **comunicação entre processos** (*pipes*) e **sinais**.

Neste trabalho você vai construir o seu próprio shell, o `mysh`. Ele deverá executar programas externos, tratar comandos internos (`cd`, `pwd`, `exit`, `export`), redirecionar entrada e saída para arquivos, conectar vários programas por pipes, executar comandos em segundo plano e sobreviver ao `Ctrl+C`. O trabalho pode ser feito em **C** ou em **Python**; em ambos os casos você usará diretamente as chamadas de sistema POSIX (em Python, através do módulo `os`).

🎯 Ao final deste trabalho você deverá ser capaz de

- Explicar o que é um processo e como o sistema operacional o representa.
- Usar corretamente `fork`, a família `exec` e `wait/waitpid` para criar, executar e aguardar processos.
- Entender a tabela de descritores de arquivo e usar `open`, `close` e `dup2` para redirecionar entrada e saída.
- Conectar processos por meio de `pipe` e justificar por que as pontas não usadas devem ser fechadas.
- Tratar sinais (`SIGINT`, `SIGCHLD`) e evitar processos zumbis.
- Organizar um programa de sistema em módulos (parser, executor, builtins) e validá-lo sistematicamente.

⚠️ Atenção

Este capítulo apresenta **trechos** de código para ilustrar o uso das chamadas de sistema. Eles são propositalmente incompletos (marcados com `TUDO` e reticências). Montar o programa completo, decidir a organização e tratar os casos de erro é tarefa sua. Lembre-se: o uso de IA para gerar o código anula o trabalho, e você poderá ser chamado a explicar oralmente qualquer linha que entregar.

1.2 Fundamentação teórica

Esta seção apresenta, do zero, os conceitos necessários para o trabalho. Leia com calma antes de escrever qualquer linha de código: praticamente todos os erros que alunos cometem no mini-shell vêm de lacunas nesses conceitos, e não de problemas de programação.

1.2.1 O que é um sistema operacional

Um **sistema operacional** (SO) é o software que gerencia os recursos do computador — processador, memória, discos, dispositivos de entrada e saída — e oferece aos programas uma interface mais simples e segura para usá-los. Ele cumpre dois papéis:

- **Máquina estendida:** esconde os detalhes do hardware. Seu programa escreve em um “arquivo” sem precisar saber em que setor do disco os bytes vão parar.
- **Gerente de recursos:** decide quem usa a CPU, quanta memória cada programa recebe, quem pode abrir cada arquivo, e impede que um programa atrapalhe o outro.

A parte do SO que está sempre residente na memória e executa com privilégios máximos é chamada de **núcleo** (*kernel*). Programas como o **bash**, o **ls** ou o seu **mysh** *não* fazem parte do núcleo: são programas de usuário comuns.

Modo usuário e modo núcleo

Os processadores modernos possuem (pelo menos) dois modos de execução:

Modo núcleo (*kernel mode*) todas as instruções são permitidas, inclusive acessar dispositivos e alterar tabelas de memória.

Modo usuário (*user mode*) instruções privilegiadas são proibidas; se um programa tentar executá-las, o hardware gera uma exceção e o SO toma o controle.

Seus programas rodam em modo usuário. Então, como um programa consegue ler um arquivo ou criar outro processo, se ele próprio não pode acessar o disco nem mexer nas tabelas do sistema? Pedindo ao núcleo, por meio de uma **chamada de sistema**.

Definição: Chamada de sistema (*system call*)

É o mecanismo pelo qual um programa em modo usuário solicita um serviço ao núcleo do sistema operacional. A chamada provoca uma troca controlada para o modo núcleo (via uma instrução especial, como **syscall** no x86-64), o núcleo executa o serviço e devolve o resultado, retornando ao modo usuário.

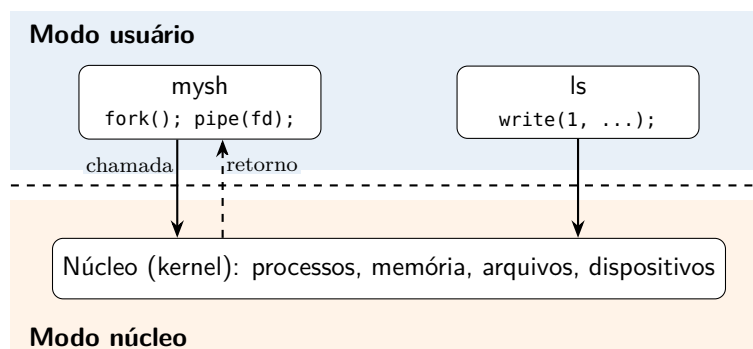


Figura 1.1: Programas de usuário pedem serviços ao núcleo por meio de chamadas de sistema.

No Linux, as chamadas de sistema são expostas aos programas C pela biblioteca padrão (**glibc**) como funções comuns: `fork()`, `read()`, `write()`, `open()`... Em Python, o módulo `os` oferece funções equivalentes com quase os mesmos nomes (`os.fork()`, `os.read()`, `os.open()`...). A documentação de cada uma está nas *man pages*: `man 2 fork` mostra a seção 2 (chamadas de sistema) e `man 3 execvp` a seção 3 (funções de biblioteca).

Dica

Quase toda chamada de sistema devolve `-1` em caso de erro e guarda o motivo na variável global `errno`. A função `perror("msg")` imprime “msg: descrição do erro”. Em Python, as funções do módulo `os` lançam a exceção `OSError` (ou subclasses como `FileNotFoundError`), que você deve capturar com `try/except`. **Sempre** verifique o retorno.

1.2.2 Processos

Definição: Processo

Um **processo** é um programa em execução. Enquanto o *programa* é um arquivo passivo no disco (por exemplo, `/usr/bin/ls`), o processo é uma entidade ativa: possui código carregado na memória, dados, uma pilha, valores nos registradores da CPU, arquivos abertos e um identificador único.

Um mesmo programa pode originar vários processos simultâneos: se três usuários executam `vim`, existem três processos distintos rodando o mesmo programa.

Identificador de processo (PID)

Cada processo recebe do núcleo um número inteiro único, o **PID** (*Process IDentifier*). Todo processo (exceto o primeiro) também conhece o PID de quem o criou, o **PPID** (*Parent PID*). Em C, use `getpid()` e `getppid()`; em Python, `os.getpid()` e `os.getppid()`. No terminal, experimente:

>_ Observando processos no terminal

```
$ echo $$          # PID do shell atual
4312
$ ps -o pid,ppid,stat,cmd
  PID   PPID  STAT  CMD
  4312   4300  Ss    bash
  4390   4312  R+    ps -o pid,ppid,stat,cmd
```

Repare que o PPID do **ps** é justamente o PID do **bash**: foi o shell que criou o processo do **ps**. É exatamente isso que o seu **mysh** vai fazer.

Espaço de endereçamento

Cada processo enxerga sua própria memória, chamada **espaço de endereçamento**, dividida tipicamente em regiões:

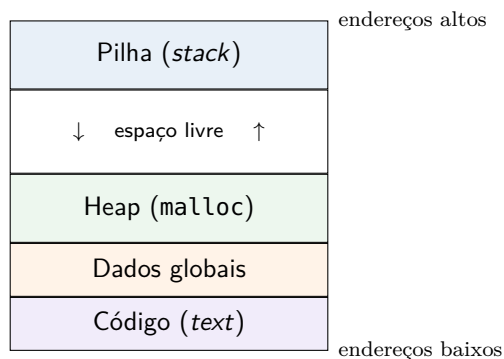


Figura 1.2: Organização típica do espaço de endereçamento de um processo.

O ponto essencial para este trabalho: **processos não compartilham memória**. Se o processo filho alterar uma variável, o pai não verá a alteração. É por isso que o comando **cd** precisa de tratamento especial, como veremos adiante.

O Bloco de Controle de Processo (PCB)

Para gerenciar os processos, o núcleo mantém para cada um deles uma estrutura de dados chamada **PCB** (*Process Control Block*; no Linux, a `struct task_struct`). Ela guarda, entre outras informações:

- PID, PPID, usuário dono do processo;
- estado do processo (executando, pronto, bloqueado, terminado);
- contexto da CPU (registradores, contador de programa) salvo quando o processo sai da CPU;
- informações de memória (onde estão as páginas do processo);
- a **tabela de descritores de arquivo** (arquivos abertos);
- diretório de trabalho corrente, variáveis de ambiente;
- tratamento configurado para cada sinal.

Guarde os três últimos itens: diretório corrente, descritores de arquivo e tratamento de sinais são exatamente os atributos que o seu shell vai manipular.

Estados de um processo

Durante sua vida, um processo passa por diferentes estados:

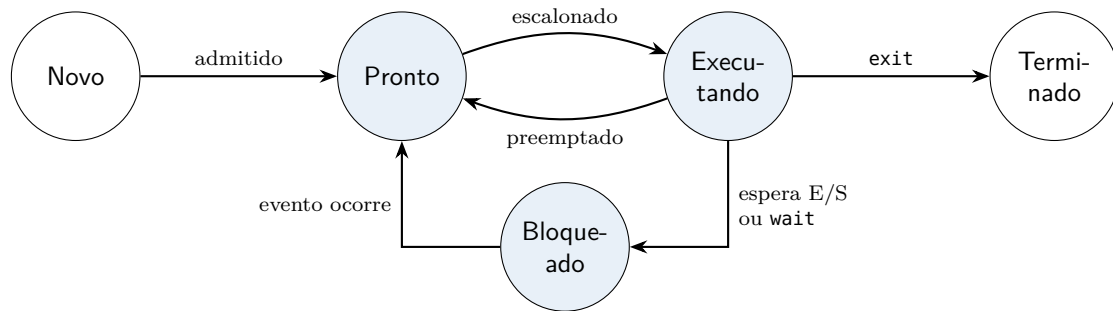
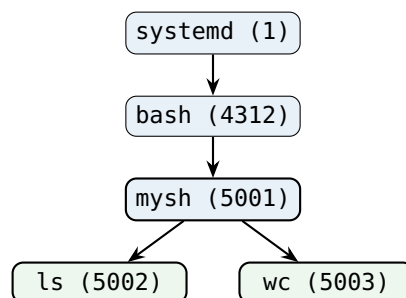


Figura 1.3: Diagrama de estados de um processo.

Quando o shell chama `waitpid` para esperar um filho, ele passa ao estado *bloqueado* e não consome CPU até que o filho termine. Quando um processo lê de um pipe vazio, também fica bloqueado até que haja dados — ou até que o pipe sinalize fim de arquivo.

Árvore de processos

Como todo processo é criado por outro, os processos formam uma **árvore**. No Linux, a raiz é o processo de PID 1 (`systemd` ou `init`), criado pelo núcleo durante a inicialização. O comando `ps tree -p` mostra essa árvore. Quando o seu shell executa `ls | wc -l`, a árvore terá, momentaneamente, o seguinte formato:

Figura 1.4: Árvore de processos durante a execução de `ls | wc -l` dentro do `mysh`.

1.2.3 Criando processos: `fork`, `exec` e `wait`

Sistemas UNIX separam a criação de um processo em duas operações independentes: **duplicar** o processo atual (`fork`) e **substituir** o programa que ele executa (`exec`). Essa separação, que parece estranha à primeira vista, é justamente o que permite ao shell ajustar redirecionamentos e pipes *entre* as duas etapas.

`fork()`: duplicando o processo

Definição: `fork()`

Cria um novo processo (o **filho**) que é uma cópia quase exata do processo que chamou (o **pai**): mesmo código, cópia dos dados, da pilha, do heap, dos descritores de arquivo abertos e do diretório corrente. Ambos continuam executando *a partir da instrução seguinte ao* `fork`.

A pergunta natural é: se os dois processos executam o mesmo código a partir do mesmo ponto, como cada um sabe quem é? Pelo **valor de retorno**:

Retorno de <code>fork()</code>	Significado
<code>0</code>	you are in the filho process
<code>> 0</code> (PID do filho)	you are in the pai process
<code>-1</code> (C) / exceção <code>OSError</code> (Python)	failure: no child was created

💡 Cópia sob demanda (*copy-on-write*)

Copiar toda a memória do pai a cada `fork` seria caro. Por isso o Linux usa *copy-on-write*: pai e filho inicialmente compartilham as mesmas páginas físicas, marcadas como somente leitura; só quando um deles escreve em uma página é que ela é efetivamente copiada. Para o programador, o efeito é o mesmo de uma cópia completa: alterações de um não aparecem no outro.

Família `exec`: trocando de programa

📖 Definição: `exec`

Substitui a imagem (código, dados, pilha, heap) do processo atual por um novo programa lido do disco. O PID **não muda**, e os descritores de arquivo abertos **são mantidos** (salvo os marcados com `0_CLOEXEC`). Se a chamada tiver sucesso, ela **nunca retorna**: o código que a chamou deixou de existir.

Há várias variantes, cujos sufixos indicam como os argumentos são passados: `l` (lista de argumentos), `v` (vetor), `p` (procura o programa nos diretórios da variável `PATH`) e `e` (recebe um ambiente explícito). Para o shell, a mais conveniente é `execvp(nome, argv)`: recebe um vetor de argumentos terminado em `NULL` e busca o executável no `PATH`, do mesmo jeito que o `bash` faz. Em Python, a equivalente é `os.execvp(nome, lista_de_args)`.

⚠️ Atenção

Se `execvp` retornar, é porque **falhou** (por exemplo, o comando não existe). Nesse caso o processo filho ainda está executando o código do *shell*! Se você não o encerrar imediatamente com `_exit(127)` (C) ou `os._exit(127)` (Python), terá dois shells lendo o mesmo teclado — um erro clássico e confuso.

`wait` e `waitpid`: esperando o filho

O pai usa `waitpid(pid, &status, opcoes)` para esperar que um filho específico termine e para recolher seu código de saída. Com `pid = -1`, espera qualquer filho. A opção `WNOHANG` torna a chamada não bloqueante: se nenhum filho terminou, ela retorna `0` imediatamente. O inteiro `status` é decodificado com macros:

Macro (C) / função (Python <code>os.</code>)	Significado
<code>WIFEXITED(s)</code>	o filho terminou normalmente (chamou <code>exit</code>)
<code>WEXITSTATUS(s)</code>	código passado a <code>exit</code> (0 a 255)
<code>WIFSIGNALED(s)</code>	o filho foi morto por um sinal
<code>WTERMSIG(s)</code>	número do sinal que o matou

O padrão fork-exec-wait

Juntando as três chamadas, obtemos o ciclo básico que todo shell executa para cada comando externo:

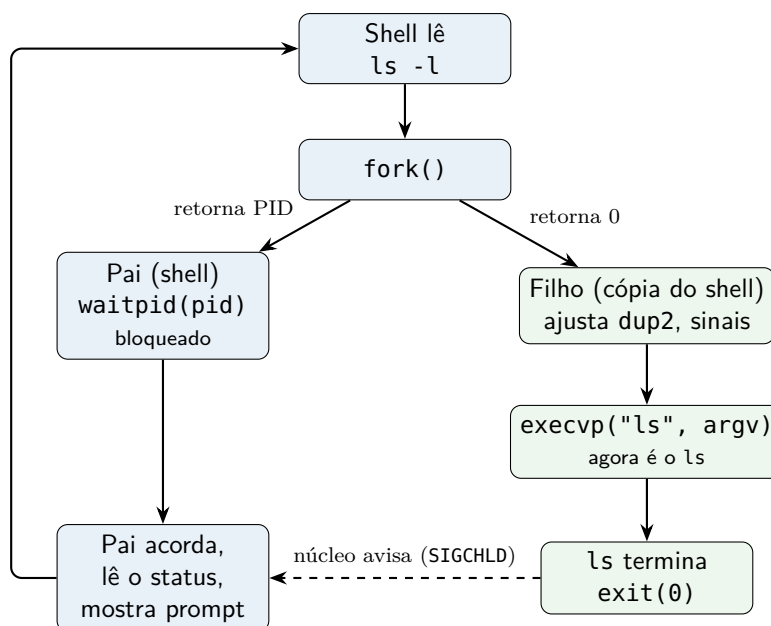


Figura 1.5: O ciclo `fork-exec-wait` executado pelo shell para cada comando.

Observe o bloco “ajusta `dup2`, sinais” no filho: é nessa janela, entre o `fork` e o `exec`, que o shell configura redirecionamentos e pipes. Como o `exec` preserva os descritores abertos, o programa novo (`ls`) herda a configuração sem saber que ela foi feita.

Processos zumbis e órfãos

Definição: Processo zumbi

Um processo que já terminou, mas cujo pai ainda não recolheu seu status com `wait/waitpid`. O núcleo libera sua memória, mas mantém a entrada na tabela de processos (PID e código de saída) até que o pai a recolha. Aparece no `ps` com estado `Z` e a marca `<defunct>`.

Zumbis não consomem CPU nem memória significativa, mas ocupam PIDs; se um shell de longa duração nunca recolher seus filhos em segundo plano, a tabela de processos pode se esgotar. Por isso o seu shell precisa recolher os filhos em *background* — tipicamente com `waitpid(-1, &s, WNOHANG)` em um tratador de `SIGCHLD` ou antes de exibir cada prompt.

Definição: Processo órfão

Um processo cujo pai terminou antes dele. Órfãos são “adotados” pelo processo de PID 1 (ou por um processo *subreaper*), que passa a recolhê-los. Órfãos não são um problema em si.

1.2.4 Descritores de arquivo

No UNIX, “tudo é arquivo”: arquivos em disco, o terminal, pipes, conexões de rede — todos são lidos com `read` e escritos com `write`. Para referenciar um arquivo aberto, o processo usa um número inteiro pequeno, o **descritor de arquivo** (*file descriptor*, `fd`).

Definição: Descritor de arquivo

Um índice na **tabela de descritores** do processo (guardada no PCB). Cada posição aponta para uma estrutura do núcleo que representa um arquivo aberto (com a posição de leitura/escrita atual e o modo de abertura). A chamada **open** devolve sempre o **menor** índice livre.

Por convenção, todo processo nasce com três descritores já abertos:

fd	Nome	Constante C	Uso padrão
0	entrada padrão	STDIN_FILENO	teclado
1	saída padrão	STDOUT_FILENO	tela
2	saída de erro	STDERR_FILENO	tela

Programas como **ls**, **sort** ou **grep** simplesmente leem do fd 0 e escrevem no fd 1. Eles **não sabem** se esses descritores apontam para o terminal, para um arquivo ou para um pipe. Essa é a ideia-chave que torna redirecionamento e pipes possíveis.

As chamadas **open**, **close** e **dup2**

- **open(caminho, flags, modo)** abre (ou cria) um arquivo e devolve um novo descritor. Flags úteis: **O_RDONLY** (só leitura), **O_WRONLY** (só escrita), **O_CREAT** (cria se não existir), **O_TRUNC** (apaga o conteúdo anterior), **O_APPEND** (escreve no final). O **modo** (ex.: **0644**) define as permissões do arquivo criado.
- **close(fd)** libera a posição **fd** da tabela.
- **dup2(antigo, novo)** faz a posição **novo** passar a apontar para o mesmo arquivo aberto que **antigo** (fechando antes o que estivesse em **novo**). Depois disso, os dois números referenciam o mesmo arquivo.

Redirecionamento passo a passo

Veja o que acontece na tabela de descritores do processo filho quando o shell executa **ls > saida.txt**:

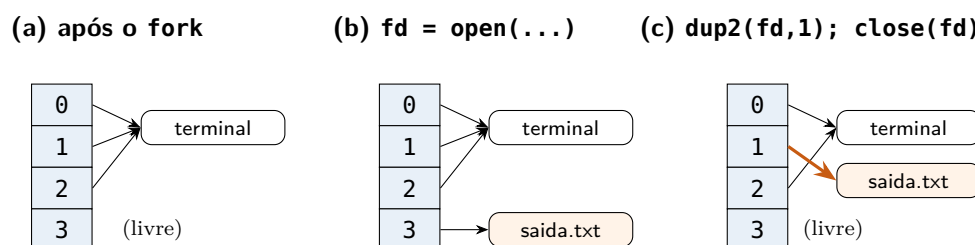


Figura 1.6: Redirecionamento da saída padrão: após o **dup2**, tudo que o **ls** escrever no fd 1 vai para o arquivo.

Depois do passo (c), o filho chama **execvp("ls", ...)**. O **ls** escreve no fd 1, como sempre — mas agora o fd 1 aponta para **saida.txt**.

Sintaxe	Descritor alterado	Flags do <code>open</code>
<code>cmd < arq</code>	0 (entrada)	<code>O_RDONLY</code>
<code>cmd > arq</code>	1 (saída)	<code>O_WRONLY</code> <code>O_CREAT</code> <code>O_TRUNC</code>
<code>cmd >> arq</code>	1 (saída)	<code>O_WRONLY</code> <code>O_CREAT</code> <code>O_APPEND</code>

⚠ Atenção

O redirecionamento deve ser feito **no filho**, depois do `fork` e antes do `exec`. Se você fizer `dup2` no processo do shell, o próprio shell perderá o terminal e o prompt passará a ser escrito no arquivo!

1.2.5 Pipes: comunicação entre processos

📖 Definição: Pipe

Um canal de comunicação **unidirecional** mantido pelo núcleo, com um buffer interno (tipicamente 64 KiB no Linux). A chamada `pipe(fd)` cria o canal e devolve dois descritores: `fd[0]`, a ponta de **leitura**, e `fd[1]`, a ponta de **escrita**. Os bytes escritos em `fd[1]` saem, na mesma ordem (FIFO), em `fd[0]`.

Um pipe só é útil entre processos porque os descritores são **herdados no fork**: o shell cria o pipe, faz `fork` duas vezes, e cada filho fica com uma cópia das duas pontas. Um deles liga a ponta de escrita ao seu `fd 1` e o outro liga a ponta de leitura ao seu `fd 0`.

Regras de bloqueio e de fim de arquivo

O comportamento de um pipe segue regras precisas, que explicam a maioria dos “travamentos” em shells de alunos:

1. `read` em um pipe **vazio** bloqueia o leitor até que chegue algum dado...
2. ... **exceto** se *todas* as pontas de escrita do pipe, em *todos* os processos, estiverem fechadas. Nesse caso `read` retorna 0, que significa **fim de arquivo** (EOF).
3. `write` em um pipe **cheio** bloqueia o escritor até que o leitor consuma dados.
4. `write` em um pipe cujas pontas de leitura foram todas fechadas gera o sinal `SIGPIPE` (que, por padrão, mata o escritor).

💡 Por que fechar as pontas não usadas?

Programas como `sort` e `wc` só produzem o resultado quando recebem EOF na entrada. Pela regra 2, o EOF só chega quando **nenhum** processo mantém a ponta de escrita aberta. Se o próprio shell (ou o processo `wc`) esquecer uma cópia de `fd[1]` aberta, o `wc` esperará para sempre por dados que nunca virão — e o shell ficará bloqueado no `waitpid`. Resultado: o terminal “trava”. **Regra de ouro:** cada processo fecha todas as pontas de pipe que não usa, e o shell fecha todas depois de criar os filhos.

Pipeline com três comandos

Para `cat f | sort | uniq`, são necessários **dois** pipes (em geral, N comandos exigem $N - 1$ pipes). O primeiro comando escreve no pipe A; o segundo lê de A e escreve em B; o terceiro lê de B.

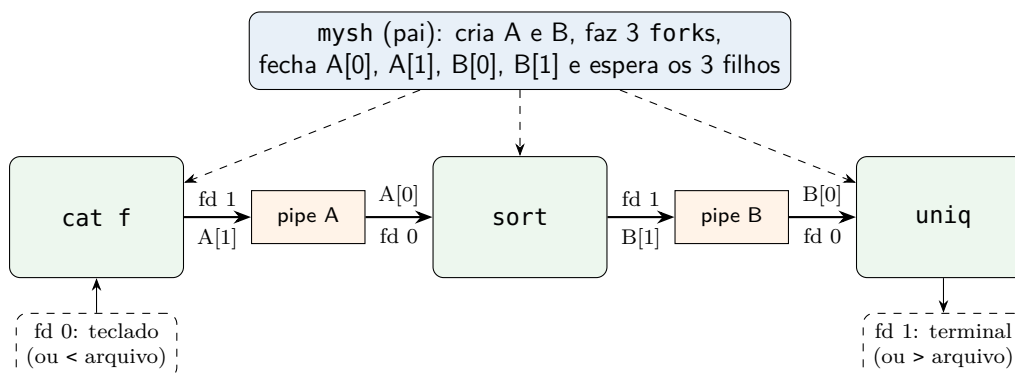


Figura 1.7: Pipeline de três comandos. Cada filho usa só as pontas indicadas e fecha as demais.

A tabela abaixo resume, para um pipeline de N comandos numerados de 0 a $N - 1$ e pipes numerados de 0 a $N - 2$, o que cada filho i faz:

Filho	Entrada padrão (fd 0)	Saída padrão (fd 1)
$i = 0$	original (ou < arquivo)	escrita do pipe 0
$0 < i < N - 1$	leitura do pipe $i - 1$	escrita do pipe i
$i = N - 1$	leitura do pipe $N - 2$	original (ou >/>> arquivo)

1.2.6 Sinais

Definição: Sinal

Uma notificação assíncrona enviada a um processo pelo núcleo (ou por outro processo, via `kill`) para avisar que um evento ocorreu. Cada sinal tem um número e um nome. Ao receber um sinal, o processo executa uma **ação**: a ação padrão (`SIG_DFL`), ignorá-lo (`SIG_IGN`) ou executar uma função tratadora (*handler*) definida pelo programa.

Sinal	Quando ocorre	Ação padrão	Uso no mini-shell
<code>SIGINT</code>	usuário pressiona <code>Ctrl+C</code>	terminar	shell ignora; filho morre
<code>SIGCHLD</code>	um filho terminou ou parou	ignorar	recolher filhos em background
<code>SIGPIPE</code>	escrita em pipe sem leitor	terminar	(informativo)
<code>SIGTERM</code>	pedido de término (<code>kill</code>)	terminar	—
<code>SIGKILL</code>	término forçado (<code>kill -9</code>)	terminar	não pode ser tratado
<code>SIGTSTP</code>	usuário pressiona <code>Ctrl+Z</code>	parar	opcional (controle de jobs)

Quem recebe o `Ctrl+C`?

Quando você pressiona `Ctrl+C`, o driver do terminal envia `SIGINT` para **todos os processos do grupo em primeiro plano** do terminal — o que, em um mini-shell simples, inclui tanto o shell

quanto o filho que está executando. Por isso a estratégia é:

1. **No shell:** ignorar `SIGINT` (ou instalar um tratador que apenas imprime uma nova linha). Assim o shell não morre.
2. **No filho,** logo após o `fork` e antes do `exec`: restaurar a ação padrão (`SIG_DFL`). Assim o comando em execução é interrompido normalmente.

💡 Sinais ignorados sobrevivem ao `exec`

O `exec` restaura para o padrão os sinais que tinham *tratador*, mas **mantém ignorados** os sinais que estavam com `SIG_IGN`. Se o shell ignora `SIGINT` e o filho não restaurar `SIG_DFL` antes do `exec`, o `sleep 100` também ignorará o `Ctrl+C`!

Tratadores de sinal: `signal` e `sigaction`

Em C, a função portátil e recomendada é `sigaction`, que permite configurar flags como `SA_RESTART` (reinicia automaticamente chamadas como `read` interrompidas pelo sinal). A antiga `signal` funciona, mas seu comportamento varia entre sistemas. Em Python, usa-se `signal.signal(sinal, acao)`.

⚠️ Atenção

Um tratador de sinal pode interromper o programa em *qualquer* ponto. Por isso, dentro dele, use apenas funções *async-signal-safe* (por exemplo, `waitpid` e `write`; **não** `printf` nem `malloc`), e mantenha o tratador curto. Consulte `man 7 signal-safety`. Salve e restaure `errno` no tratador de `SIGCHLD`.

1.2.7 Comandos internos (*builtins*)

Alguns comandos **não podem** ser programas externos, porque precisam alterar o estado do *próprio processo do shell*. O exemplo clássico é o `cd`.

💡 Por que `cd` não pode rodar em um filho?

O diretório corrente é um atributo do processo, guardado no seu PCB. Se o shell fizesse `fork` e o filho chamasse `chdir("/tmp")`, apenas o diretório do **filho** mudaria; o filho terminaria logo em seguida e o shell continuaria exatamente onde estava. Pelo mesmo motivo, `exit` (que deve encerrar o *shell*) e `export` (que deve alterar o ambiente herdado pelos *próximos* comandos) precisam ser executados pelo próprio shell.

Builtin	Chamada usada	Observação
<code>cd [dir]</code>	<code>chdir</code>	sem argumento, ir para <code>\$HOME</code>
<code>pwd</code>	<code>getcwd</code>	pode ser externo, mas é trivial internamente
<code>exit [n]</code>	—	encerra o laço principal
<code>export NOME=valor</code>	<code>setenv</code> / <code>os.environ</code>	afeta filhos futuros

1.2.8 Análise da linha de comando

Antes de executar algo, o shell precisa entender o que foi digitado. Esse processo tem duas etapas, inspiradas em compiladores:

1. **Análise léxica (tokenização)**: dividir a linha em *tokens*. Espaços separam palavras; |, <, >, >> e & são operadores; texto entre aspas forma um único token (sem aspas).
2. **Análise sintática (parsing)**: percorrer os tokens e montar uma estrutura: uma lista de comandos simples separados por |, cada um com seus argumentos e redirecionamentos, mais a indicação de segundo plano.

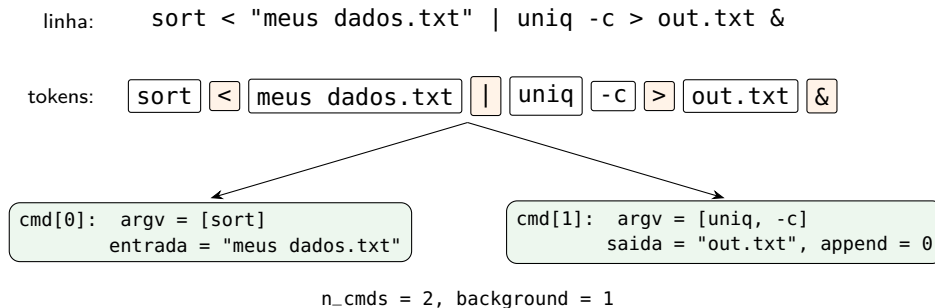


Figura 1.8: Da linha digitada à estrutura de comandos.

O parser também deve detectar **erros de sintaxe**, como `ls |` (pipe sem comando depois), `| wc` (pipe sem comando antes), `cat <` (redirecionamento sem arquivo) ou `&` fora do final da linha, e reportá-los sem executar nada.

1.2.9 Resumo das chamadas de sistema do trabalho

C	Python (os.)	Função no mini-shell
<code>fork()</code>	<code>fork()</code>	criar um processo para cada comando externo
<code>execvp(f, argv)</code>	<code>execvp(f, args)</code>	substituir o filho pelo programa pedido
<code>waitpid(p, &s, o)</code>	<code>waitpid(p, o)</code>	aguardar o filho em primeiro plano; recolher zumbis
<code>pipe(fd)</code>	<code>pipe()</code>	criar o canal entre dois comandos
<code>dup2(a, b)</code>	<code>dup2(a, b)</code>	ligar arquivo/pipe à entrada ou saída padrão
<code>open(p, fl, m)</code>	<code>open(p, fl, m)</code>	abrir arquivos de redirecionamento
<code>close(fd)</code>	<code>close(fd)</code>	fechar descritores intermediários e pontas de pipe
<code>chdir(p)</code>	<code>chdir(p)</code>	builtin <code>cd</code>
<code>getcwd(b, n)</code>	<code>getcwd()</code>	builtin <code>pwd</code> e prompt
<code>sigaction(...)</code>	<code>signal.signal(...)</code>	tratar <code>SIGINT</code> e <code>SIGCHLD</code>
<code>_exit(n)</code>	<code>_exit(n)</code>	encerrar o filho se o <code>exec</code> falhar

Dica

Em Python, existe o módulo `subprocess`, que esconde todas essas chamadas. **Ele não pode ser usado neste trabalho**: o objetivo é justamente exercitar `fork`, `exec`, `pipe` e `dup2`. Use apenas as funções de baixo nível do módulo `os` (e `signal`). O módulo `shlex` pode ajudar na tokenização, mas você deve entender e documentar o que ele faz.

1.3 Especificação do trabalho

O programa deve implementar um laço REPL (*read-eval-print loop*) que:

1. exibe um prompt (ex.: `mysh>`);

2. lê uma linha de comando;
3. faz o *parsing* da linha (tokenização respeitando aspas);
4. executa o comando conforme as regras abaixo;
5. retorna ao passo 1 até receber **exit** (ou fim de entrada, **Ctrl+D**).

1.3.1 Requisitos funcionais

Funcionalidade	Exemplo	Descrição
Comandos simples	<code>ls -la</code>	executar via <code>fork + execvp</code>
Comandos internos	<code>cd, exit, pwd, export</code>	não devem criar processo filho
Redirecionamento de saída	<code>ls > out.txt</code>	truncar o arquivo
Redirecionamento com <i>append</i>	<code>ls >> out.txt</code>	anexar ao final
Redirecionamento de entrada	<code>sort < dados.txt</code>	ler de arquivo
Pipe simples	<code>ls -l grep .c</code>	um pipe
Pipes múltiplos (desejável)	<code>cat f sort uniq wc -l</code>	<i>N</i> pipes encadeados
Execução em <i>background</i>	<code>sleep 10 &</code>	não bloquear o shell; reportar o PID
Tratamento de Ctrl+C	—	não matar o shell, apenas o filho ativo
Histórico simples (desejável)	<code>!!</code>	repetir o último comando

1.3.2 Requisitos não funcionais

- Tratar erros de `fork/exec/open/pipe` sem derrubar o shell.
- Evitar processos zumbis (usar `waitpid` com `WNOHANG` para comandos em *background* e/ou tratamento de `SIGCHLD`).
- Limites razoáveis de argumentos (ex.: 64) e de tamanho de linha (ex.: 1024 bytes), configuráveis por constantes (`#define` em C, constantes no topo do módulo em Python).
- Rodar em Linux (nativo ou WSL); compilar sem avisos com `-Wall -Wextra` (C).

1.3.3 Estrutura de entrega

Estrutura sugerida — implementação em C

```
mini-shell/
|-- src/
|   |-- main.c
|   |-- parser.c   / parser.h
|   |-- executor.c / executor.h
|   '-- builtins.c / builtins.h
|-- Makefile
|-- testes/
|   '-- casos_de_teste.txt
'-- RELATORIO.pdf
```

Estrutura sugerida — implementação em Python

```
mini-shell/
```

```

|-- mysh/
|   |-- __main__.py      # laço principal (python3 -m mysh)
|   |-- parser.py
|   |-- executor.py
|   |-- builtins.py
|-- Makefile             # alvos: run, test, clean
|-- testes/
|   |-- casos_de_teste.txt
|-- RELATORIO.pdf

```

1.4 Roteiro de implementação

O desenvolvimento deve ser **incremental**: cada passo acrescenta uma funcionalidade e termina com testes. Não passe para o passo seguinte enquanto os testes do passo atual não funcionarem — depurar parsing, pipes e sinais ao mesmo tempo é a receita para perder dias.

Módulo	Responsabilidade
main	ponto de entrada; laço principal (REPL); instalação dos tratadores de sinal
parser	transforma a linha de texto em uma estrutura de comandos
executor	cria processos, conecta pipes, aplica redirecionamentos, espera filhos
builtins	comandos internos que não criam processo filho

1.4.1 Passo 0 — Preparação do ambiente

1. Garanta um ambiente Linux (nativo, máquina virtual ou WSL). Em macOS e Windows nativo várias chamadas se comportam de forma diferente.
2. C: instale **gcc**, **make**, **gdb** e **valgrind**. Python: use Python 3.8 ou superior (nenhuma biblioteca externa é necessária).
3. Crie a estrutura de diretórios e um **Makefile** mínimo que já compile um **main** vazio. Em Python, o **Makefile** pode ter apenas alvos como **run** (**python3 -m mysh**) e **clean**.
4. Leia as páginas de manual: **man 2 fork**, **man 3 exec**, **man 2 waitpid**, **man 2 pipe**, **man 2 dup2**, **man 2 open**, **man 2 sigaction**.

</> C | esqueleto de Makefile

```

CC      = gcc
CFLAGS  = -Wall -Wextra -g -std=c11 -D_POSIX_C_SOURCE=200809L
OBJS    = src/main.o src/parser.o src/executor.o src/builtins.o

mysh: $(OBJS)
    # TODO: regra de ligação

clean:
    # TODO: remover objetos e executável

```

1.4.2 Passo 1 — Estrutura de dados do comando

O que fazer. Definir como uma linha digitada será representada internamente. São necessárias duas estruturas:

- **Comando simples:** vetor de argumentos (`argv[0]` é o nome do programa), número de argumentos, arquivo de entrada (`<`), arquivo de saída (`>` ou `>>`) e um indicador de *append*.
- **Linha completa:** sequência de comandos simples (um por trecho separado por `|`), quantidade de comandos e indicador de *background* (`&`).

Por que primeiro. Parser e executor conversam por meio dessas estruturas. Definir bem a interface agora evita reescrever os dois depois.

C | parser.h — estruturas (a completar)

```
#define MAX_ARGS 64
#define MAX_CMDS 16
#define MAX_LINE 1024

typedef struct {
    char *argv[MAX_ARGS + 1]; /* +1: execvp exige argv terminado em NULL */
    int argc;
    char *arq_entrada;         /* NULL se não houver '<' */
    char *arq_saida;           /* NULL se não houver '>' ou '>>' */
    int append;                /* 1 para '>>', 0 para '>' */
} Comando;

typedef struct {
    /* TODO: vetor de Comando, número de comandos, flag de background */
} LinhaComando;
```

Python | parser.py — estruturas (a completar)

```
from dataclasses import dataclass, field
from typing import List, Optional

@dataclass
class Comando:
    argv: List[str] = field(default_factory=list)
    arq_entrada: Optional[str] = None
    arq_saida: Optional[str] = None
    append: bool = False

@dataclass
class LinhaComando:
    # TODO: lista de Comando e flag de background
    ...
```

Verifique antes de seguir

- O `argv` tem espaço para o `NULL` final (C)?
- Você consegue representar `sort < a.txt > b.txt, ls | wc -l e sleep 5 &` com essas estruturas? Escreva-as à mão no papel.

1.4.3 Passo 2 — Tokenização e *parsing* da linha

O que fazer. Implementar duas funções no módulo `parser`:

`tokenizar_linha` recebe a linha crua e devolve a lista de tokens. Percorra a linha caractere a

caractere, como uma pequena máquina de estados: fora de aspas, espaço encerra o token atual; " ou ' alterna o estado “dentro de aspas” (e não entra no token); |, <, & e > são operadores e viram tokens próprios — atenção a >>, que é *um* operador de dois caracteres.

interpretar_tokens percorre os tokens e preenche a **LinhaComando**: palavra comum vira argumento do comando atual; | inicia novo comando; <, > e >> consomem o *próximo* token como nome de arquivo; & só é válido como último token.

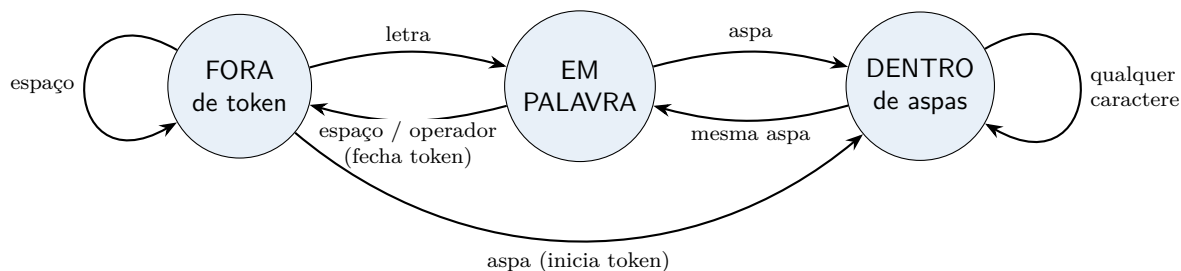


Figura 1.9: Máquina de estados sugerida para o tokenizador.

`</>` C | parser.c — tokenizador (esqueleto)

```

/* Devolve o número de tokens ou -1 em erro (ex.: aspas não fechadas).
 * tokens[i] deve apontar para strings alocadas (strdup) ou para um buffer
 * próprio -- documente sua escolha e libere a memória depois. */
int tokenizar_linha(const char *linha, char *tokens[], int max_tokens) {
    enum { FORA, PALAVRA, ASPAS } estado = FORA;
    char aspa = 0;
    char buf[MAX_LINE];
    int n = 0, len = 0;

    for (const char *p = linha; *p != '\0'; p++) {
        switch (estado) {
            case FORA:
                /* TODO: pular espaços; operador -> token próprio;
                 * aspa -> ASPAS; outro caractere -> PALAVRA */
                break;
            case PALAVRA: /* TODO */ break;
            case ASPAS: /* TODO */ break;
        }
    }
    /* TODO: fechar o último token; aspas abertas no fim = erro */
    return n;
}

```

`Python` | parser.py — interpretação dos tokens (esqueleto)

```

OPERADORES = {"|", "<", ">", ">>", "&"}

def interpretar_tokens(tokens):
    """Converte a lista de tokens em LinhaComando.
    Lança ValueError com mensagem clara em caso de erro de sintaxe."""
    linha = LinhaComando(...)
    atual = Comando()
    i = 0
    while i < len(tokens):

```

```

t = tokens[i]
if t == "|":
    # TODO: comando atual vazio -> erro "próximo a '|'"
    ...
elif t in ("<", ">", ">>"):
    # TODO: exigir um próximo token que não seja operador
    ...
elif t == "&":
    # TODO: válido apenas como último token
    ...
else:
    atual.argv.append(t)
i += 1
# TODO: validar e anexar o último comando
return linha

```

Dica

Em Python, `shlex.split(linha)` já trata aspas, mas **não** separa operadores colados (`ls>out` vira um único token). Se usá-lo, pesquise a opção `punctuation_chars` da classe `shlex.shlex`, e explique no relatório como ela funciona. Em C, `strtok` é tentador, mas não lida com aspas; a máquina de estados é mais robusta.

Verifique antes de seguir

Escreva uma função de depuração que imprime a estrutura resultante e teste, *antes de executar qualquer comando*:

- `ls -la` → 1 comando, `argv = [ls, -la]`;
- `echo ola mundo` → `argv = [echo, ola, mundo]`;
- `echo "ola mundo"` → `argv = [echo, ola mundo]`;
- `sort < a.txt > b.txt` e `ls >> log.txt`;
- `cat f | sort | uniq | wc -l` → 4 comandos;
- erros: `ls |, | wc, cat <, echo "abc, & ls`.

1.4.4 Passo 3 — Laço principal (REPL) sem executar nada

O que fazer. Na função principal: exibir o prompt, ler uma linha, chamar o parser e, por enquanto, apenas imprimir a estrutura resultante.

Casos que já devem ser tratados:

- **linha vazia** (só Enter ou só espaços): reexibir o prompt;
- **fim de entrada** (Ctrl+D): `fgets` retorna NULL / `input()` lança `EEOFError`; encerrar o shell de forma limpa;
- **erro de sintaxe**: imprimir a mensagem e voltar ao prompt.

C | main.c — laço principal (esqueleto)

```

int main(void) {
    char linha[MAX_LINE];
    /* TODO (Passo 8): instalar tratadores de sinal aqui */
}

```

```

for (;;) {
    printf("mysh> ");
    fflush(stdout);           /* o prompt não termina com \n! */
    if (fgets(linha, sizeof linha, stdin) == NULL) {
        /* TODO: EOF (Ctrl+D) ou erro -> sair do laço */
    }
    /* TODO: remover o '\n' final; ignorar linha vazia;
     *      parser; imprimir estrutura (por enquanto) */
}
return 0;
}

```

Python | __main__.py — laço principal (esqueleto)

```

def main():
    while True:
        try:
            linha = input("mysh> ")
        except EOFError:
            # TODO: Ctrl+D -> imprimir nova linha e sair
            ...
        if not linha.strip():
            continue
        # TODO: tokenizar, interpretar (tratando ValueError) e,
        #      por enquanto, apenas imprimir a estrutura

```

⚠ Atenção

Em C, o `printf` usa um buffer que só é esvaziado ao encontrar `\n`. Como o prompt não termina com quebra de linha, chame `fflush(stdout)` após imprimi-lo. Além disso, **antes de todo fork**, esvazie o buffer de `stdout`: senão o texto pendente é duplicado no filho e aparece duas vezes.

✓ Verifique antes de seguir

O shell mostra o prompt, imprime a estrutura de cada linha digitada, ignora linhas vazias, reporta erros de sintaxe e termina com `Ctrl+D`.

1.4.5 Passo 4 — Execução de comando simples

O que fazer. Criar `executar_comando_simples` no módulo `executor`, aplicando o padrão `fork-exec-wait`:

1. chamar `fork`; em caso de falha, reportar e retornar ao prompt;
2. **no filho:** chamar `execvp(argv[0], argv)`; se retornar, imprimir a mensagem “mysh: <cmd>: comando não encontrado” (na saída de erro) e terminar com `_exit(127)`;
3. **no pai:** chamar `waitpid` para o PID do filho e guardar o código de saída (útil para depuração e para um eventual \$?).

 C | executor.c — estrutura do fork (fragmento)

```

pid_t pid = fork();
if (pid < 0) {
    perror("mysh: fork");
    return -1;
}
if (pid == 0) {
    /* ---- FILHO ---- */
    /* TODO (Passo 8): restaurar SIGINT para SIG_DFL */
    /* TODO (Passo 6): aplicar redirecionamentos */
    execvp(cmd->argv[0], cmd->argv);
    /* só chega aqui se o execvp falhou */
    /* TODO: mensagem de erro em stderr; _exit(127) */
}
/* ---- PAI ---- */
int status;
/* TODO: waitpid(pid, &status, 0) tratando EINTR;
 *      decodificar status com WIFEXITED / WEXITSTATUS */

```

 Python | executor.py — estrutura do fork (fragmento)

```

import os, sys

def executar_comando_simples(cmd):
    try:
        pid = os.fork()
    except OSError as e:
        print(f"mysh: fork: {e.strerror}", file=sys.stderr)
        return -1
    if pid == 0:
        # ---- FILHO ----
        try:
            # TODO (Passos 6 e 8): redirecionamentos e sinais
            os.execvp(cmd.argv[0], cmd.argv)
        except FileNotFoundError:
            # TODO: mensagem "comando não encontrado" em stderr
            ...
        finally:
            os._exit(127) # NUNCA deixe o filho voltar ao laço do shell
    # ---- PAI ----
    # TODO: os.waitpid(pid, 0) e decodificar com os.WIFEXITED / os.WEXITSTATUS

```

 Por que `_exit` e não `exit` no filho?

`exit()` (C) e `sys.exit()` (Python) executam rotinas de finalização do processo, como esvaziar buffers de `stdio` e, em Python, blocos `finally` e `atexit`. No filho que falhou no `exec`, esses buffers são cópias dos do pai, e esvaziá-los pode duplicar saídas. `_exit` encerra o processo imediatamente.

 Verifique antes de seguir

- `ls`, `ls -l /tmp`, `echo teste`, `date` funcionam.
- `xyzabc` mostra erro e o shell continua funcionando (digite `exit` uma única vez e confirme que o shell termina — se precisar de dois `exit`, o filho não terminou após a falha do `exec`).

- Em outro terminal, `ps -ef | grep mysh` não mostra filhos `<defunct>` após os comandos terminarem.

1.4.6 Passo 5 — Comandos internos (*builtins*)

O que fazer. No módulo `builtins`:

- `eh_builtin(nome)`: indica se o comando é interno;
- `builtin_cd`: `chdir` para o argumento (ou `$HOME` sem argumento), reportando erro se o diretório não existir;
- `builtin_pwd`: imprime o resultado de `getcwd`;
- `builtin_exit`: sinaliza ao laço principal que deve terminar (aceitando opcionalmente um código numérico);
- `builtin_export` (opcional): `NOME=valor` → `setenv` (C) ou `os.environ[...]` (Python).

No executor (ou no laço principal), verifique `eh_builtin` antes de qualquer `fork`.

C | `builtins.c` — tabela de despacho (esqueleto)

```
typedef int (*FuncBuiltin)(Comando *cmd);

static int builtin_cd(Comando *cmd);
static int builtin_pwd(Comando *cmd);
/* TODO: exit, export */

static const struct { const char *nome; FuncBuiltin f; } BUILTINS[] = {
    { "cd", builtin_cd },
    { "pwd", builtin_pwd },
    /* TODO */
};

static int builtin_cd(Comando *cmd) {
    const char *destino = /* TODO: argv[1] ou getenv("HOME") */ NULL;
    /* TODO: chdir(destino); se falhar, perror com o nome do comando */
    return 0;
}
```

Python | `builtins.py` — tabela de despacho (esqueleto)

```
import os

def builtin_cd(cmd):
    destino = ... # TODO: argv[1] se existir, senão os.environ["HOME"]
    try:
        os.chdir(destino)
    except OSError as e:
        ... # TODO: mensagem "mysh: cd: <dir>: <motivo>"

def builtin_pwd(cmd):
    ... # TODO: os.getcwd()

BUILTINS = {"cd": builtin_cd, "pwd": builtin_pwd} # TODO: exit, export
```

✓ Verifique antes de seguir

- `cd /tmp` seguido de `pwd` imprime `/tmp`; `cd sozinho` volta ao `$HOME`; `cd /nao/existe` mostra erro.
- `export MINHA=42` seguido de `printenv MINHA` imprime `42`.
- Experimento: execute `cd` temporariamente *via fork* e observe que o diretório do shell não muda. Registre isso no relatório.

1.4.7 Passo 6 — Redirecionamento de entrada e saída

O que fazer. Criar `aplicar_redirecionamentos(cmd)`, chamada **no filho**, depois do `fork` e antes do `exec`:

1. se houver arquivo de entrada: abrir com `O_RDONLY`, `dup2` para o fd 0, fechar o descritor original;
2. se houver arquivo de saída: abrir com `O_WRONLY|O_CREAT` mais `O_TRUNC` ou `O_APPEND` (conforme o indicador), permissões `0644`, `dup2` para o fd 1 e fechar o original;
3. se o `open` falhar (ex.: arquivo de entrada inexistente), imprimir erro e terminar o filho com `_exit(1)` — sem executar o comando.

✎ C | executor.c — redirecionamento (fragmento)

```
static void aplicar_redirecionamentos(const Comando *cmd) {
    if (cmd->arq_entrada != NULL) {
        int fd = open(cmd->arq_entrada, O_RDONLY);
        if (fd < 0) { perror(cmd->arq_entrada); _exit(1); }
        /* TODO: dup2 para STDIN_FILENO e close(fd) */
    }
    if (cmd->arq_saida != NULL) {
        int flags = O_WRONLY | O_CREAT /* | TODO: O_TRUNC ou O_APPEND */;
        /* TODO: open com modo 0644, tratar erro, dup2, close */
    }
}
```

🐍 Python | executor.py — redirecionamento (fragmento)

```
def aplicar_redirecionamentos(cmd):
    if cmd.arq_entrada is not None:
        fd = os.open(cmd.arq_entrada, os.O_RDONLY) # pode lançar OSError
        # TODO: os.dup2(fd, 0) e os.close(fd)
    if cmd.arq_saida is not None:
        flags = os.O_WRONLY | os.O_CREAT
        # TODO: acrescentar os.O_APPEND ou os.O_TRUNC; os.open(..., 0o644)
        ...
```

⚠ Atenção

Em Python, `print` escreve em `sys.stdout`, um objeto com buffer próprio que não percebe o `dup2`. Isso não afeta o programa executado pelo `exec`, mas afeta seus *builtins*: se quiser suportar `pwd > arq`, escreva com `os.write(1, ...)` ou salve/restaure o fd 1 com `os.dup` — e explique a solução no relatório.

✓ Verifique antes de seguir

- `echo ola > saida.txt; cat saida.txt` → `ola`.
- `echo mais >> saida.txt; cat < saida.txt` → duas linhas.
- `echo novo > saida.txt` trunca o arquivo (sobra uma linha).
- `sort < inexistente.txt` mostra erro e o shell continua.
- `ls -l /proc/<pid>/fd` (em outro terminal, durante um `sleep 30 > x.txt`) mostra o fd 1 apontando para `x.txt` e nenhum descritor extra aberto.

1.4.8 Passo 7 — Pipes (um ou múltiplos)

O que fazer. Criar `executar_linha_com_pipes(linha)`:

1. com um único comando, delegar para o Passo 4 (ou para o builtin);
2. com $N > 1$ comandos, criar os $N - 1$ pipes **antes** de criar os filhos;
3. para cada comando i , fazer `fork`; no filho: se $i > 0$, ligar o fd 0 à leitura do pipe $i - 1$; se $i < N - 1$, ligar o fd 1 à escrita do pipe i ; **fechar todas as pontas de todos os pipes**; aplicar redirecionamentos (`<` só faz sentido no primeiro, `>` no último); restaurar sinais; `exec`;
4. no pai: fechar todas as pontas de todos os pipes e esperar todos os filhos (guardando os PIDs em um vetor).

✎ C | executor.c — pipeline (esqueleto, sem a lógica central)

```
int pipes[MAX_CMDS - 1][2];
pid_t pids[MAX_CMDS];
int n = linha->n_cmds;

for (int i = 0; i < n - 1; i++) {
    if (pipe(pipes[i]) < 0) { /* TODO: tratar erro e limpar */ }
}
for (int i = 0; i < n; i++) {
    pids[i] = fork();
    if (pids[i] == 0) {
        /* TODO: se i > 0      -> dup2(leitura do pipe i-1, 0)      */
        /* TODO: se i < n - 1 -> dup2(escrita do pipe i, 1)        */
        /* TODO: fechar TODAS as pontas de TODOS os pipes        */
        /* TODO: redirecionamentos, sinais, execvp, _exit(127)    */
    }
}
/* TODO: pai fecha todas as pontas; waitpid para cada pids[i] */
```

🐍 Python | executor.py — pipeline (esqueleto, sem a lógica central)

```
def executar_pipeline(linha):
    n = len(linha.comandos)
    pipes = [os.pipe() for _ in range(n - 1)] # cada item: (leitura, escrita)
    pids = []
    for i, cmd in enumerate(linha.comandos):
        pid = os.fork()
        if pid == 0:
            # TODO: ligar fd 0 / fd 1 aos pipes vizinhos com os.dup2
```

```

    # TODO: fechar todas as pontas de todos os pipes
    # TODO: redirecionamentos, sinais, execvp, os._exit(127)
    ...
    pids.append(pid)
    # TODO: pai fecha todas as pontas e espera cada pid

```

Dica

Uma alternativa que economiza descritores é criar cada pipe *dentro* do laço, guardando apenas a ponta de leitura do pipe anterior. As duas abordagens são aceitas; descreva no relatório a que você escolheu e por quê.

Atenção

Sintoma clássico: `ls | wc -l` imprime o resultado, mas o shell não volta ao prompt; ou `cat arq | sort` nunca imprime nada. Quase sempre é uma ponta de escrita que ficou aberta (no pai ou em algum filho), impedindo o EOF. Confira com `ls -l /proc/<pid>/fd` em outro terminal.

Verifique antes de seguir

Teste em ordem crescente de complexidade:

- `ls | grep .c` e `ls -l | wc -l`;
- `cat arq | sort | uniq`;
- `cat /etc/passwd | cut -d: -f1 | sort | head -3`;
- `cat entrada.txt | sort > ordenado.txt` (pipe + redirecionamento);
- `sort < entrada.txt | uniq -c`;
- `nao_existe | wc -l` (erro no primeiro estágio; shell não trava).

1.4.9 Passo 8 — Execução em *background* e tratamento de sinais

O que fazer.

1. **Background:** se a linha terminar com `&`, criar os processos normalmente, imprimir `[job]` PID e voltar ao prompt **sem** `waitpid` bloqueante.
2. **Recolher zumbis:** instalar um tratador de `SIGCHLD` que, em laço, chame a função `waitpid(-1, &st, WNOHANG)` até ela retornar 0 ou -1 (vários filhos podem terminar e gerar um único sinal). Alternativa aceita: recolher os filhos em `background` com `WNOHANG` antes de exibir cada prompt.
3. **Ctrl+C:** o shell ignora `SIGINT` (ou imprime apenas uma nova linha); cada filho restaura `SIG_DFL` antes do `exec`.
4. Instalar os tratadores **uma única vez**, no início de `main`.

C | main.c — instalação de tratadores (fragmento)

```

static void tratador_sigchld(int sig) {
    (void)sig;
    int errno_salvo = errno;
    /* TODO: laço com waitpid(-1, &st, WNOHANG) > 0 */

```

```

    errno = errno_salvo;
}

static void instalar_sinais(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = tratador_sigchld;
    sa.sa_flags = SA_RESTART;          /* read/fgets não falham com EINTR */
    sigaction(SIGCHLD, &sa, NULL);
    /* TODO: SIGINT -> SIG_IGN (ou tratador que só escreve "\n") */
}

```

Python | __main__.py — instalação de tratadores (fragmento)

```

import os, signal

def tratador_sigchld(signum, frame):
    # TODO: laço com os.waitpid(-1, os.WNOHANG); tratar ChildProcessError
    ...

def instalar_sinais():
    signal.signal(signal.SIGCHLD, tratador_sigchld)
    # TODO: signal.signal(signal.SIGINT, ...)

# No filho, antes do execvp:
# signal.signal(signal.SIGINT, signal.SIG_DFL)

```

⚠ Atenção

Conflito entre SIGCHLD e waitpid do primeiro plano. Se o tratador recolher com `waitpid(-1, ...)` o filho que o shell está esperando em primeiro plano, o `waitpid(pid, ...)` do laço principal falhará com `ECHILD`. Soluções possíveis: (a) o tratador só recolhe PIDs registrados como background; (b) bloquear `SIGCHLD` (`sigprocmask`) enquanto espera o primeiro plano; (c) tratar `ECHILD`/`EINTR` no `waitpid` principal. Documente a solução adotada. Em Python, o `os.waitpid` lança `ChildProcessError` nesse caso.

✓ Verifique antes de seguir

- `sleep 30` & imprime algo como `[1] 8421` e o prompt volta imediatamente; outros comandos funcionam em seguida.
- Após 30s, `ps -ef | grep defunct` (em outro terminal) não mostra zumbis do `mysh`.
- `sleep 100` + `Ctrl+C`: o `sleep` termina e o shell mostra um novo prompt.
- `Ctrl+C` no prompt vazio não encerra o shell.
- `sleep 5 | sleep 5 &` (pipeline em background) funciona.

1.4.10 Passo 9 — Robustez, memória e histórico

O que revisar.

- Linhas maiores que o limite e mais argumentos que `MAX_ARGS` devem gerar erro, nunca escrita fora do vetor.

- Mensagens de erro consistentes, sempre com o nome do comando.
- Em C, liberar a memória alocada para tokens e estruturas a cada iteração; conferir com `valgrind --leak-check=full ./mysh`.
- Verificar descritores vazados: no prompt, `ls -l /proc/$$/fd` (executado *pelo mysh*, mostrando os fds do filho) deve listar só 0, 1, 2 e o próprio diretório lido.
- (Opcional) Histórico: guardar a última linha válida; ao receber `!!`, substituí-la antes do parsing e ecoá-la na tela.

C | main.c — histórico !! (fragmento)

```
static char ultima[MAX_LINE] = "";
/* ... dentro do laço, após ler 'linha' e antes do parser ... */
if (strcmp(linha, "!!") == 0) {
    /* TODO: se não houver histórico -> erro; senão copiar 'ultima'
     *      para 'linha' e ecoar o comando */
}
/* TODO: após parsing bem-sucedido, atualizar 'ultima' */
```

1.5 Exemplos de entrada e saída

Os cenários abaixo formam o roteiro mínimo de testes. As saídas de programas externos (datas, listagens, PIDs) naturalmente variam de máquina para máquina; o que importa é o *comportamento*.

1. Comando simples e com argumentos

>_ fork / exec / wait

```
mysh> date
ter 15 set 2026 14:32:10 -03
mysh> ls -l /tmp
total 4
-rw-r--r-- 1 aluno aluno 120 set 15 14:10 teste.txt
mysh>
```

Valida: criação do filho, busca no `PATH`, montagem do vetor de argumentos e espera pelo término.

2. Comando inexistente

>_ tratamento de erro do exec

```
mysh> comandoquenaoexiste
mysh: comandoquenaoexiste: comando não encontrado
mysh>
```

Valida: o filho termina após a falha do `exec`; o shell continua.

3. Builtins `cd`, `pwd` e `exit`

>_ comandos internos

```
mysh> cd /tmp
mysh> pwd
/tmp
mysh> cd /diretorio/inexistente
mysh: cd: /diretorio/inexistente: No such file or directory
mysh> exit
$
```

Valida: builtins alteram o estado do próprio shell, sem `fork`.

4. Redirecionamentos

>_ >, » e <

```
mysh> echo "teste de redirecionamento" > saida.txt
mysh> echo "segunda linha" >> saida.txt
mysh> cat saida.txt
teste de redirecionamento
segunda linha
mysh> wc -l < saida.txt
2
```

Valida: abertura com as flags corretas e `dup2` para os fds 0 e 1.

5. Pipes simples e encadeados

>_ pipes

```
mysh> ls -l | grep ".txt"
-rw-r--r-- 1 aluno aluno 52 set 15 14:33 saida.txt
mysh> cat /etc/passwd | cut -d: -f1 | sort | head -3
adm
aluno
bin
```

Valida: criação de $N - 1$ pipes e encadeamento correto dos descritores.

6. Pipe combinado com redirecionamento

>_ pipe + >

```
mysh> cat entrada.txt | sort > ordenado.txt
mysh> cat ordenado.txt
linha1
linha2
linha3
```

Valida: o último estágio respeita o redirecionamento de saída.

7. Execução em background

>_ execução em background

```
mysh> sleep 5 &
[1] 8421
mysh> echo continuo livre
continuo livre
mysh>
[1] concluído sleep 5      (notificação opcional)
```

Valida: o shell não espera o filho e, depois, recolhe o zumbi.

8. Robustez do parser

>_ entradas problemáticas

```
mysh>
mysh> echo ola mundo
ola mundo
mysh> ls |
mysh: erro de sintaxe próximo a '|'
mysh> cat <
mysh: erro de sintaxe: esperado nome de arquivo após '<'
mysh> echo "aspas abertas
mysh: erro de sintaxe: aspas não fechadas
```

9. Ctrl+C durante a execução

>_ sinais

```
mysh> sleep 100
^C
mysh>
```

Valida: SIGINT termina o filho, mas não o shell.

1.6 Experimentos e validação funcional

Diferente dos demais trabalhos, o mini-shell não produz métricas numéricas comparativas. Seus “experimentos” consistem em **validação funcional sistemática**, comparando o comportamento do **mysh** com o de um shell padrão (**bash**). Organize um roteiro cobrindo:

1. **Comandos simples:** programas comuns, com e sem argumentos, comando inexistente.
2. **Builtins:** **cd** para diretório válido, inválido e sem argumento; **pwd**; **exit**; **export**.
3. **Redirecionamento:** entrada, saída, *append* e combinação de entrada e saída no mesmo comando; arquivo de entrada inexistente; arquivo de saída sem permissão de escrita.
4. **Pipes:** dois comandos, três ou mais, pipe com redirecionamento nas pontas, erro em um estágio intermediário.

5. **Background:** processo em background enquanto outros comandos são digitados; ausência de zumbis confirmada com `ps`.
6. **Sinais:** `Ctrl+C` com comando em primeiro plano, no prompt vazio e durante um pipeline.
7. **Recursos:** ausência de descritores vazados (`/proc/<pid>/fd`) e de vazamentos de memória (`valgrind`, em C).

Dica

Automatize parte dos testes: um shell que lê da entrada padrão pode receber comandos de um arquivo, por exemplo `./mysh < testes/casos_de_teste.txt`, e sua saída pode ser comparada com `bash < testes/casos_de_teste.txt` usando `diff`. (Nesse modo, o prompt aparecerá misturado à saída; você pode omiti-lo quando a entrada não for um terminal, verificando `isatty(0)` em C ou `sys.stdin.isatty()` em Python.)

Para cada caso, registre em uma tabela: comando exato, resultado esperado, resultado obtido e observações, como no modelo:

#	Cenário	Comando	Mecanismo testado	OK?
1	comando simples	<code>date</code>	<code>fork/exec/wait</code>	
2	com argumentos	<code>ls -l /tmp</code>	montagem de <code>argv</code>	
3	inexistente	<code>xyzabc</code>	falha do <code>exec</code>	
4	builtin	<code>cd /tmp; pwd</code>	execução interna	
5	saída	<code>echo x > a.txt</code>	<code>dup2</code> no <code>fd 1</code>	
6	entrada	<code>wc -l < a.txt</code>	<code>dup2</code> no <code>fd 0</code>	
7	pipe	<code>ls grep .c</code>	<code>pipe</code> + <code>EOF</code>	
8	pipes múltiplos	<code>cat f sort uniq wc -l</code>	$N - 1$ pipes	
9	background	<code>sleep 5 &</code>	<code>WNOHANG</code> , <code>SIGCHLD</code>	
10	sinal	<code>sleep 100 + Ctrl+C</code>	<code>SIGINT</code>	
11	sintaxe	<code>ls </code>	validação do parser	

1.7 Relatório

O relatório segue o modelo SBC (ver orientações gerais) com as seções: Introdução; Fundamentação (processos, `fork/exec`, descritores, pipes, sinais); Metodologia/Implementação (arquitetura, estruturas de dados, decisões de projeto como o tratamento de pipes múltiplos e do conflito `SIGCHLD/waitpid`); Resultados Experimentais (tabela de testes); Discussão; Conclusão (dificuldades, aprendizados, limitações).

Na Discussão, responda obrigatoriamente:

1. Como seu shell trata comandos inexistentes? O que acontece, passo a passo, com o processo filho quando o `execvp` falha?
2. O que acontece com os descritores de arquivo herdados após múltiplos `dup2` em uma cadeia de pipes? Desenhe a tabela de descritores de cada processo para um pipeline de três comandos.
3. Por que é necessário fechar as pontas não usadas dos pipes no processo pai e nos filhos? Mostre (com um experimento) o que acontece quando uma delas é deixada aberta.

1.8 Critérios de avaliação e checklist

Critério	Peso
Execução de comandos simples	20%
Redirecionamentos	20%
Pipes	25%
Background e sinais	15%
Builtins e robustez (tratamento de erros)	10%
Relatório (decisões de projeto, dificuldades)	10%

Checklist de entrega

Ambiente e compilação

- ☐ Compila/executa em Linux (nativo ou WSL) sem erros. **(obrigatório)**
- ☐ C: compilação sem avisos com `-Wall -Wextra`.
- ☐ Makefile na raiz; `make` gera o executável; `make clean` remove os artefatos.

Funcionalidade

- ☐ Comandos simples (`ls -l`) e pipelines (`ls | grep x | wc -l`).
- ☐ Redirecionamentos `<`, `>` e `>>` (criação e `append` corretos).
- ☐ Background (`&`) não bloqueia o prompt; sem zumbis.
- ☐ Builtins `cd`, `exit`, `pwd` (e `export`/histórico, se implementados).
- ☐ Comando inexistente gera mensagem de erro sem travar o shell.
- ☐ `Ctrl+C` interrompe só o filho; `Ctrl+D` encerra o shell.

Qualidade de código

- ☐ Código organizado nos módulos parser, executor, builtins, main.
- ☐ Retorno de todas as chamadas de sistema verificado.
- ☐ C: sem vazamentos relevantes (`valgrind --leak-check=full`).
- ☐ Python: sem uso de `subprocess`, `os.system` ou `os.popen`.

Testes, relatório e integridade

- ☐ Testados todos os cenários da seção de exemplos, casos de erro e casos extremos (linha vazia, pipeline longo).
- ☐ Relatório no Overleaf, modelo SBC, com fontes `.tex` e PDF. **(obrigatório)**
- ☐ Código e relatório de autoria própria, sem uso de IA para gerá-los. **(obrigatório)**
- ☐ Identificação dos autores e prazo respeitado.

1.9 Leituras recomendadas

- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. Cap. 2 (Processos e Threads) e seções sobre o UNIX/Linux.

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. Capítulos sobre Processos e IPC.
- KERRISK, M. *The Linux Programming Interface*. Caps. 20–22 (sinais), 24–27 (criação, término e execução de processos) e 44 (pipes).
- STEVENS, W. R.; RAGO, S. A. *Advanced Programming in the UNIX Environment*. Caps. 8 (controle de processos), 10 (sinais) e 15 (IPC).
- ARPACI-DUSSEAU, R.; ARPACI-DUSSEAU, A. *Operating Systems: Three Easy Pieces*, capítulo “Process API” — <https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf>.
- BRENNAN, S. *Write a Shell in C* (tutorial didático; não cobre pipes nem redirecionamento) — <https://brennan.io/2015/01/16/write-a-shell-in-c/>.
- Man pages do Linux, em <https://man7.org/linux/man-pages/>: `fork(2)`, `exec(3)`, `wait(2)`, `pipe(2)`, `dup(2)`, `open(2)`, `sigaction(2)`, `signal(7)` e `signal-safety(7)`.
- Python: módulos `os` (<https://docs.python.org/3/library/os.html>), `signal` e `shlex`.

Atenção

Estude códigos de terceiros apenas para entender *abordagens*. Copiar funções, mesmo trocando nomes de variáveis, configura plágio e anula o trabalho.

Capítulo 2

Simulador de Escalonador de CPU

2.1 Visão geral

Em um computador moderno, dezenas ou centenas de programas parecem executar “ao mesmo tempo”: o navegador, o editor de texto, o reprodutor de música, serviços do próprio sistema. Porém, um núcleo de processador executa *uma única instrução de um único programa por vez*. A ilusão de simultaneidade é criada pelo **sistema operacional (SO)**, que alterna rapidamente a CPU entre os programas. A parte do SO que decide *qual programa usa a CPU agora e por quanto tempo* chama-se **escalonador de CPU** (*CPU scheduler*).

Neste trabalho você construirá um **simulador**: um programa que lê uma lista de processos fictícios (com instante de chegada, tempo de CPU necessário e prioridade), “executa” esses processos de acordo com diferentes políticas de escalonamento e calcula métricas que permitem comparar as políticas entre si. Nada é executado de verdade — o tempo é simulado por um contador inteiro. O objetivo é entender, com números concretos, por que não existe um algoritmo de escalonamento “melhor” em todos os aspectos.

🎯 Ao final deste trabalho você deverá ser capaz de

- Explicar o que é um processo, seu ciclo de vida e o papel do escalonador e do *dispatcher*.
- Distinguir escalonamento preemptivo de não preemptivo.
- Calcular à mão e por programa as métricas de tempo de espera, *turnaround*, resposta e *throughput*.
- Implementar os algoritmos FCFS, SJF, SRTF, Round Robin e Prioridade (com *aging*) de forma modular.
- Produzir diagramas de Gantt e comparar experimentalmente os algoritmos, discutindo seus *trade-offs*.

👉 Dica

Leia a fundamentação teórica inteira **antes** de programar e resolva à mão (lápiz e papel) os exemplos da Seção 2.5. Se você não consegue produzir o diagrama de Gantt à mão, não conseguirá depurar o simulador.

2.2 Fundamentação teórica

2.2.1 Multiprogramação: por que escalonar?

Nos primeiros computadores, um programa era carregado, executado até o fim e só então o próximo era carregado. O problema é que programas passam boa parte do tempo *esperando* por dispositivos lentos (disco, rede, teclado). Enquanto um programa espera a leitura de um disco — milhões de ciclos de processador —, a CPU fica ociosa.

A **multiprogramação** resolve isso mantendo vários programas carregados na memória: quando o programa em execução precisa esperar por uma operação de entrada/saída (E/S), o SO entrega a CPU a outro programa que esteja pronto. Em sistemas de **tempo compartilhado** (*time-sharing*), a CPU também é retirada periodicamente de um programa, mesmo que ele não esteja esperando nada, para que todos os usuários tenham a sensação de resposta rápida.

Sempre que a CPU fica livre, o SO precisa responder a uma pergunta: *entre os programas prontos, qual executa agora?* A regra usada para responder a essa pergunta é o **algoritmo de escalonamento**.

2.2.2 Processo e Bloco de Controle de Processo (PCB)

Definição: Processo

Um **processo** é um programa em execução. Enquanto o programa é apenas um arquivo passivo no disco (código), o processo é uma entidade ativa: possui o código carregado na memória, seus dados, sua pilha, o valor atual dos registradores da CPU (incluindo o contador de programa, que indica a próxima instrução) e recursos do SO (arquivos abertos, por exemplo).

Para gerenciar os processos, o SO mantém para cada um deles uma estrutura de dados chamada **PCB** (*Process Control Block*, Bloco de Controle de Processo). O PCB guarda tudo o que é preciso para suspender o processo e, mais tarde, retomá-lo exatamente do ponto onde parou:

- identificador do processo (**PID**);
- estado atual (pronto, executando, bloqueado...);
- contador de programa e demais registradores salvos;
- informações de escalonamento: prioridade, tempo de CPU já consumido, ponteiros para filas;
- informações de memória e de E/S (tabelas de páginas, arquivos abertos).

Ligação com o trabalho

No simulador, a **struct** (C) ou a classe (Python) que representa um processo é o seu “PCB simplificado”: ela conterà PID, chegada, *burst*, prioridade e os campos que mudam durante a simulação (tempo restante, instante de início, instante de término).

2.2.3 Estados de um processo

Durante sua vida, um processo passa por diferentes **estados**. O modelo clássico de cinco estados está na Figura 2.1.

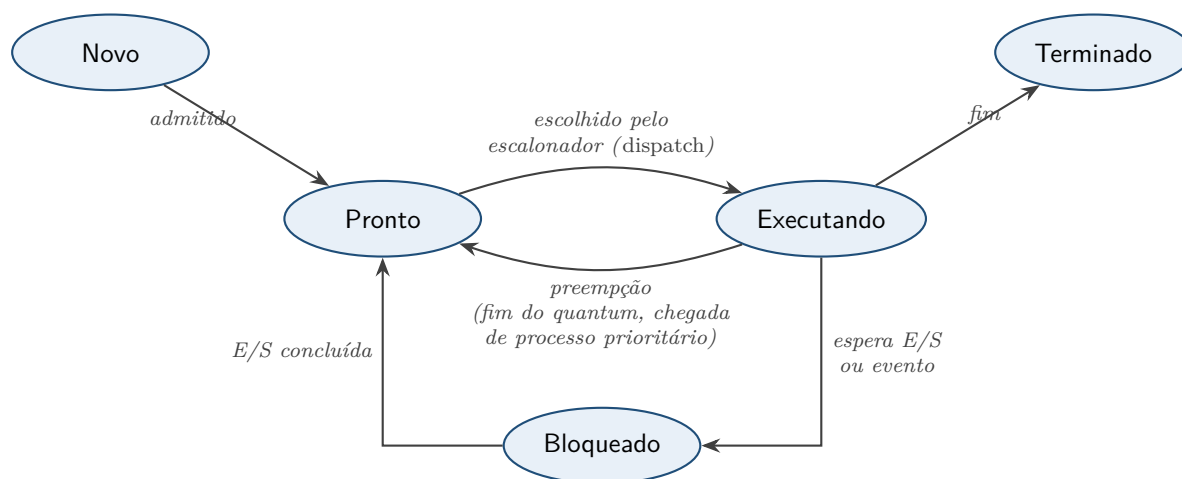


Figura 2.1: Diagrama de estados de um processo.

Novo: o processo está sendo criado.

Pronto: o processo tem tudo o que precisa para executar, exceto a CPU. Fica numa **fila de prontos** (*ready queue*).

Executando: suas instruções estão sendo executadas pela CPU. Em uma CPU de um núcleo, *no máximo um* processo está neste estado.

Bloqueado (ou Esperando): o processo aguarda um evento (fim de leitura de disco, chegada de pacote de rede, tecla digitada). Mesmo que a CPU esteja livre, ele não pode executar.

Terminado: o processo concluiu e seus recursos estão sendo liberados.

O escalonador de CPU atua exatamente na transição *Pronto* → *Executando*: ele escolhe *qual* processo da fila de prontos recebe a CPU.

2.2.4 Ciclos de CPU e de E/S

A execução de um processo real alterna entre **rajadas de CPU** (*CPU bursts*), em que ele calcula, e **rajadas de E/S**, em que ele espera um dispositivo:



- Processos **CPU-bound** (limitados pela CPU) têm rajadas de CPU longas e poucas operações de E/S — por exemplo, compressão de vídeo, cálculo científico.
- Processos **I/O-bound** (limitados por E/S) têm rajadas de CPU curtas e muitas esperas — por exemplo, um editor de texto, que passa a maior parte do tempo esperando teclas.

Um bom escalonador favorece processos I/O-bound: eles usam a CPU rapidamente e logo liberam-na, mantendo os dispositivos ocupados em paralelo com a CPU. **Neste trabalho**, por simplicidade, cada processo tem *uma única* rajada de CPU (o *burst*) e não faz E/S; o “tamanho do burst” é que diferencia processos curtos de longos.

2.2.5 Escalonador, *dispatcher* e troca de contexto

Definição: Escalonador e *dispatcher*

O **escalonador de curto prazo** é o componente que *decide* qual processo pronto receberá a CPU. O ***dispatcher*** é o componente que *efetiva* a decisão: salva o contexto do processo que sai, carrega o contexto do processo que entra, muda para modo usuário e salta para a instrução onde o novo processo havia parado.

A operação feita pelo dispatcher chama-se **troca de contexto** (*context switch*):

1. salvar os registradores do processo A no PCB de A;
2. atualizar o estado de A (para Pronto ou Bloqueado) e colocá-lo na fila adequada;
3. escolher B (decisão do escalonador);
4. restaurar os registradores de B a partir do PCB de B e retomar sua execução.

Atenção

A troca de contexto é **puro custo**: durante ela, nenhum processo de usuário progride. Tipicamente dura de alguns microssegundos a dezenas de microssegundos, e ainda causa efeitos indiretos (a memória *cache* “esfria” porque o novo processo usa outros dados). Algoritmos que trocam de processo com muita frequência pagam esse custo muitas vezes. Nosso simulador *não* descontará esse custo do tempo, mas **contará quantas trocas ocorreram**, para que você possa discuti-lo.

2.2.6 Escalonamento preemptivo e não preemptivo

A decisão de escalonamento pode ocorrer em quatro situações:

1. o processo em execução bloqueia (pede E/S);
2. o processo em execução termina;
3. um processo passa de Bloqueado para Pronto, ou um novo processo chega;
4. uma interrupção de relógio (*timer*) indica que o processo em execução esgotou sua fatia de tempo.

Definição: Preempção

Um escalonador é **não preemptivo** (cooperativo) quando só troca de processo nas situações 1 e 2: uma vez que o processo recebe a CPU, ele a mantém até terminar ou bloquear. É **preemptivo** quando também pode *retirar à força* a CPU do processo em execução (situações 3 e 4), devolvendo-o à fila de prontos.

A preempção melhora a responsividade (um processo longo não “sequestra” a CPU), mas exige cuidado: o processo interrompido pode estar no meio de uma atualização de dados compartilhados — o que motivará o trabalho de sincronização (Capítulo de Sincronização).

2.2.7 Critérios e métricas de escalonamento

Para comparar algoritmos precisamos de métricas objetivas. Considere, para cada processo *i*:

Símbolo	Significado
C_i	instante de chegada (<i>arrival</i>)
B_i	tempo de CPU necessário (<i>burst</i>)
S_i	instante em que o processo recebe a CPU pela primeira vez (<i>start</i>)
F_i	instante em que o processo termina (<i>finish/completion</i>)

Definição: Métricas por processo

$$\begin{aligned} \text{Turnaround}_i &= F_i - C_i && (\text{tempo total no sistema}) \\ \text{Espera}_i &= \text{Turnaround}_i - B_i && (\text{tempo total na fila de prontos}) \\ \text{Resposta}_i &= S_i - C_i && (\text{tempo até ser atendido pela 1ª vez}) \end{aligned}$$

Definição: Métricas agregadas

Para n processos e duração total da simulação T (instante em que o último processo termina, descontado o início):

$$\begin{aligned} \overline{\text{Espera}} &= \frac{1}{n} \sum_{i=1}^n \text{Espera}_i, & \overline{\text{Turnaround}} &= \frac{1}{n} \sum_{i=1}^n \text{Turnaround}_i \\ \text{Throughput} &= \frac{n}{T} \text{ (processos concluídos por unidade de tempo)} \\ \text{Utilização da CPU} &= \frac{\sum_i B_i}{T} \times 100\% \end{aligned}$$

Objetivos típicos: *maximizar* utilização e throughput; *minimizar* turnaround, espera e resposta. Esses objetivos conflitam entre si — e é isso que o trabalho quer que você observe.

Dica

Note que a fórmula da espera funciona tanto para algoritmos preemptivos quanto não preemptivos: em qualquer caso, o tempo que o processo passou no sistema sem estar executando é Turnaround – Burst. Assim você não precisa “cronometrar” a espera durante a simulação: basta registrar F_i . Já a resposta exige registrar S_i no *primeiro* despacho do processo.

2.2.8 Diagrama de Gantt

O **diagrama de Gantt** é uma linha do tempo que mostra qual processo ocupa a CPU em cada intervalo. Cada bloco é um intervalo de execução contínua; abaixo, marcam-se os instantes de início e fim. Todos os exemplos a seguir usam a carga de referência da Tabela 2.1, em que **menor número = maior prioridade**.

Tabela 2.1: Carga de referência (`processos.txt`).

PID	Chegada	Burst	Prioridade
P1	0	8	2
P2	1	4	1
P3	2	9	3
P4	3	5	2

2.2.9 FCFS — *First-Come, First-Served*

A política mais simples: a CPU é concedida na **ordem de chegada**, como numa fila de banco. É não preemptiva: quem começa, termina.

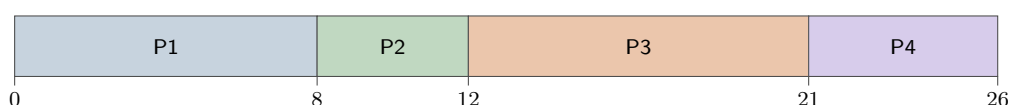


Figura 2.2: FCFS na carga de referência.

PID	Chegada	Burst	Início	Término	Espera	Turnaround
P1	0	8	0	8	0	8
P2	1	4	8	12	7	11
P3	2	9	12	21	10	19
P4	3	5	21	26	18	23
Médias					8,75	15,25

Confira uma linha: P4 chega em 3 e termina em 26, logo turnaround = $26 - 3 = 23$ e espera = $23 - 5 = 18$. Em FCFS não preemptivo, resposta = espera (cada processo executa de uma vez).

💡 Efeito comboio (*convoy effect*)

Se um processo muito longo chega primeiro, todos os curtos que chegam depois ficam presos atrás dele, como carros atrás de um caminhão numa estrada de pista única. Exemplo: bursts 100, 1, 1, 1 chegando em 0,1,2,3. Espera média em FCFS: $(0 + 99 + 99 + 99)/4 = 74,25$. Se os quatro estivessem disponíveis em $t = 0$ e os curtos executassem primeiro, as esperas seriam 0, 1, 2 e 3, com média de apenas 1,5. FCFS é justo na ordem, mas pode ser péssimo na média.

2.2.10 SJF — *Shortest Job First*

Sempre que a CPU fica livre, escolhe-se, **entre os processos que já chegaram**, aquele com **menor burst**. Na versão não preemptiva, o escolhido executa até o fim. Empates são desfeitos por ordem de chegada (e depois por PID).

Na carga de referência: em $t = 0$ só P1 chegou, então P1 executa até 8 (não há preempção!). Em $t = 8$, estão prontos P2 (4), P3 (9) e P4 (5): escolhe-se P2. Em $t = 12$: P4 (5) antes de P3 (9).

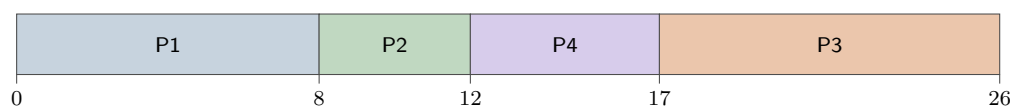


Figura 2.3: SJF não preemptivo na carga de referência.

PID	Chegada	Burst	Início	Término	Espera	Turnaround
P1	0	8	0	8	0	8
P2	1	4	8	12	7	11
P3	2	9	17	26	15	24
P4	3	5	12	17	9	14
Médias					7,75	14,25

💡 Por que o SJF é ótimo?

Se todos os processos estão disponíveis ao mesmo tempo, o SJF produz a **menor espera média possível**. Intuição: o processo executado na posição k faz esperar todos os $n - k$ que vêm depois dele. Colocar um processo curto antes de um longo reduz a espera do longo em pouco (o burst do curto) e reduz a espera do curto em muito (o burst do longo). Trocar qualquer par “longo antes de curto” sempre diminui a soma das esperas; logo a ordem crescente de burst é a ótima.

⚠️ Atenção

O SJF tem dois problemas práticos: (1) um SO real **não conhece** o burst futuro de um processo — ele precisa *estimá-lo* (por exemplo, pela média exponencial dos bursts anteriores: $\tau_{n+1} = \alpha \tau_n + (1 - \alpha) \tau_n$); (2) um processo longo pode sofrer **inanição** (*starvation*) se processos curtos continuarem chegando indefinidamente. No simulador, o burst é conhecido (vem do arquivo), o que torna o SJF implementável.

2.2.11 SRTF — *Shortest Remaining Time First*

É o SJF **preemptivo**: a cada chegada de processo, compara-se o burst do recém-chegado com o **tempo restante** do processo em execução; se o novo for menor, ocorre preempção.

Na carga de referência: P1 inicia em 0. Em $t = 1$ chega P2 (4) e P1 ainda precisa de 7: P2 toma a CPU. Em $t = 2$ chega P3 (9) e em $t = 3$ chega P4 (5); nenhum é menor que o restante de P2 (3 e 2). P2 termina em 5; restantes: P1=7, P3=9, P4=5 $\Rightarrow$ P4. Em 10: P1 (7) antes de P3 (9).



Figura 2.4: SRTF na carga de referência.

PID	1º início	Término	Espera	Turnaround	Resposta
P1	0	17	9	17	0
P2	1	5	0	4	0
P3	17	26	15	24	15
P4	5	10	2	7	2
Médias			6,50	13,00	4,25

O SRTF é ótimo para a espera média quando os processos chegam em instantes diferentes, mas gera mais trocas de contexto e agrava o risco de inanição dos processos longos.

2.2.12 RR — *Round Robin* (alternância circular)

Projetado para sistemas interativos. Define-se uma fatia de tempo, o **quantum** q . A fila de prontos é uma fila circular (FIFO):

1. retira-se o processo da frente da fila e ele executa por *no máximo* q unidades;
2. se terminar antes, libera a CPU; senão, é **preemptado** e volta para o **final** da fila;
3. processos que chegam entram no final da fila.

⚠ Atenção

Há uma ambiguidade clássica: se um processo chega *exatamente* no instante em que outro sofre preempção, quem entra primeiro na fila? **Convenção adotada neste roteiro:** processos que chegaram durante (ou no fim de) um quantum entram na fila **antes** do processo preemptado. Qualquer que seja a sua escolha, documente-a no relatório — ela altera o resultado.

Exemplo com $q = 3$ na carga de referência (acompanhe a fila):

Intervalo	Executa	Fila de prontos ao final do intervalo
0–3	P1 (resta 5)	P2, P3, P4, P1
3–6	P2 (resta 1)	P3, P4, P1, P2
6–9	P3 (resta 6)	P4, P1, P2, P3
9–12	P4 (resta 2)	P1, P2, P3, P4
12–15	P1 (resta 2)	P2, P3, P4, P1
15–16	P2 (termina)	P3, P4, P1
16–19	P3 (resta 3)	P4, P1, P3
19–21	P4 (termina)	P1, P3
21–23	P1 (termina)	P3
23–26	P3 (termina)	—

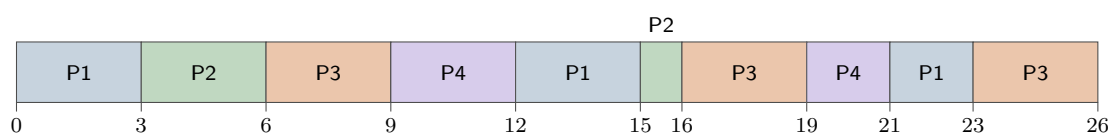


Figura 2.5: Round Robin ($q = 3$) na carga de referência: 10 intervalos, 9 trocas de contexto.

PID	1º início	Término	Espera	Turnaround	Resposta
P1	0	23	15	23	0
P2	3	16	11	15	2
P3	6	26	15	24	4
P4	9	21	13	18	6
Médias			13,50	20,00	3,00

Observe o *trade-off*: a espera média piorou em relação ao FCFS (13,50 contra 8,75), mas a **resposta média** caiu de 8,75 para 3,00 — todo processo é atendido rapidamente, o que um usuário interativo percebe como “sistema ágil”.

💡 A escolha do quantum

- **q muito grande** (maior que todos os bursts): nenhum processo é preemptado e o RR degenera em FCFS. Na carga de referência, $q = 10$ produz exatamente o resultado do FCFS.
- **q muito pequeno**: a resposta é excelente, mas o número de trocas de contexto explode. Se cada troca custa s e o quantum é q , a fração da CPU desperdiçada é aproximadamente $s/(q + s)$: com $q = s$, metade da CPU vai para o SO!
- Regra prática: escolher q de modo que cerca de 80% dos bursts sejam menores que q .

2.2.13 Escalonamento por Prioridade

Cada processo recebe um número de prioridade; a CPU vai para o processo pronto de **maior prioridade**. Neste trabalho, **menor número = maior prioridade**; empates por ordem de chegada. Pode ser não preemptivo (decide só quando a CPU fica livre) ou preemptivo (um processo mais prioritário que chega toma a CPU). O SJF é um caso particular: prioridade = burst.

Na carga de referência (não preemptivo): em $t = 0$ só P1; em $t = 8$ estão prontos P2 (1), P3 (3), P4 (2) $\Rightarrow$ P2, depois P4, depois P3. Por coincidência, a ordem é a mesma do SJF: espera média 7,75 e turnaround médio 14,25.

Um exemplo que evidencia melhor a política usa a carga abaixo:

PID	Chegada	Burst	Prioridade
P1	0	5	3
P2	1	3	1
P3	2	8	4
P4	3	6	2

Esperas: P1=0, P2=4, P3=12, P4=5; média 5,25; turnaround médio 10,75.

Definição: Inanição (*starvation*) e envelhecimento (*aging*)

Inanição ocorre quando um processo pronto espera *indefinidamente* porque sempre há outro mais prioritário (ou mais curto, no SJF) para executar. **Aging** é a técnica que resolve isso: a prioridade *efetiva* de um processo **melhora gradualmente enquanto ele espera**. Por exemplo, “a cada 5 unidades de tempo na fila de prontos, a prioridade efetiva diminui 1 (melhora)”. Eventualmente, o processo esquecido torna-se o mais prioritário e executa.

No exemplo acima, com aging de 1 nível a cada 5 unidades de espera: P3 chegou em 2 e, em $t = 7$, completa 5 unidades de espera: prioridade efetiva $4 \rightarrow 3$; em $t = 12$, $3 \rightarrow 2$. Nesta carga curta o resultado final não muda (P3 já seria o último), mas numa carga em que processos de prioridade 1 chegam continuamente, **sem** aging P3 nunca executaria, e **com** aging ele acaba alcançando prioridade 1 e é atendido. É exatamente esse contraste que você deverá demonstrar experimentalmente.

Dica

Guarde **duas** prioridades no processo: a *original* (lida do arquivo, nunca muda) e a *efetiva* (a que o aging altera e que o escalonador usa). Assim o relatório pode mostrar “P3: prioridade $4 \rightarrow 2$ após 10 u.t. de espera”.

2.2.14 Resumo comparativo

Tabela 2.2: Resultados na carga de referência (convenções deste roteiro).

Algoritmo	Espera média	Turnaround médio	Resposta média	Trocas de contexto
FCFS	8,75	15,25	8,75	3
SJF	7,75	14,25	7,75	3
SRTF	6,50	13,00	4,25	4
RR ($q = 3$)	13,50	20,00	3,00	9
Prioridade (não preempt.)	7,75	14,25	7,75	3

Aqui “trocas de contexto” = número de blocos do Gantt menos 1 (mudanças do processo que ocupa a CPU; não contamos a carga do primeiro processo).

Algoritmo	Preemptivo?	Inanição?	Ponto forte / fraco
FCFS	Não	Não	Simples e justo na ordem; sofre efeito comboio.
SJF	Não	Sim	Espera média mínima; exige conhecer o burst.
SRTF	Sim	Sim	Ainda melhor na espera; mais trocas de contexto.
RR	Sim	Não	Ótima resposta; turnaround alto; depende de q .
Prioridade	Ambos	Sim (sem aging)	Expressa importância; precisa de aging.

2.2.15 Como simular o tempo

Há duas formas de fazer o relógio do simulador avançar:

Tempo discreto (*tick a tick*): um laço incrementa t de 1 em 1; a cada unidade decide-se quem executa. É simples e natural para SRTF e aging, mas desperdiça iterações quando nada acontece (ex.: CPU ociosa esperando uma chegada em $t = 1000$).

Orientada a eventos: o relógio *salta* diretamente para o próximo evento relevante: fim do burst, fim do quantum ou próxima chegada. Em FCFS/SJF, o próximo evento é sempre “processo atual termina” ($t += \text{burst}$). No SRTF, é o menor entre “termina o atual” e “chega o próximo processo”.

Dica

Ambas são aceitas. Recomenda-se orientação a eventos para FCFS, SJF, Prioridade e RR (saltos de **burst** ou **quantum**), e tick a tick para SRTF se você achar mais fácil. Não esqueça do caso de **CPU ociosa**: se não há processo pronto, o relógio salta para a próxima chegada.

2.2.16 Leitura extra: filas multinível

Sistemas reais combinam as ideias acima. Nas **filas multinível com realimentação** (*Multilevel Feedback Queue*, MLFQ), há várias filas de prioridade, cada uma com seu quantum; um processo que esgota o quantum é rebaixado (provavelmente é CPU-bound), e um processo que bloqueia cedo permanece nas filas altas (provavelmente é interativo); periodicamente todos são promovidos (uma forma de aging). O escalonador do Linux (CFS/EEVDF) segue outra ideia: dar a cada processo uma fração “justa” da CPU proporcional ao seu peso. Não é necessário implementar isso, mas o capítulo *Multi-Level Feedback Queue* do livro OSTEP é uma excelente leitura complementar.

2.3 Especificação do trabalho

2.3.1 Entrada

Um arquivo texto, um processo por linha: **PID Chegada Burst Prioridade**. Linhas iniciadas por # e linhas vazias são ignoradas. Todos os valores são inteiros não negativos ($\text{burst} > 0$).

 entradas/processos.txt

#	PID	Chegada	Burst	Prioridade
1	0	8	2	
2	1	4	1	
3	2	9	3	
4	3	5	2	

2.3.2 Algoritmos

Algoritmo	Sigla	Preemptivo?	Observação
First-Come, First-Served	FCFS	Não	<i>Baseline</i>
Shortest Job First	SJF	Não	Burst conhecido
Shortest Remaining Time First	SRTF	Sim	SJF preemptivo
Round Robin	RR	Sim	Parâmetro: quantum
Prioridade	PRIO	Configurável	Com <i>aging</i> parametrizável

2.3.3 Saídas obrigatórias

Para cada algoritmo:

1. diagrama de Gantt em texto;
2. tabela por processo: início, término, espera, turnaround, resposta;
3. métricas agregadas: médias de espera, turnaround e resposta; throughput; número de trocas de contexto;
4. (via script) gráfico comparativo entre algoritmos.

2.3.4 Linha de comando sugerida

>_ Uso

```
$ ./escalonador entradas/processos.txt --alg FCFS
$ ./escalonador entradas/processos.txt --alg RR --quantum 3
$ ./escalonador entradas/processos.txt --alg PRI0 --aging 5
$ ./escalonador entradas/processos.txt --alg TODOS --csv saidas/res.csv
$ python3 escalonador.py entradas/processos.txt --alg RR --quantum 3
```

2.3.5 Estrutura de entrega

📄 Estrutura em C

```
cpu-scheduler/
|-- Makefile
|-- include/
|   |-- processo.h   leitor.h   escalonador.h   metricas.h   gantt.h
|-- src/
|   |-- main.c
|   |-- leitor.c
|   |-- fcfs.c   sjf.c   srtf.c   rr.c   prioridade.c
|   |-- metricas.c   gantt.c
|-- entradas/      carga1.txt, carga2.txt, ...
|-- saidas/        resultados .txt/.csv
|-- analise.py     (opcional: graficos)
+-- relatorio/     fontes .tex + PDF (modelo SBC)
```

📄 Estrutura em Python

```
cpu-scheduler/
|-- Makefile          (alvos: run, test, graficos, clean)
|-- escalonador/
|   |-- __init__.py
|   |-- processo.py   leitor.py   metricas.py   gantt.py
|   |-- fcfs.py   sjf.py   srtf.py   rr.py   prioridade.py
|-- main.py
|-- entradas/   saidas/
|-- analise.py
+-- relatorio/
```

2.4 Roteiro de implementação

Implemente na ordem abaixo. Cada passo termina com uma verificação: **não avance** sem que ela passe. Os trechos de código são *ilustrativos e incompletos* — as partes marcadas com **TODO** e ... são responsabilidade sua.

2.4.1 Passo 1 — Estrutura de dados do processo

Represente o processo (nosso “PCB simplificado”). Separe os campos *lidos do arquivo* (nunca mudam) dos campos de *estado da simulação* (mudam a cada execução de algoritmo).

 C | include/processo.h (parcial)

```
typedef struct {
    /* dados de entrada */
    int pid, chegada, burst, prioridade;
    /* estado da simulação */
    int restante;           /* burst ainda não executado (SRTF, RR) */
    int prio_efetiva;       /* alterada pelo aging */
    int inicio;             /* 1 despacho; -1 = nunca executou */
    int termino;           /* -1 enquanto não terminou */
    int espera_fila;        /* contador usado pelo aging */
    /* TODO: campos de métricas, se preferir guardá-las aqui */
} Processo;
```

 Python | escalonador/processo.py (parcial)

```
from dataclasses import dataclass, field

@dataclass
class Processo:
    pid: int
    chegada: int
    burst: int
    prioridade: int
    restante: int = field(init=False)
    inicio: int = -1
    termino: int = -1
    # TODO: prio_efetiva, espera_fila ...

    def __post_init__(self):
        self.restante = self.burst
        # TODO: inicializar prio_efetiva
```

Atenção

Cada algoritmo **modifica** os processos (restante, início, término). Se você rodar FCFS e depois SJF sobre o *mesmo* vetor, o SJF recebe processos “já terminados”. Sempre execute cada algoritmo sobre uma **cópia** da carga original: em C, copie o vetor (`memcpy` ou `laço`); em Python, use `copy.deepcopy` ou recrie os objetos.

✓ Verifique antes de seguir

Crie à mão um processo (1, 0, 8, 2) e imprima todos os campos: `restante` deve valer 8, `inicio` e `termino` devem valer -1.

2.4.2 Passo 2 — Leitura da carga de trabalho

A função `carregar_processos` recebe o caminho do arquivo e devolve a coleção de processos. Ela deve ignorar comentários e linhas vazias e validar os valores.

✎ C | `src/leitor.c` (fragmento)

```
int carregar_processos(const char *caminho, Processo v[], int max) {
    FILE *f = fopen(caminho, "r");
    if (!f) { perror(caminho); return -1; }
    char linha[256];
    int n = 0;
    while (fgets(linha, sizeof linha, f)) {
        if (linha[0] == '#' || linha[0] == '\n') continue;
        /* TODO: sscanf dos 4 inteiros; checar retorno == 4,
           valores >= 0 e n < max; inicializar estado */
    }
    ...
}
```

🐍 Python | `escalador/leitor.py` (fragmento)

```
def carregar_processos(caminho):
    processos = []
    with open(caminho) as f:
        for num, linha in enumerate(f, start=1):
            linha = linha.strip()
            if not linha or linha.startswith('#'):
                continue
            # TODO: dividir em 4 inteiros, validar e criar Processo
            ...
    return processos
```

✓ Verifique antes de seguir

Leia `processos.txt` e imprima a lista: 4 processos, com os valores da Tabela 2.1. Teste também um arquivo inexistente e um arquivo com uma linha malformada (1 0 x 2): o programa deve emitir mensagem clara, sem travar.

2.4.3 Passo 3 — Infraestrutura de métricas

Implemente as métricas *antes* dos algoritmos: assim todos os algoritmos as reutilizam. As fórmulas estão na Seção 2.2.7.

✎ C | `include/metricas.h` (parcial)

```
typedef struct {
    double espera_media, turnaround_medio, resposta_media;
```

```
double throughput;
int trocas_contexto;
} Metricas;

/* pré-condição: todos os processos com termino != -1 */
Metricas calcular_metricas(const Processo v[], int n,
                           const Intervalo g[], int ng);
```

Python | escalonador/metricas.py (fragmento)

```
def metricas_processo(p):
    turnaround = p.termino - p.chegada
    espera = ... # TODO: use a fórmula da seção de métricas
    resposta = ... # TODO
    return turnaround, espera, resposta
```

Verifique antes de seguir

Preencha à mão os campos **inicio/termino** com os valores do FCFS da Seção 2 (P1: 0/8, P2: 8/12, P3: 12/21, P4: 21/26) e chame a função: as médias devem ser $\text{espera} = 8,75$ e $\text{turnaround} = 15,25$. Você testou as métricas sem ter nenhum algoritmo pronto!

2.4.4 Passo 4 — Registro da linha do tempo (Gantt)

Cada algoritmo, ao decidir que um processo ocupa a CPU de **ini** até **fim**, chama **registrar_execucao**. Se o intervalo novo é do mesmo processo e começa onde o anterior terminou, *estenda* o anterior em vez de criar outro (isso é essencial no SRTF tick a tick). O número de trocas de contexto é o número de intervalos menos um.

C | src/gantt.c (fragmento)

```
typedef struct { int pid, ini, fim; } Intervalo;

void registrar_execucao(Intervalo g[], int *ng, int pid, int ini, int fim) {
    if (*ng > 0 && g[*ng - 1].pid == pid && g[*ng - 1].fim == ini) {
        /* TODO: estender o último intervalo */
        return;
    }
    /* TODO: acrescentar novo intervalo (checar limite do vetor) */
}
```

Python | escalonador/gantt.py (fragmento)

```
def imprimir_gantt(intervalos):
    barra = '|'
    marcas = ''
    for pid, ini, fim in intervalos:
        celula = f' P{pid} '.center(max(4, 2 * (fim - ini)))
        # TODO: acumular celula + '|' em barra e posicionar o
        # número 'ini' em marcas alinhado ao '|' correspondente
        ...
    print(barra)
    print(marcas)
```

Dica

Represente também intervalos de **CPU ociosa** (por exemplo, com `pid = -1` impresso como `--`). Uma carga em que o primeiro processo chega em $t = 5$ deve mostrar `| -- | P1 | ...`.

Verifique antes de seguir

Registre manualmente $(1, 0, 1)$, $(1, 1, 3)$, $(2, 3, 5)$: devem restar **2** intervalos, **P1** 0-3 e **P2** 3-5, e 1 troca de contexto.

2.4.5 Passo 5 — FCFS — o modelo estrutural

Todos os algoritmos terão a mesma “forma”: recebem a coleção de processos e devolvem o Gantt, deixando os campos `inicio/termino` preenchidos. Fixe essa interface agora.

C | include/escalonador.h (parcial)

```
/* Cada algoritmo é uma função com esta assinatura:
   modularidade = acrescentar um algoritmo sem alterar os outros */
typedef int (*FuncEscalonador)(Processo v[], int n,
                               Intervalo g[], const Config *cfg);

int escalonar_fcfs(Processo v[], int n, Intervalo g[], const Config *cfg);
int escalonar_sjf (Processo v[], int n, Intervalo g[], const Config *cfg);
/* TODO: srtf, rr, prioridade */
```

Lógica do FCFS (em comentário — a implementação é sua):

C | src/fcfs.c (esqueleto da lógica)

```
int escalonar_fcfs(Processo v[], int n, Intervalo g[], const Config *cfg) {
    int t = 0, ng = 0;
    qsort(v, n, sizeof(Processo), cmp_chegada_pid);
    for (int i = 0; i < n; i++) {
        /* 1. se t < v[i].chegada: CPU ociosa -> avançar t */
        /* 2. registrar inicio do processo */
        /* 3. registrar intervalo [t, t + burst) no Gantt */
        /* 4. avançar t e registrar termino */
        /* TODO */
    }
    return ng;
}
```

Python | escalonador/fcfs.py (fragmento)

```
def escalonar_fcfs(processos, cfg):
    gantt = []
    t = 0
    for p in sorted(processos, key=lambda p: (p.chegada, p.pid)):
        # TODO: tratar CPU ociosa, preencher inicio/termino,
        #         registrar (p.pid, ini, fim) em gantt
        ...
    return gantt
```

✓ Verifique antes de seguir

Na carga de referência: Gantt P1 0-8 | P2 8-12 | P3 12-21 | P4 21-26, espera média 8,75, turnaround médio 15,25, 3 trocas de contexto.

2.4.6 Passo 6 — SJF (não preemptivo) e SRTF (preemptivo)

A diferença estrutural em relação ao FCFS: a ordem **não é fixa**. A cada vez que a CPU fica livre, é preciso olhar apenas os processos *já chegados e não terminados* e escolher o de menor burst.

✎ C | Seleção do próximo processo (fragmento)

```
/* Retorna o índice do processo pronto de menor burst em t,
   ou -1 se nenhum processo chegou ainda. */
static int escolher_menor_burst(Processo v[], int n, int t) {
    int melhor = -1;
    for (int i = 0; i < n; i++) {
        if (v[i].termino != -1 || v[i].chegada > t) continue;
        /* TODO: comparar burst; desempate por chegada e depois pid */
    }
    return melhor;
}
```

🐍 Python | Seleção do próximo processo (fragmento)

```
prontos = [p for p in processos if p.chegada <= t and p.termino == -1]
if not prontos:
    # TODO: CPU ociosa -> saltar t para a próxima chegada
    ...
escolhido = min(prontos, key=lambda p: (p.burst, p.chegada, p.pid))
```

Para o **SRTF**, o critério passa a ser **restante** (não **burst**) e a decisão é refeita a cada unidade de tempo (ou a cada chegada). Esboço da lógica, tick a tick:

🐍 Python | escalonador/srtf.py (pseudocódigo comentado)

```
while concluidos < len(processos):
    # 1. prontos = chegados e não terminados
    # 2. se vazio: t += 1 (ou salta para a próxima chegada); continue
    # 3. p = pronto com menor (restante, chegada, pid)
    # 4. se p.inicio == -1: p.inicio = t      <- tempo de resposta!
    # 5. registrar (p.pid, t, t + 1) no Gantt (o registro funde)
    # 6. p.restante -= 1; t += 1
    # 7. se p.restante == 0: p.termino = t; concluidos += 1
    ...
```

✓ Verifique antes de seguir

SJF, carga de referência: P1 0-8 | P2 8-12 | P4 12-17 | P3 17-26, espera 7,75, turnaround 14,25.

SRTF, carga de referência: P1 0-1 | P2 1-5 | P4 5-10 | P1 10-17 | P3 17-26, espera 6,50, turnaround 13,00, resposta 4,25.

SRTF, carga (1,0,7) (2,2,4) (3,4,1) (4,5,4): P1 0-2 | P2 2-4 | P3 4-5 | P2

5-7 | P4 7-11 | P1 11-16, espera média 3,00, turnaround médio 7,00.

2.4.7 Passo 7 — Round Robin

Use uma **fila FIFO** de índices de processos prontos. Em C, um vetor circular com *ini/fim* (ou um vetor grande com dois índices) resolve; em Python, `collections.deque`.

C | Fila circular simples (fragmento)

```
typedef struct { int itens[MAX_PROC]; int ini, fim, tam; } Fila;

static void enfileirar(Fila *f, int x) {
    f->itens[f->fim] = x;
    f->fim = (f->fim + 1) % MAX_PROC;
    f->tam++;
}
/* TODO: desenfileirar, fila_vazia */
```

Python | escalonador/rr.py (fragmento da lógica)

```
from collections import deque

fila = deque()
# ... admitir os processos com chegada <= t
while fila or ainda_ha_chegadas:
    # TODO: se fila vazia, saltar t para a próxima chegada e admitir
    p = fila.popleft()
    fatia = min(cfg.quantum, p.restante)
    # TODO: marcar inicio, registrar Gantt, avançar t, descontar restante
    # ATENÇÃO À ORDEM (convenção do roteiro):
    # 1) admitir quem chegou até t (inclusive)
    # 2) só então, se p.restante > 0, fila.append(p)
    ...
```

Atenção

O erro mais comum no RR é inverter a ordem “admitir chegadas” × “reinsere o preemptado”. Outro erro frequente: ordenar a fila a cada iteração — a fila do RR é FIFO, nunca é ordenada.

Verifique antes de seguir

Carga de referência, $q = 3$: P1 0-3 | P2 3-6 | P3 6-9 | P4 9-12 | P1 12-15 | P2 15-16 | P3 16-19 | P4 19-21 | P1 21-23 | P3 23-26; espera 13,50; turnaround 20,00; resposta 3,00; 9 trocas.

Outros quanta (espera/turnaround): $q = 1$: 12,50/19,00; $q = 2$: 12,75/19,25; $q = 4$: 11,75/18,25; $q = 10$: idêntico ao FCFS (8,75/15,25).

2.4.8 Passo 8 — Prioridade com *aging*

A estrutura é a do SJF (escolha dinâmica entre prontos), trocando o critério para `prio_efetiva`. O *aging* é aplicado a cada avanço do relógio aos processos que estão *na fila de prontos* (chegaram, não terminaram, não estão executando).

</> C | src/prioridade.c (fragmento do aging)

```

/* Chamada a cada unidade de tempo simulada.
   cfg->aging == 0 desativa o mecanismo. */
static void aplicar_aging(Processo v[], int n, int t, int executando,
                          const Config *cfg) {
    if (cfg->aging <= 0) return;
    for (int i = 0; i < n; i++) {
        if (i == executando || v[i].termino != -1 || v[i].chegada > t)
            continue;
        v[i].espera_fila++;
        /* TODO: a cada cfg->aging unidades, melhorar prio_efetiva
           (sem passar do limite, ex.: 0) e registrar em log */
    }
}

```

Python | Log do aging (fragmento)

```

if p.espera_fila % cfg.aging == 0 and p.prio_efetiva > 0:
    antiga = p.prio_efetiva
    p.prio_efetiva -= 1
    print(f'[t={t}] Aging: P{p.pid} prioridade {antiga} -> {p.prio_efetiva}')

```

👉 Dica

Se o algoritmo de prioridade for não preemptivo e você avançar o relógio em saltos de burst inteiro, o aging deve ser aplicado *acumulando* as unidades de espera do salto (`espera_fila += duração`) — ou avance tick a tick. Escolha uma forma e documente. Também deixe configurável se a versão é preemptiva ou não.

✅ Verifique antes de seguir

Carga (1,0,5,3) (2,1,3,1) (3,2,8,4) (4,3,6,2): P1 0-5 | P2 5-8 | P4 8-14 | P3 14-22, espera média 5,25. Com aging a cada 5 u.t., o log deve mostrar P3 melhorando de 4 para 3 em $t = 7$ e de 3 para 2 em $t = 12$.
Carga de referência (não preemptivo): mesma ordem do SJF, espera 7,75.

2.4.9 Passo 9 — Orquestração, saída e exportação CSV

O main lê os argumentos, carrega a carga *uma vez* e executa cada algoritmo sobre uma cópia. Uma tabela de “estratégias” deixa o código modular.

</> C | src/main.c (fragmento)

```

static const struct { const char *nome; FuncEscalonador f; } ALGS[] = {
    { "FCFS", escalonar_fcfs },
    { "SJF", escalonar_sjf },
    /* TODO: SRTF, RR, PRIO */
};
...
for (size_t a = 0; a < sizeof ALGS / sizeof ALGS[0]; a++) {
    Processo copia[MAX_PROG];
    memcpy(copia, originais, n * sizeof(Processo));
    /* TODO: executar, imprimir Gantt, tabela, métricas, CSV */
}

```

}

Python | main.py (fragmento)

```
import argparse, copy

ALGORITMOS = {'FCFS': escalonar_fcfs, 'SJF': escalonar_sjf} # TODO: demais

ap = argparse.ArgumentParser()
ap.add_argument('arquivo')
ap.add_argument('--alg', default='TODOS')
ap.add_argument('--quantum', type=int, default=2)
# TODO: --aging, --csv
...
for nome, func in ALGORITMOS.items():
    gantt = func(copy.deepcopy(originais), args)
    # TODO: imprimir e exportar
```

Formato CSV sugerido (uma linha por processo e algoritmo):

saidas/resultados.csv

```
carga,algoritmo,quantum,pid,chegada,burst,inicio,termino,espera,turnaround,resposta
carga1,FCFS,,1,0,8,0,8,0,8,0
carga1,FCFS,,2,1,4,8,12,7,11,7
...
```

2.4.10 Passo 10 — Robustez e qualidade

- Argumentos ausentes ou inválidos (`--quantum 0`, algoritmo desconhecido) geram mensagem de uso.
- Em C: compile com `-Wall -Wextra` sem avisos e verifique vazamentos com `valgrind`.
- Casos de borda: processo único; todos chegando em $t = 0$; CPU ociosa entre processos; bursts muito diferentes (1 vs. 50).

C | Makefile (trecho)

```
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11 -g -Iinclude
SRC      = $(wildcard src/*.c)
escalador: $(SRC)
    $(CC) $(CFLAGS) -o $@ $^
# TODO: alvos clean e test
```

2.5 Exemplos de entrada e saída

Use os exemplos abaixo como testes de regressão. Todos seguem as convenções deste roteiro (menor número = maior prioridade; desempate por chegada e PID; no RR, chegadas entram antes do preemptado).

2.5.1 FCFS

```
>_ ./escalonador entradas/processos.txt -alg FCFS
```

	P1	P2	P3	P4
0	8	12	21	26

PID	Chegada	Burst	Inicio	Termino	Espera	Turnaround	Resposta
1	0	8	0	8	0	8	0
2	1	4	8	12	7	11	7
3	2	9	12	21	10	19	10
4	3	5	21	26	18	23	18

Espera media: 8.75 | Turnaround medio: 15.25 | Resposta media: 8.75
Throughput: 0.154 proc/u.t. | Trocas de contexto: 3

2.5.2 SJF

```
>_ ./escalonador entradas/processos.txt -alg SJF
```

	P1	P2	P4	P3
0	8	12	17	26

Espera media: 7.75 | Turnaround medio: 14.25 | Trocas de contexto: 3

2.5.3 SRTF (carga que evidencia preempção)

```
entradas/srtf.txt
```

#	PID	Chegada	Burst	Prioridade
1	0	7	1	
2	2	4	1	
3	4	1	1	
4	5	4	1	

```
>_ ./escalonador entradas/srtf.txt -alg SRTF
```

	P1	P2	P3	P2	P4	P1
0	2	4	5	7	11	16

Espera media: 3.00 | Turnaround medio: 7.00 | Resposta media: 0.50

Em $t = 4$, P3 (burst 1) chega com restante menor que o de P2 (2) e o preemta.

2.5.4 Round Robin, quantum 3

```
>_ ./escalonador entradas/processos.txt -alg RR -quantum 3
```

	P1	P2	P3	P4	P1	P2	P3	P4	P1	P3
0	3	6	9	12	15	16	19	21	23	26

Espera media: 13.50 | Turnaround medio: 20.00 | Resposta media: 3.00
Trocas de contexto: 9

2.5.5 Prioridade não preemptiva com aging

entradas/prioridade.txt

#	PID	Chegada	Burst	Prioridade
1		0	5	3
2		1	3	1
3		2	8	4
4		3	6	2

```
>_ ./escalonador entradas/prioridade.txt -alg PRIO -aging 5
```

```
[t=7] Aging: P3 prioridade 4 -> 3 (esperando ha 5 u.t.)
[t=12] Aging: P3 prioridade 3 -> 2 (esperando ha 10 u.t.)
| P1 | P2 | P4 | P3 |
0 5 8 14 22
Espera media: 5.25 | Turnaround medio: 10.75
```

2.5.6 Casos de borda

- **Empate no SJF** — (1,0,4,1) e (2,0,4,2): P1 0-4 | P2 4-8 (desempate por PID).
- **Processo único** — (1,0,10,1): todos os algoritmos dão P1 0-10, espera 0, turnaround 10.
- **CPU ociosa** — (1,0,2,1) e (2,5,3,1): P1 0-2 | -- 2-5 | P2 5-8; espera média 0.

2.5.7 Tabela consolidada

```
>_ ./escalonador entradas/processos.txt -alg TODOS -quantum 3
```

Algoritmo	Espera	Turnaround	Resposta	Trocas
FCFS	8.75	15.25	8.75	3
SJF	7.75	14.25	7.75	3
SRTF	6.50	13.00	4.25	4
RR (q=3)	13.50	20.00	3.00	9
PRI0	7.75	14.25	7.75	3

2.6 Experimentos

1. **Cargas de trabalho.** Prepare ao menos três (idealmente as seis abaixo), com um gerador que use **semente fixa** (documentada no relatório):
 - pequena (5 processos), conferível à mão;
 - CPU-bound (bursts 15–40) e I/O-bound (bursts 1–5), 20 processos cada;
 - mista, 50 processos;
 - todos chegando em $t = 0$;
 - um processo muito longo chegando primeiro (efeito comboio);
 - adversarial para inanição: um processo de prioridade baixa (ou burst longo) e um fluxo contínuo de processos curtos/prioritários.

2. Todos os algoritmos $\times$ todas as cargas.

3. **Sensibilidade ao quantum.** RR com $q \in \{1, 2, 4, 8, 16\}$: registre espera, resposta e número de trocas. Espera-se ver a resposta crescer e as trocas caírem com q , até o RR coincidir com o FCFS.
4. **Efeito do aging.** Na carga adversarial, rode Prioridade *com* e *sem* aging e compare o tempo de espera do processo de menor prioridade.

 Python | `gerador_cargas.py(fragmento)`

```
import random

def gerar(perfil, n, semente):
    rng = random.Random(semente)      # reprodutível!
    faixas = {'cpu': (15, 40), 'io': (1, 5)}
    linhas = []
    for pid in range(1, n + 1):
        # TODO: sortear chegada, burst (conforme perfil) e prioridade 1..5
        ...
    # TODO: ordenar por chegada e escrever no formato de entrada
```

 Python | `analise.py (fragmento do gráfico)`

```
import csv
import matplotlib.pyplot as plt

# TODO: ler saidas/resumo.csv e montar {algoritmo: espera_media}
algoritmos = [...]
esperas = [...]

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(algoritmos, esperas)
ax.set_ylabel('Tempo médio de espera (u.t.)')
ax.set_title('Carga mista, 50 processos, semente 42')
# TODO: repetir para turnaround/resposta; gráfico de linha para o quantum
fig.savefig('saidas/espera.pdf', bbox_inches='tight')
```

Para a carga de referência, o gráfico de sensibilidade ao quantum fica assim (valores desta carga; os seus experimentos devem usar cargas maiores):

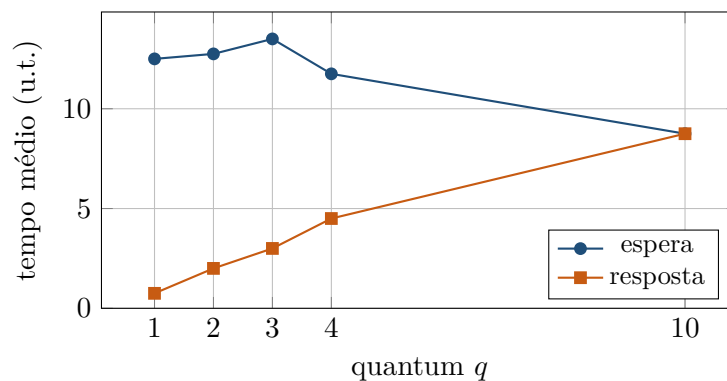


Figura 2.6: RR na carga de referência: a resposta cresce com o quantum; para $q \geq 9$ o RR vira FCFS.

⚠ Atenção

Não confie em uma única carga pequena para tirar conclusões: note que a espera *não* é monotônica em q neste exemplo de 4 processos. Com 50 processos as tendências ficam muito mais claras. Todo gráfico deve ter título, rótulos nos eixos com unidade e identificar a carga usada.

2.7 Perguntas para o relatório

Responda na seção de Discussão, sempre com **evidência experimental** (tabelas e gráficos seus):

1. Qual algoritmo minimizou o tempo médio de espera em cada carga de teste? Por quê? Relacione com a otimalidade do SJF/SRTF.
2. Como o tamanho do quantum no Round Robin afeta o tempo de resposta e o número de trocas de contexto (*overhead*)? Qual quantum você recomendaria para cada carga?
3. Em que cenário o SJF pode causar *starvation*? Mostre o cenário e explique como o *aging* o mitiga, comparando os tempos de espera com e sem aging.
4. Onde o efeito comboio apareceu nos seus experimentos com FCFS?
5. Quais convenções de desempate e de ordem de fila você adotou e como elas influenciam os resultados?
6. Limitações do simulador: ausência de custo de troca de contexto, de E/S e de múltiplas CPUs. Como cada uma mudaria as conclusões?

2.8 Critérios de avaliação e checklist

Critério	Peso
Corretude de cada algoritmo (FCFS, SJF, SRTF, RR, PRIO)	50% (10% cada)
Cálculo correto das métricas	20%
Qualidade da análise comparativa	20%
Organização e modularidade do código	10%

Checklist de entrega

- ☐ Compila/executa em Linux; **make** gera o executável (C) ou roda o programa (Python); **make clean** limpa.
- ☐ Em C: sem avisos com **-Wall -Wextra**; sem vazamentos no **valgrind**.
- ☐ Leitura correta do arquivo, com tratamento de arquivo inexistente/malformatado.
- ☐ FCFS, SJF, SRTF, RR (quantum configurável) e Prioridade (aging configurável) corretos — todos os exemplos da Seção 2.5 conferem.
- ☐ Gantt, tabela por processo e métricas agregadas (inclusive resposta, throughput e trocas de contexto).
- ☐ Cada algoritmo em arquivo/módulo separado; execução sobre cópias da carga.
- ☐ Cargas variadas com semente documentada; variação de quantum; experimento de aging.

- ☐ Gráficos com título, legenda e rótulos.
- ☐ Relatório no Overleaf (modelo SBC), com fontes `.tex` e PDF; perguntas respondidas.
- ☐ Código e relatório de autoria própria (sem plágio e sem geração por IA).

2.9 Leituras recomendadas

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. Capítulo “Escalonamento de CPU”.
- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. Cap. 2, seção “Escalonamento”.
- STALLINGS, W. *Sistemas Operacionais: Princípios e Projeto*. Capítulo “Escalonamento de Uniprocessadores”.
- ARPACI-DUSSEAU, R.; ARPACI-DUSSEAU, A. *Operating Systems: Three Easy Pieces*. Capítulos *Scheduling: Introduction* e *Multi-Level Feedback Queue* (gratuitos em <https://pages.cs.wisc.edu/~remzi/OSTEP/>).
- Documentação Python: `collections.deque`, `heapq`, `dataclasses`; Matplotlib (`bar`, `broken_barh` para Gantt).

Capítulo 3

Simulador de Paginação

3.1 Visão geral

Todo programa em execução precisa estar, ao menos em parte, na memória principal (RAM). Acontece que a RAM é limitada e precisa ser compartilhada por dezenas ou centenas de processos ao mesmo tempo. O sistema operacional resolve esse problema com uma técnica chamada **memória virtual paginada**: cada processo enxerga um espaço de endereçamento próprio, dividido em pedaços de tamanho fixo (*páginas*), e só as páginas realmente necessárias ficam carregadas em pedaços correspondentes da memória física (*quadros* ou *frames*). Quando o processo acessa uma página que não está na RAM ocorre uma **falta de página** (*page fault*), e se não houver quadro livre o SO precisa escolher uma página “vítima” para retirar da memória.

Neste trabalho você vai construir um **simulador** que recebe uma sequência de referências a páginas e um número de quadros disponíveis, e reproduz o comportamento de diferentes **algoritmos de substituição de página** (FIFO, LRU, Ótimo e Segunda Chance, e opcionalmente LFU), contando as faltas de página e exibindo o estado da memória a cada passo. Com o simulador pronto, você fará experimentos variando o número de quadros e observará fenômenos clássicos, como a **Anomalia de Belady**.

🕒 Ao final deste trabalho você deverá ser capaz de

- Explicar a diferença entre endereço lógico (virtual) e endereço físico, e o papel da MMU.
- Descrever como a paginação divide a memória em páginas e quadros e como a tabela de páginas traduz endereços.
- Explicar o que é uma falta de página e quais passos o SO executa para tratá-la.
- Implementar e comparar os algoritmos de substituição FIFO, LRU, Ótimo e Segunda Chance.
- Reproduzir e explicar a Anomalia de Belady.
- Relacionar o princípio da localidade com o desempenho dos algoritmos.

⚠️ Atenção

Este é um trabalho de **simulação**: você não vai mexer na memória real do computador nem no núcleo do Linux. O simulador trabalha apenas com *números de página* lidos de um arquivo. Toda a “memória física” é um simples vetor de quadros dentro do seu programa.

3.2 Fundamentação teórica

3.2.1 A hierarquia de memória

Um computador possui vários tipos de memória, que diferem em velocidade, capacidade e custo. Quanto mais próxima do processador, mais rápida, menor e mais cara ela é. A Figura 3.1 ilustra essa organização, chamada de **hierarquia de memória**.

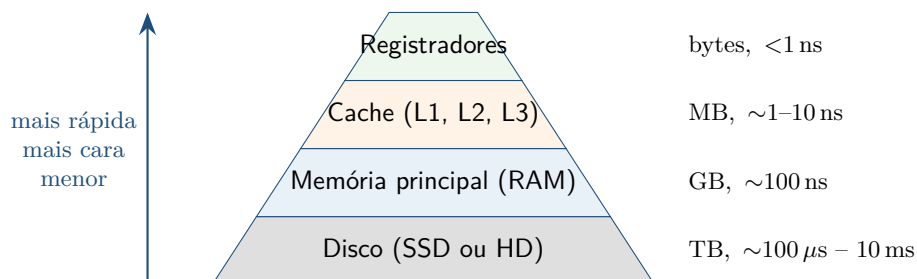


Figura 3.1: Hierarquia de memória: velocidade, capacidade e custo por byte.

O processador só executa instruções e lê dados que estejam na RAM (passando pelas caches). O disco é muito maior, mas é entre mil e cem mil vezes mais lento. A ideia central da **memória virtual** é usar a RAM como uma espécie de “cache” do disco: mantém-se na RAM apenas o que está sendo usado agora e deixa-se o resto no disco. A arte está em decidir *o que* manter na RAM — e é exatamente isso que os algoritmos de substituição de página fazem.

3.2.2 Endereço lógico, endereço físico e a MMU

Quando você escreve `int x = v[10];` em C, o compilador gera uma instrução que lê um endereço de memória. Esse endereço **não** é a posição real na RAM: é um **endereço lógico** (ou **virtual**), relativo ao espaço de endereçamento do processo.

Definição: Endereço lógico e endereço físico

Endereço lógico (virtual) é o endereço gerado pela CPU ao executar o programa; cada processo tem seu próprio espaço de endereços lógicos, que normalmente começa em 0. **Endereço físico** é a posição real de um byte nos chips de RAM. A conversão de lógico para físico é feita, a cada acesso, por um componente de hardware chamado **MMU** (*Memory Management Unit*), configurado pelo sistema operacional.

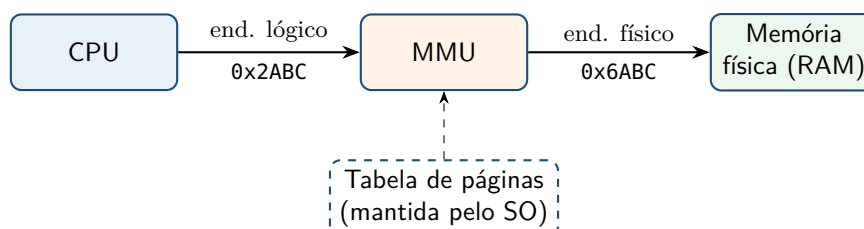


Figura 3.2: A MMU traduz cada endereço lógico em físico consultando a tabela de páginas.

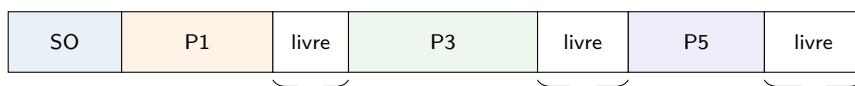
Essa separação traz três grandes vantagens:

1. **Isolamento:** um processo não consegue acessar a memória de outro, porque só enxerga seus próprios endereços lógicos.

2. **Realocação:** o SO pode colocar o processo em qualquer lugar da RAM sem recompilar o programa.
3. **Memória virtual:** o processo pode ser maior do que a RAM disponível, porque partes dele podem ficar no disco.

3.2.3 Por que não alocar a memória de forma contígua?

A solução mais simples seria dar a cada processo um único bloco contínuo de RAM. O problema é a **fragmentação externa**: à medida que processos entram e saem, a memória livre fica picotada em buracos pequenos, espalhados entre processos. Pode haver 100 MB livres no total, mas nenhum buraco contínuo de 30 MB — e aí um processo de 30 MB não consegue ser carregado.



buracos livres somam espaço suficiente, mas nenhum sozinho cabe um novo processo

Figura 3.3: Fragmentação externa na alocação contígua.

A **paginação** elimina a fragmentação externa: em vez de exigir um bloco contínuo, o processo é quebrado em pedaços de tamanho fixo que podem ser colocados em *quaisquer* quadros livres, em qualquer ordem. (O Capítulo 4, do alocador de memória, explora a fragmentação em detalhe.)

3.2.4 Páginas e quadros

Definição: Página e quadro

O espaço de endereçamento **lógico** de um processo é dividido em blocos de tamanho fixo chamados **páginas**. A memória **física** é dividida em blocos do mesmo tamanho chamados **quadros** (*frames* ou *molduras*). Qualquer página pode ser carregada em qualquer quadro livre. Tamanhos típicos: 4 KiB (o padrão do Linux em x86-64), podendo existir páginas “enormes” de 2 MiB ou 1 GiB.

Como página e quadro têm o mesmo tamanho, basta saber *em qual quadro* está cada página: o deslocamento (*offset*) dentro da página é o mesmo dentro do quadro.

3.2.5 A tabela de páginas e a tradução de endereços

Se o tamanho da página é uma potência de 2, digamos 2^d bytes, um endereço lógico de m bits pode ser dividido em duas partes:

- os $m - d$ bits mais significativos formam o **número da página** (p);
- os d bits menos significativos formam o **deslocamento** (d) dentro da página.

A MMU usa p como índice na **tabela de páginas** do processo, obtém o número do quadro f e forma o endereço físico concatenando f com o mesmo deslocamento.

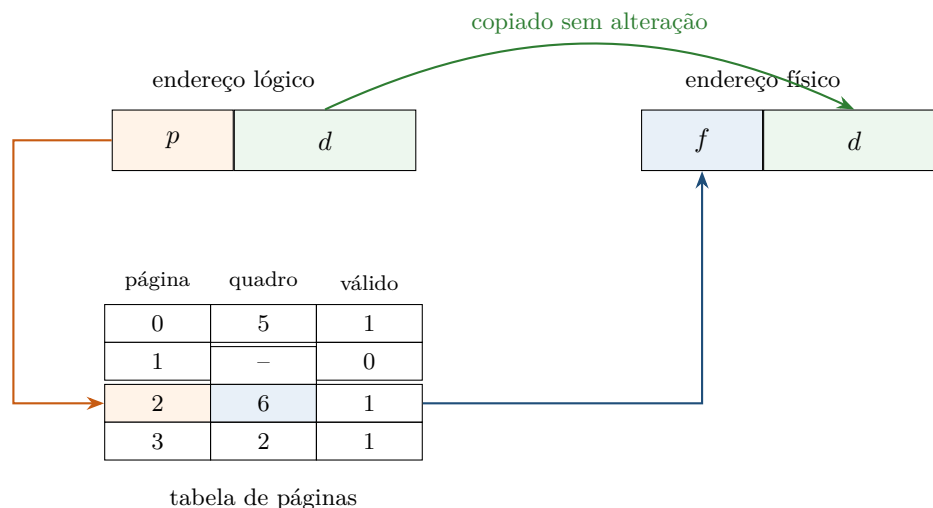


Figura 3.4: Tradução de endereço com tabela de páginas: o número da página é trocado pelo número do quadro; o deslocamento é mantido.

💡 Exemplo numérico de tradução

Considere endereços lógicos de 16 bits e páginas de 4 KiB (2^{12} bytes). Então $d = 12$ bits de deslocamento e $16 - 12 = 4$ bits para o número da página (até 16 páginas).

O endereço lógico $0x2ABC$ em binário é:

$$\underbrace{0010}_{p=2} \underbrace{1010\ 1011\ 1100}_{d=0xABC}$$

Consultando a tabela da Figura 3.4, a página 2 está no quadro 6 (0110). O endereço físico é, portanto,

$$\underbrace{0110}_{f=6} \underbrace{1010\ 1011\ 1100}_d = 0x6ABC.$$

Se o processo acessasse o endereço $0x1004$ (página 1), a MMU encontraria o bit *válido* igual a 0 e geraria uma **falta de página**.

Em notação aritmética: $p = \lfloor \text{endereço} / \text{tam_página} \rfloor$ e $d = \text{endereço} \bmod \text{tam_página}$. No simulador deste trabalho a entrada já vem na forma de *números de página*, mas é importante entender de onde eles vêm. Se quiser, você pode aceitar também endereços e convertê-los assim:

🔧 C | de endereço para número de página

```
#define TAM_PAGINA 4096
unsigned pagina = endereco / TAM_PAGINA; /* ou endereco >> 12 */
unsigned desloc = endereco % TAM_PAGINA; /* ou endereco & 0xFFF */
```

🐍 Python | de endereço para número de página

```
TAM_PAGINA = 4096
pagina, desloc = divmod(endereco, TAM_PAGINA)
```

3.2.6 Os bits de uma entrada da tabela de páginas

Além do número do quadro, cada entrada da tabela de páginas (PTE, *Page Table Entry*) guarda alguns bits de controle, atualizados pelo hardware ou pelo SO:

Bit	Significado
Válido/presente	1 se a página está carregada em algum quadro; 0 se está no disco (ou nem existe). Acesso a página com bit 0 → falta de página.
Referência (<i>accessed</i> , R)	Ligado pelo hardware sempre que a página é lida ou escrita. O SO pode zerá-lo periodicamente. Base da Segunda Chance.
Modificado (<i>dirty</i> , M)	Ligado quando a página é <i>escrita</i> . Se uma página suja for escolhida como vítima, ela precisa ser gravada no disco antes; se estiver limpa, basta descartá-la.
Proteção	Permissões (leitura, escrita, execução, acesso só em modo núcleo).

3.2.7 TLB: acelerando a tradução

A tabela de páginas fica na própria RAM. Se toda tradução exigisse ler a tabela, cada acesso à memória custaria *dois* acessos. Para evitar isso, a MMU tem uma pequena memória associativa muito rápida, a **TLB** (*Translation Lookaside Buffer*), que guarda as traduções usadas recentemente. Se a tradução está na TLB (*TLB hit*) o custo extra é quase nulo; se não está (*TLB miss*), a tabela é consultada e a TLB é atualizada.

💡 Exemplo: tempo de acesso com TLB

Suponha acesso à RAM de 100 ns, consulta à TLB de 20 ns e taxa de acerto da TLB de 80%. Então:

$$T = 0,8 \times (20 + 100) + 0,2 \times (20 + 100 + 100) = 96 + 44 = 140 \text{ ns.}$$

Sem TLB seriam sempre 200 ns. A TLB funciona bem pelo mesmo motivo que os algoritmos de substituição funcionam: o **princípio da localidade** (Seção 3.2.11).

A TLB não faz parte do simulador obrigatório, mas note que ela também é uma pequena memória que precisa de política de substituição — frequentemente uma aproximação de LRU.

3.2.8 Memória virtual e paginação sob demanda

Com a tabela de páginas e o bit *válido*, o SO pode deixar de carregar o programa inteiro na RAM. Na **paginação sob demanda** (*demand paging*), uma página só é trazida do disco para a RAM quando é acessada pela primeira vez. As páginas que ainda não foram carregadas (ou que foram retiradas) ficam em uma área do disco chamada **área de troca** (*swap*).

📖 Definição: Falta de página (page fault)

Uma **falta de página** é uma exceção gerada pela MMU quando o processo acessa uma página cujo bit *válido* é 0. A CPU interrompe a instrução e transfere o controle ao SO, que carrega a página do disco e depois *reexecuta* a instrução que falhou. Para o processo, tudo é transparente — ele só fica mais lento.

3.2.9 Tratamento de uma falta de página, passo a passo

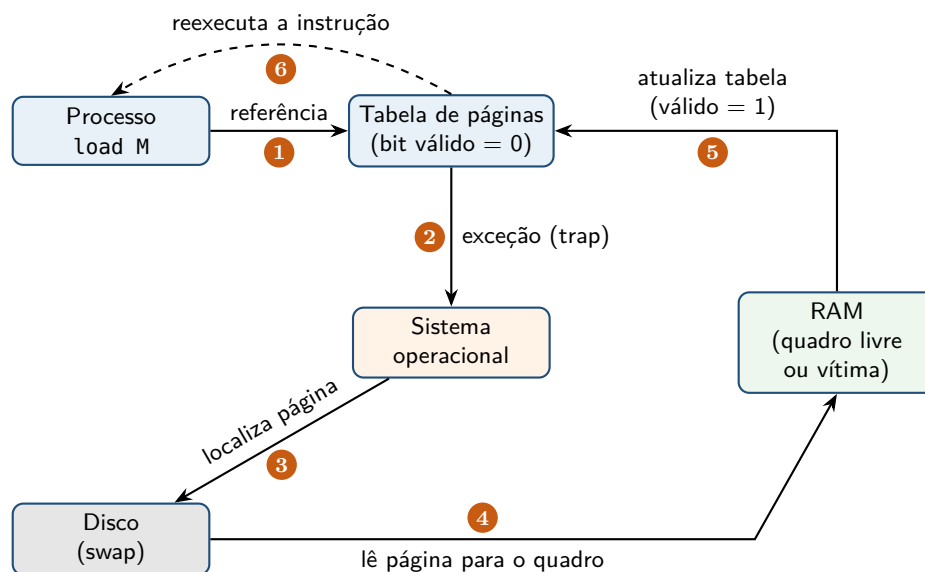


Figura 3.5: Etapas do tratamento de uma falta de página.

1. O processo referencia um endereço; a MMU consulta a tabela de páginas.
2. O bit *válido* é 0: a MMU gera uma exceção (*trap*) e o SO assume o controle.
3. O SO verifica se o acesso é legítimo (se não for, o processo recebe **SIGSEGV** — o famoso *segmentation fault*) e localiza a página no disco.
4. O SO escolhe um quadro livre. **Se não houver quadro livre, executa o algoritmo de substituição** para escolher uma vítima; se a vítima estiver suja (bit *M* = 1), grava-a no disco. Depois lê a página desejada para o quadro.
5. A tabela de páginas é atualizada (quadro, válido = 1; a entrada da vítima passa a válido = 0).
6. A instrução interrompida é reexecutada, agora com sucesso.

O seu simulador reproduz, de forma abstrata, os passos 2, 4 e 5: detectar a falta, escolher a vítima e atualizar o estado dos quadros.

3.2.10 Tempo efetivo de acesso

Faltas de página são *extremamente* caras, pois envolvem o disco. Seja p a probabilidade de uma referência causar falta de página ($0 \leq p \leq 1$), t_{mem} o tempo de acesso à memória e t_{falta} o tempo de tratamento da falta. O **tempo efetivo de acesso** (EAT, *Effective Access Time*) é:

$$EAT = (1 - p) \cdot t_{mem} + p \cdot t_{falta}.$$

💡 Exemplo: o impacto de uma taxa de faltas “pequena”

Com $t_{mem} = 100$ ns e $t_{falta} = 8$ ms = 8 000 000 ns, e apenas uma falta a cada mil acessos ($p = 0,001$):

$$EAT = 0,999 \times 100 + 0,001 \times 8\,000\,000 \approx 8\,100 \text{ ns.}$$

O computador fica cerca de **80 vezes mais lento!** Para que a degradação seja menor que 10% ($EAT < 110\text{ ns}$), é preciso $p < 1,25 \times 10^{-6}$, ou seja, menos de uma falta a cada 800 000 acessos. Por isso a escolha do algoritmo de substituição importa tanto: cada falta evitada economiza milhões de ciclos.

3.2.11 Princípio da localidade, working set e thrashing

Se os programas acessassem a memória de forma totalmente aleatória, nenhum algoritmo conseguiria prever quais páginas serão usadas e a memória virtual seria inviável. Felizmente, programas reais apresentam **localidade de referência**:

- **Localidade temporal:** um endereço acessado agora tende a ser acessado de novo em breve (variáveis de laço, pilha, código de um laço).
- **Localidade espacial:** endereços próximos a um endereço acessado tendem a ser acessados em breve (percorrer um vetor, executar instruções em sequência).

Definição: Working set (conjunto de trabalho)

O **working set** $W(t, \Delta)$ de um processo é o conjunto de páginas distintas referenciadas nas últimas Δ referências até o instante t . Intuitivamente, é o conjunto de páginas que o processo está “usando de verdade” naquela fase da execução. Se o working set cabe nos quadros alocados ao processo, as faltas são raras.

Quando a soma dos working sets de todos os processos é maior que a memória física, cada processo, ao executar, tira da memória páginas de que outro (ou ele mesmo) precisará logo em seguida. O sistema passa quase todo o tempo trocando páginas com o disco e quase nada executando código útil. Esse colapso chama-se **thrashing** (ou “paginação excessiva”). A Figura 3.6 mostra o efeito típico.

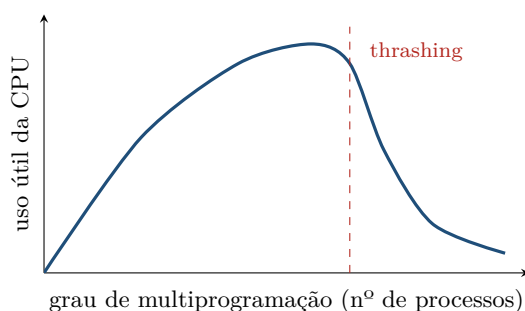


Figura 3.6: Thrashing: a partir de certo ponto, colocar mais processos na memória derruba a utilização da CPU.

3.2.12 Algoritmos de substituição de página

Quando ocorre uma falta de página e todos os quadros estão ocupados, o SO precisa escolher uma **página vítima**. O objetivo é minimizar o número total de faltas. Todos os algoritmos abaixo são avaliados da mesma forma: dada uma **sequência de referências** (*reference string*) e um número de quadros, conta-se quantas faltas ocorrem. Usaremos como exemplo a sequência clássica

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

com 3 quadros (20 referências, 6 páginas distintas: 0, 1, 2, 3, 4, 7).

⚠ Atenção

As primeiras faltas, que acontecem enquanto ainda há quadros livres, são chamadas **faltas compulsórias** (ou “de partida a frio”): nenhum algoritmo consegue evitá-las, pois a página nunca esteve na memória. Os algoritmos só diferem quanto às faltas que acontecem *depois* que a memória enche.

FIFO (First-In, First-Out)

Regra: substitui a página que está há *mais tempo na memória* (a que foi carregada primeiro), não importa se ela foi usada recentemente. Implementa-se com uma fila: toda página carregada entra no fim; a vítima é sempre a do início.

- **Vantagem:** simplíssimo, custo $O(1)$ por falta.
- **Desvantagem:** ignora o uso. Uma página muito usada (por exemplo, a que contém uma variável global) será expulsa só por ser “velha”. Além disso, sofre da Anomalia de Belady.

Na sequência de exemplo com 3 quadros, o FIFO produz **15 faltas** (traço completo na Seção 3.5).

Ótimo (OPT, algoritmo de Belady)

Regra: substitui a página que *será usada mais tarde no futuro* (ou que nunca mais será usada). Belady provou em 1966 que essa escolha produz o **menor número possível de faltas** para qualquer sequência e qualquer número de quadros.

- **Problema:** exige conhecer o futuro. Um SO real não sabe quais endereços o programa vai acessar. Por isso o Ótimo é **irrealizável** na prática e serve apenas como **limite inferior** de comparação: “meu algoritmo gerou 12 faltas; o melhor possível seria 9”.
- Em um simulador ele é implementável, porque a sequência inteira está no arquivo.

Na sequência de exemplo com 3 quadros o Ótimo produz **9 faltas**. Veja a primeira decisão: na referência à página 2 (4.^a referência), os quadros contêm {7, 0, 1}. As próximas ocorrências são: 0 na posição 5, 1 na posição 14, 7 na posição 18. A página 7 é a que será usada mais tarde, então é a vítima.

LRU (Least Recently Used)

Regra: substitui a página que *não é usada há mais tempo*. A ideia é usar o passado recente como previsão do futuro próximo (localidade temporal): se uma página não é usada há muito tempo, provavelmente não será usada logo. O LRU é como um “Ótimo olhando para trás”.

Diferença fundamental em relação ao FIFO: no FIFO, um *acerto* (*hit*) não muda nada; no LRU, todo acerto **atualiza** a informação de uso da página, protegendo-a de ser escolhida.

Na sequência de exemplo com 3 quadros o LRU produz **12 faltas** — entre o FIFO (15) e o Ótimo (9).

Como implementar o LRU. Há duas abordagens clássicas:

1. **Contador (carimbo de tempo):** mantém-se um relógio lógico global incrementado a cada referência; cada quadro guarda o valor do relógio no seu último uso. Na substituição,

procura-se o quadro com o *menor* carimbo. Atualizar é $O(1)$; escolher a vítima exige varrer os quadros, $O(n)$.

2. **Pilha / lista ordenada por uso:** mantém-se uma lista em que a página mais recentemente usada fica no topo. A cada referência, a página é movida para o topo; a vítima é sempre a do fundo. Com uma lista duplamente encadeada *mais* uma tabela de dispersão (*hash*) que aponta para os nós, todas as operações ficam $O(1)$ — é a mesma estrutura de um “cache LRU”.

Dica

Em hardware real, manter o LRU exato a cada acesso à memória é caro demais (seria preciso atualizar uma estrutura a cada instrução). Por isso os sistemas operacionais usam **aproximações** do LRU baseadas no bit de referência, como a Segunda Chance.

Aproximações do LRU: bit de referência e Segunda Chance (relógio)

O hardware liga o **bit de referência** (R) de uma página sempre que ela é acessada. O SO não sabe *quando* foi o acesso, mas sabe *se houve* acesso desde a última vez que zerou o bit. Isso já basta para uma boa aproximação.

Segunda Chance: é um FIFO com uma pequena modificação. Na hora de escolher a vítima, examina-se a página mais antiga (início da fila):

- se o bit R dela é **0**, ela é a vítima;
- se o bit R é **1**, ela ganha uma “segunda chance”: o bit é zerado e a página vai para o fim da fila (como se tivesse acabado de chegar); examina-se a próxima.

Se todas as páginas tiverem $R = 1$, o algoritmo dá uma volta completa zerando todos os bits e acaba escolhendo a página original do início — degenerando para FIFO.

Algoritmo do relógio (clock): é a mesma lógica, implementada de forma mais eficiente. Em vez de mover páginas numa fila, os quadros são dispostos num *vetor circular* e um **ponteiro** (o “ponteiro do relógio”) indica a página mais antiga. Dar segunda chance é só zerar o bit e avançar o ponteiro.

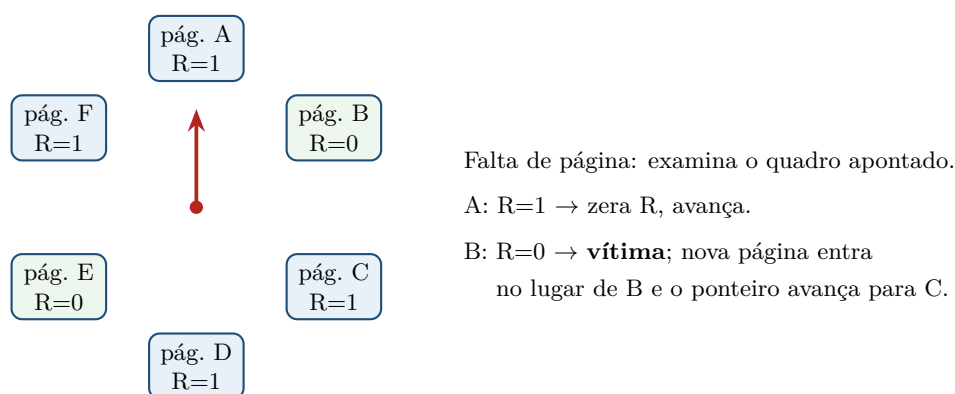


Figura 3.7: Algoritmo do relógio (Segunda Chance com vetor circular).

Atenção

Convenção adotada neste roteiro: quando uma página é *carregada* por uma falta, seu bit R é ligado ($R = 1$), porque ela acabou de ser referenciada; quando ocorre um *acerto*, o bit também é ligado. Com essa convenção a Segunda Chance produz **14 faltas** na sequência

de exemplo com 3 quadros. Alguns livros carregam a página com $R = 0$; nesse caso o resultado com 3 quadros passa a ser **11 faltas**. As duas convenções são aceitáveis, *desde que você documente a sua no relatório* e use-a de forma consistente.

LFU (*Least Frequently Used*) — desejável

Regra: substitui a página com o *menor número de acessos* desde que foi carregada. Em caso de empate, usa-se um critério secundário (por exemplo, FIFO ou LRU entre as empatadas — documente o seu).

Problema clássico: uma página muito usada no início do programa acumula um contador alto e “nunca mais sai”, mesmo que não seja mais usada. Variantes resolvem isso “envelhecendo” os contadores (dividindo-os por 2 periodicamente).

Resumo comparativo

Algoritmo	Vítima	Informação necessária	Faltas (3 quadros)
FIFO	mais antiga na memória	ordem de chegada	15
Segunda Chance	mais antiga com $R = 0$	ordem de chegada + bit R	14
LRU	usada há mais tempo	instante do último uso	12
Ótimo	usada mais longe no futuro	toda a sequência futura	9
LFU	menos acessada	contador de acessos	(experimental!)

3.2.13 A Anomalia de Belady e os algoritmos de pilha

Intuitivamente, mais memória deveria significar menos (ou, no máximo, as mesmas) faltas de página. Em 1969, Belady, Nelson e Shedler mostraram que isso **não é verdade para o FIFO**. Com a sequência

1 2 3 4 1 2 5 1 2 3 4 5

o FIFO produz **9 faltas com 3 quadros** e **10 faltas com 4 quadros**. Aumentar a memória piorou o desempenho! Esse fenômeno é a **Anomalia de Belady**.

Referência	1	2	3	4	1	2	5	1	2	3	4	5
FIFO, 3 quadros	F	F	F	F	F	F	F	–	–	F	F	–
FIFO, 4 quadros	F	F	F	F	–	–	F	F	F	F	F	F

F = falta de página; – = acerto. Total: 9 faltas com 3 quadros, 10 com 4 quadros.

Por que acontece? Com 3 quadros, quando a página 5 chega (7.^a referência), as páginas 1 e 2 tinham acabado de ser recarregadas e ficam na memória, gerando dois acertos logo em seguida. Com 4 quadros, as páginas 1 e 2 acertaram nas referências 5 e 6 — mas, como o FIFO *ignora acertos*, elas continuam sendo as “mais antigas” e são expulsas justamente antes de serem usadas de novo. O conjunto de páginas na memória com 4 quadros *não contém* o conjunto que estaria na memória com 3 quadros.

Definição: Algoritmo de pilha (stack algorithm)

Um algoritmo de substituição é um **algoritmo de pilha** se, para toda sequência de referências e todo instante, o conjunto de páginas na memória com n quadros está *contido* no conjunto de páginas na memória com $n + 1$ quadros. Algoritmos de pilha **nunca** sofrem a Anomalia de Belady, porque tudo que é acerto com n quadros também é acerto com $n + 1$. LRU e Ótimo são algoritmos de pilha; FIFO e Segunda Chance **não** são.

No LRU, com $n+1$ quadros a memória sempre contém as $n+1$ páginas usadas mais recentemente, que obviamente incluem as n usadas mais recentemente. Na sequência de Belady o LRU produz 10 faltas com 3 quadros e 8 com 4; o Ótimo, 7 e 6.

3.3 Especificação do trabalho

3.3.1 Entrada

O simulador recebe:

1. um arquivo com a **sequência de referências a páginas** (números inteiros não negativos), uma por linha ou separadas por espaços; linhas vazias e linhas iniciadas por # devem ser ignoradas;
2. o **número de quadros** (ou uma lista/faixa de valores, ex.: de 1 a 7);
3. opcionalmente, o algoritmo a executar (ou todos, por padrão).

entradas/sequencia1.txt

```
# sequência clássica (Silberschatz)
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
```

Sugestão de linha de comando (adapte à sua implementação e documente):

>_ Exemplos de chamada

```
$ ./paginacao entradas/sequencia1.txt 3          # todos os algoritmos, 3 quadros
$ ./paginacao entradas/sequencia1.txt 1-7 --csv  # faixa de quadros, gera CSV
$ python3 paginacao.py entradas/sequencia1.txt 3 --alg lru
```

3.3.2 Algoritmos

Algoritmo	Obrigatório?	Observação
FIFO	sim	base de comparação
LRU	sim	comentar a complexidade da implementação escolhida
Ótimo (Belady)	sim	limite teórico; usa a sequência futura
Segunda Chance	sim	documentar a convenção do bit R
LFU	desejável	documentar o critério de desempate

3.3.3 Saídas

Para cada algoritmo e cada número de quadros:

1. o **traço de execução**: para cada referência, o estado dos quadros após o acesso, se houve falta e, em caso de substituição, qual página saiu;
2. o **total de faltas** e a **taxa de faltas** (faltas/referências $\times 100\%$);
3. uma **tabela consolidada** (quadros $\times$ algoritmo), exportada em CSV, que alimentará o gráfico de faltas por número de quadros.

3.3.4 Estrutura de entrega

Versão em C

```
paging-simulator/
|-- Makefile
|-- src/
|   |-- main.c
|   |-- leitor.c / leitor.h
|   |-- quadros.c / quadros.h
|   |-- traco.c / traco.h
|   |-- fifo.c
|   |-- lru.c
|   |-- otimo.c
|   |-- segunda_chance.c
|   '-- algoritmos.h
|-- entradas/
|   |-- sequencial.txt
|   '-- sequencia2.txt
|-- resultados/ (CSV gerados)
|-- analise.py (gráficos)
'-- relatorio/ (.tex + PDF)
```

Versão em Python

```
paging-simulator/
|-- Makefile (alvos run, test)
|-- src/
|   |-- main.py
|   |-- leitor.py
|   |-- quadros.py
|   |-- traco.py
|   |-- fifo.py
|   |-- lru.py
|   |-- otimo.py
|   '-- segunda_chance.py
|-- entradas/
|   |-- sequencial.txt
|   '-- sequencia2.txt
|-- resultados/ (CSV gerados)
|-- analise.py (gráficos)
'-- relatorio/ (.tex + PDF)
```

3.4 Roteiro de implementação

O roteiro abaixo segue uma ordem pensada para que cada passo possa ser testado antes do próximo. Os trechos de código são **fragmentos ilustrativos**: mostram estruturas, assinaturas e a “forma” da solução, mas deixam as partes essenciais para você (marcadas com **TODO**). Escolha uma linguagem e siga apenas os quadros dela.

⚠ Atenção

Os trechos aqui apresentados são pontos de partida, não soluções. O código entregue deve ser escrito por você, e você deve ser capaz de explicar cada linha em uma eventual arguição oral.

3.4.1 Passo 1 — Estrutura de dados da memória física

Objetivo: representar o conjunto de quadros e saber quais páginas estão carregadas.

Precisamos de:

- um vetor de tamanho fixo (`n_quadros`) em que cada posição guarda o número da página carregada, ou um valor especial (ex.: `-1` / `None`) para quadro vazio;
- informação auxiliar por quadro, que alguns algoritmos usam: instante do último uso (LRU), bit de referência (Segunda Chance), contador de acessos (LFU);
- uma forma de responder “a página x está na memória? em qual quadro?”.

C | quadros.h — estado da memória física

```
#define VAZIO -1

typedef struct {
    int pagina;           /* página carregada ou VAZIO */
    int ultimo_uso;       /* relógio lógico do último acesso (LRU) */
    int bit_ref;          /* bit de referência (Segunda Chance) */
    int contador;         /* número de acessos (LFU, opcional) */
} Quadro;

typedef struct {
    Quadro *q;            /* vetor com n_quadros posições */
    int n_quadros;
    int ocupados;
} Memoria;

Memoria *memoria_criar(int n_quadros); /* todos VAZIO */
void     memoria_destruir(Memoria *m);
int      memoria_buscar(const Memoria *m, int pagina); /* índice ou -1 */
```

Python | quadros.py — estado da memória física

```
VAZIO = None

class Memoria:
    def __init__(self, n_quadros):
        self.quadros = [VAZIO] * n_quadros # página em cada quadro
        self.posicao = {}                   # página -> índice do quadro
        # TODO: estruturas auxiliares que seus algoritmos precisarem

    def buscar(self, pagina):
        """Retorna o índice do quadro ou None."""
        ... # TODO
```

Dica

Buscar linearmente no vetor de quadros é $O(n)$ por referência, o que é perfeitamente aceitável para dezenas de quadros. Em Python, o dicionário `posicao` dá busca em $O(1)$ — mas lembre-se de mantê-lo sincronizado com o vetor em toda carga e remoção. Em C, se as páginas forem números pequenos, um vetor `onde[pagina]` faz o mesmo papel. Comente a sua escolha no relatório.

Atenção

Mantenha as posições dos quadros fixas. Quando uma página substitui outra, ela ocupa *o mesmo índice* do quadro da vítima. Assim o traço impresso fica comparável com os exemplos deste roteiro (ex.: `[2,0,1]` após a página 2 substituir a 7 no quadro 0). A

ordem de chegada (FIFO) deve ser guardada em uma estrutura separada, não pela posição no vetor.

✓ Verifique antes de seguir

Crie uma memória com 3 quadros e imprima-a: deve aparecer [-, -, -]. Carregue manualmente as páginas 7 e 0 e confirme que `buscar(0)` devolve 1 e `buscar(5)` indica “não encontrada”.

3.4.2 Passo 2 — Leitura da sequência de referências

Objetivo: transformar o arquivo em uma lista de inteiros, na ordem em que devem ser processados.

Regras: aceitar números separados por espaços e/ou quebras de linha, ignorar linhas vazias e comentários (#), rejeitar valores negativos ou não numéricos com uma mensagem de erro clara.

✎ C | leitor.c — esboço da leitura

```
int *carregar_sequencia(const char *caminho, int *tamanho) {
    FILE *f = fopen(caminho, "r");
    if (!f) { perror(caminho); return NULL; }
    int cap = 64, n = 0;
    int *seq = malloc(cap * sizeof(int));
    char linha[4096];
    while (fgets(linha, sizeof linha, f)) {
        if (linha[0] == '#') continue;
        /* TODO: percorrer a linha com strtok (ou sscanf com %n),
           converter cada token com strtol, validar,
           crescer 'seq' com realloc quando n == cap */
    }
    ...
}
```

✎ Python | leitor.py — esboço da leitura

```
def carregar_sequencia(caminho):
    seq = []
    with open(caminho) as f:
        for linha in f:
            linha = linha.split('#', 1)[0] # descarta comentário
            for token in linha.split():
                ... # TODO: converter, validar (>= 0), acrescentar
    return seq
```

✓ Verifique antes de seguir

Carregue `entradas/sequencial.txt` e imprima: devem aparecer 20 números, começando em 7 0 1 2 e terminando em 7 0 1. Teste também um arquivo com um número por linha e um arquivo com uma letra no meio (deve gerar erro, não travar).

3.4.3 Passo 3 — Registro e impressão do traço de execução

Objetivo: centralizar a impressão do estado da memória, para que os algoritmos só precisem “avisar” o que aconteceu.

A função `registrar_passo` recebe a referência atual, o estado dos quadros após o acesso, se houve falta e qual página saiu (se houve substituição). Ela pode imprimir imediatamente ou acumular os passos para imprimir no final.

 C | `traco.h` — interface do registro

```
typedef struct {
    int total_refs;
    int total_faltas;
    int verbose;           /* 0 = só totais (útil em traços longos) */
} Traco;

void registrar_passo(Traco *t, int ref, const Memoria *m,
                    int falta, int vitima /* ou VAZIO */);
void imprimir_resumo(const Traco *t, const char *algoritmo, int n_quadros);
```

 Python | `traco.py` — formatação de uma linha

```
def formatar_quadros(quadros):
    return "[" + ",".join("-" if p is None else str(p) for p in quadros) + "]"

def registrar_passo(ref, quadros, falta, vitima=None):
    status = "FAULT" if falta else "HIT"
    ... # TODO: montar a linha e, se vitima is not None, "(sai X)"
```

Formato sugerido de saída:

 **Formato do traço**

Ref	Quadros	Status
7	[7, -, -]	FAULT
0	[7, 0, -]	FAULT
1	[7, 0, 1]	FAULT
2	[2, 0, 1]	FAULT (sai 7)
0	[2, 0, 1]	HIT

 **Dica**

Para sequências longas (500+ referências), imprimir o traço inteiro é inútil. Tenha uma opção (`-q`, `--quiet` ou `verbose = 0`) que mostre só os totais.

3.4.4 Passo 4 — Algoritmo FIFO (modelo para os demais)

Objetivo: implementar o algoritmo mais simples e usá-lo como molde estrutural.

Todos os algoritmos seguem o mesmo esqueleto de laço; o que muda é (a) o que fazer num acerto e (b) como escolher a vítima:

</> C | esqueleto comum a todos os algoritmos

```

int executar_algoritmo(const int *seq, int n, int n_quadros, Traco *t) {
    Memoria *m = memoria_criar(n_quadros);
    for (int i = 0; i < n; i++) {
        int pag = seq[i];
        int idx = memoria_buscar(m, pag);
        if (idx >= 0) {
            /* ACERTO: FIFO não faz nada; LRU/SC/LFU atualizam algo */
        } else if (m->ocupados < n_quadros) {
            /* FALTA com quadro livre: carregar no primeiro vazio */
        } else {
            /* FALTA com memória cheia: escolher vítima (varia!) */
        }
        /* registrar_passo(...) */
    }
    ...
}

```

Para o FIFO, a informação extra é a **ordem de chegada**. Use uma fila (vetor circular em C, `collections.deque` em Python) com as páginas carregadas: na carga, a página entra no fim; na substituição, a vítima é a do início.

</> C | fila circular para o FIFO (fragmento)

```

typedef struct { int *v; int ini, fim, qtd, cap; } Fila;

void fila_inserir(Fila *f, int x) {
    f->v[f->fim] = x;
    f->fim = (f->fim + 1) % f->cap;
    f->qtd++;
}

int fila_remove(Fila *f); /* TODO: análoga, a partir de 'ini' */

```

Python | FIFO com deque (fragmento)

```

from collections import deque

ordem = deque()                # páginas na ordem de chegada
...
else:                          # falta com memória cheia
    vitima = ordem.popleft()    # a mais antiga
    idx = mem.posicao.pop(vitima)
    # TODO: colocar 'pag' em mem.quadros[idx], atualizar posicao,
    #        inserir 'pag' no fim da fila, contar a falta

```

⚠ Atenção

Erro comum: mover a página para o fim da fila num *acerto*. Isso transforma o FIFO em LRU! No FIFO, acertos **não alteram** a fila.

✓ Verifique antes de seguir

- Sequência de exemplo, 3 quadros: **15 faltas**; o traço deve bater com a Tabela 3.1.
- Sequência 1 2 3 4 1 2 5 1 2 3 4 5: **9 faltas** com 3 quadros e **10 faltas** com 4 quadros (Belady).
- Com 1 quadro, o número de faltas é igual ao número de referências que diferem da anterior (20 na sequência de exemplo).

3.4.5 Passo 5 — Algoritmo LRU

Objetivo: substituir a página menos recentemente usada. A diferença estrutural em relação ao FIFO é que **todo acesso** (acerto ou falta) atualiza a informação de uso.

Opção A — relógio lógico. Mantenha um contador **tempo** incrementado a cada referência. Em todo acesso, grave `quadro.ultimo_uso = tempo`. Na substituição, a vítima é o quadro com menor `ultimo_uso`.

✎ C | LRU com relógio lógico (fragmentos)

```
int tempo = 0;
...
tempo++;
if (idx >= 0) {
    m->q[idx].ultimo_uso = tempo;          /* acerto: renova o uso */
} else {
    int vit = escolher_vitima_lru(m);      /* menor ultimo_uso */
    /* TODO: substituir, gravar ultimo_uso = tempo, contar falta */
}

static int escolher_vitima_lru(const Memoria *m) {
    int menor = 0;
    /* TODO: varrer os quadros e guardar o índice do menor ultimo_uso */
    return menor;
}
```

Opção B — lista ordenada por uso. Em Python, `collections.OrderedDict` mantém a ordem de inserção e permite mover uma chave para o fim em $O(1)$ com `move_to_end`, e remover a primeira com `popitem(last=False)`. O início do dicionário é sempre o menos recentemente usado.

✎ Python | LRU com OrderedDict (fragmentos)

```
from collections import OrderedDict

uso = OrderedDict()          # página -> índice do quadro; início = LRU
...
if pag in uso:               # acerto
    uso.move_to_end(pag)     # passa a ser a mais recente
else:
    if len(uso) == n_quadros:
        vitima, idx = uso.popitem(last=False)
        # TODO: reaproveitar o quadro 'idx' para 'pag'
```

```
... # TODO: carregar, inserir em 'uso', contar a falta
```

Dica

No relatório, comente a complexidade: a Opção A tem atualização $O(1)$ e escolha de vítima $O(n)$ por falta; a Opção B com `OrderedDict` (ou lista duplamente encadeada + tabela de dispersão, em C) faz tudo em $O(1)$. Uma implementação em C que *desloca* elementos de um vetor a cada acesso é $O(n)$ por referência — aceitável didaticamente, mas mencione isso.

Verifique antes de seguir

Sequência de exemplo com 3 quadros: **12 faltas**. Ponto de controle: na 6.^a referência (página 3), os quadros são `[2, 0, 1]` e a vítima deve ser a página **1** (não a 0, pois a 0 acabou de ser usada), resultando em `[2, 0, 3]`. Sequência de Belady: 10 faltas com 3 quadros e 8 com 4.

3.4.6 Passo 6 — Algoritmo Ótimo (Belady)

Objetivo: a cada substituição, retirar a página cuja próxima referência está mais distante no futuro (ou que nunca mais será referenciada).

Crie uma função auxiliar que, dada uma página e a posição atual, devolve a posição da próxima ocorrência:

C | função auxiliar do Ótimo

```
#define NUNCA INT_MAX /* de <limits.h> */

/* Posição da próxima ocorrência de 'pag' em seq[i+1 .. n-1], ou NUNCA. */
int proxima_ocorrencia(const int *seq, int n, int i, int pag) {
    /* TODO */
}
```

Python | escolha da vítima no Ótimo (fragmento)

```
def proxima_ocorrencia(seq, i, pag):
    ... # TODO: posição da próxima ocorrência após i, ou float('inf')

# na substituição:
vítima = max(carregadas, key=lambda p: ...) # TODO: critério
```

Atenção

Empates: se várias páginas nunca mais serão usadas, qualquer uma delas é uma escolha ótima (o número de faltas não muda), mas o *traço* pode mudar. Adote um critério fixo (ex.: a de menor índice de quadro) e documente-o.

Dica

A busca ingênua custa $O(n)$ por página candidata, ou seja, $O(n \cdot q)$ por falta. Para sequências muito longas, você pode pré-calcular, de trás para frente, um vetor `prox[i]` com a próxima posição em que `seq[i]` aparece. É uma otimização opcional; mencione-a se usar.

✓ Verifique antes de seguir

Sequência de exemplo com 3 quadros: **9 faltas**. Na 4.^a referência (página 2), com [7,0,1], a vítima deve ser a 7 (próximo uso na posição 18, a mais distante). Sequência de Belady: 7 faltas com 3 quadros e 6 com 4. Para *qualquer* sequência e número de quadros, o Ótimo nunca pode ter mais faltas que os outros algoritmos — use isso como teste automático!

3.4.7 Passo 7 — Algoritmo Segunda Chance (relógio)

Objetivo: implementar a variante do FIFO com bit de referência, usando um ponteiro circular sobre os quadros.

Estado adicional: o bit `bit_ref` de cada quadro e um índice `ponteiro` que indica o quadro “mais antigo”. Convenção deste roteiro: **bit = 1 na carga e em cada acerto**.

✎ C | Segunda Chance (fragmentos)

```
int ponteiro = 0;
...
if (idx >= 0) {
    m->q[idx].bit_ref = 1;                /* acerto: liga o bit */
} else if (/* memória cheia */) {
    while (m->q[ponteiro].bit_ref == 1) {
        /* TODO: dar a segunda chance e avançar o ponteiro
           circularmente */
    }
    /* TODO: 'ponteiro' aponta a vítima: substituir, bit_ref = 1,
       avançar o ponteiro */
}
```

🐍 Python | Segunda Chance (fragmentos)

```
bits = [0] * n_quadros
ponteiro = 0
...
if idx is not None:
    bits[idx] = 1                        # acerto
elif memoria_cheia:
    while bits[ponteiro] == 1:
        ...                            # TODO: segunda chance
    # TODO: substituir no quadro 'ponteiro', ligar o bit, avançar
```

⚠ Atenção

Enquanto a memória ainda tem quadros livres, as páginas são carregadas nos quadros 0, 1, 2, ...; cuide para que, quando a memória encher, o ponteiro esteja apontando para o quadro 0 (o mais antigo). O laço `while` sempre termina: no pior caso ele dá uma volta completa zerando todos os bits e escolhe o quadro onde começou.

✓ Verifique antes de seguir

Com a convenção `bit = 1 na carga`, a sequência de exemplo com 3 quadros gera **14 faltas** (com `bit = 0 na carga` seriam 11). Ponto de controle: na 4.^a referência (página 2), os quadros [7:1,0:1,1:1] têm todos o bit ligado; o ponteiro dá a volta zerando-os e escolhe a página

7, resultando em [2:1,0:0,1:0]. O número de faltas deve ficar sempre entre o do FIFO e o do Ótimo... na maioria dos casos — verifique e comente as exceções que encontrar.

3.4.8 Passo 8 — (Desejável) Algoritmo LFU

Mantenha um contador de acessos por quadro: zere-o (ou coloque 1) na carga e incremente-o a cada acerto. A vítima é a página de menor contador; em caso de empate, aplique um critério secundário (FIFO ou LRU entre as empatadas). A estrutura é idêntica à do LRU com relógio lógico, trocando o campo consultado.

 Python | LFU: critério de escolha com desempate (fragmento)

```
# chave de ordenação: (contador de acessos, instante de chegada)
vitima = min(carregadas, key=lambda p: (contador[p], chegada[p]))
```

3.4.9 Passo 9 — Orquestração, variação de quadros e exportação CSV

Objetivo: executar todos os algoritmos, para vários números de quadros, sobre a **mesma** sequência, e produzir uma tabela consolidada.

 C | main.c — tabela de algoritmos com ponteiros para função

```
typedef int (*AlgoritmoFn)(const int *seq, int n, int n_quadros, Traco *t);

static const struct { const char *nome; AlgoritmoFn fn; } ALGS[] = {
    {"FIFO", executar_fifo}, {"LRU", executar_lru},
    {"Otimo", executar_otimo}, {"SegundaChance", executar_segunda_chance},
};
...
for (int q = q_min; q <= q_max; q++)
    for (size_t a = 0; a < sizeof ALGS / sizeof ALGS[0]; a++) {
        /* TODO: executar, imprimir, gravar linha no CSV:
           algoritmo,num_quadros,total_refs,total_faltas,taxa */
    }
```

 Python | main.py — dicionário de algoritmos e CSV

```
import csv, sys

ALGORITMOS = {"FIFO": executar_fifo, "LRU": executar_lru,
              "Otimo": executar_otimo, "SegundaChance": executar_segunda_chance}

with open("resultados/faltas.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["algoritmo", "num_quadros", "total_refs", "total_faltas", "taxa"])
    for q in range(q_min, q_max + 1):
        for nome, fn in ALGORITMOS.items():
            ... # TODO: executar e escrever a linha
```

⚠ Atenção

Cada execução deve começar com a memória **zerada**. Um erro comum é reutilizar estruturas (fila, relógio, ponteiro) de uma execução anterior, contaminando os resultados. Trate parâmetros inválidos (número de quadros ≤ 0 , algoritmo desconhecido, arquivo inexistente) com mensagens de erro, sem travar.

✓ Verifique antes de seguir

A tabela gerada para a sequência de exemplo, variando de 1 a 7 quadros, deve coincidir com a Tabela 3.3. Se algum valor divergir, compare o traço passo a passo com o da Seção 3.5.

3.4.10 Passo 10 — Gerador de sequências (para os experimentos)

Para os experimentos você vai precisar de sequências longas com padrões diferentes. Escreva um pequeno gerador com três modos e **semente fixa** (para que todos os algoritmos sejam comparados com a mesma sequência):

- **sequencial:** $\text{pagina} = i \bmod \text{num_paginas}$ (varredura de um vetor, sem reuso próximo);
- **aleatório:** páginas sorteadas uniformemente (sem localidade);
- **com localidade:** sorteia-se um pequeno *working set* (3 a 5 páginas) e gera-se uma rajada de 10 a 30 acessos dentro dele; depois sorteia-se outro *working set*, e assim por diante.

Python | gerador: modo com localidade (fragmento)

```
import random

def gerar_localidade(n_acessos, n_paginas, janela, semente):
    rng = random.Random(semente)
    seq = []
    while len(seq) < n_acessos:
        ws = rng.sample(range(n_paginas), janela)  # working set atual
        for _ in range(rng.randint(10, 30)):
            ... # TODO: sortear uma página de ws e acrescentar
    return seq[:n_acessos]
```

⌄ C | gerador: semente fixa em C

```
srand(semente); /* mesma semente => mesma sequência */
int pagina = rand() % num_paginas; /* modo aleatório */
```

3.5 Exemplos de entrada e saída

Use estes exemplos para validar a sua implementação. Todos os valores foram conferidos por simulação. Convenções: páginas substituídas ocupam o mesmo quadro da vítima; “—” indica quadro vazio.

3.5.1 FIFO com 3 quadros

Tabela 3.1: Traço completo do FIFO, sequência de exemplo, 3 quadros.

Ref	Quadros	Status	Ref	Quadros	Status
7	[7,-,-]	falta	3	[4,2,3]	falta (sai 0)
0	[7,0,-]	falta	0	[0,2,3]	falta (sai 4)
1	[7,0,1]	falta	3	[0,2,3]	acerto
2	[2,0,1]	falta (sai 7)	2	[0,2,3]	acerto
0	[2,0,1]	acerto	1	[0,1,3]	falta (sai 2)
3	[2,3,1]	falta (sai 0)	2	[0,1,2]	falta (sai 3)
0	[2,3,0]	falta (sai 1)	0	[0,1,2]	acerto
4	[4,3,0]	falta (sai 2)	1	[0,1,2]	acerto
2	[4,2,0]	falta (sai 3)	7	[7,1,2]	falta (sai 0)
			0	[7,0,2]	falta (sai 1)
			1	[7,0,1]	falta (sai 2)
Total: 15 faltas em 20 referências (75%)					

Observe a 7.^a referência (página 0): ela tinha acabado de gerar um acerto na 5.^a referência, mas mesmo assim foi expulsa na 6.^a, porque era a segunda mais antiga. É exatamente esse tipo de decisão “cega” que o LRU corrige.

3.5.2 LRU com 3 quadros

Tabela 3.2: Traço completo do LRU, sequência de exemplo, 3 quadros.

Ref	Quadros	Status	Ref	Quadros	Status
7	[7,-,-]	falta	3	[4,3,2]	falta (sai 0)
0	[7,0,-]	falta	0	[0,3,2]	falta (sai 4)
1	[7,0,1]	falta	3	[0,3,2]	acerto
2	[2,0,1]	falta (sai 7)	2	[0,3,2]	acerto
0	[2,0,1]	acerto	1	[1,3,2]	falta (sai 0)
3	[2,0,3]	falta (sai 1)	2	[1,3,2]	acerto
0	[2,0,3]	acerto	0	[1,0,2]	falta (sai 3)
4	[4,0,3]	falta (sai 2)	1	[1,0,2]	acerto
2	[4,0,2]	falta (sai 3)	7	[1,0,7]	falta (sai 2)
			0	[1,0,7]	acerto
			1	[1,0,7]	acerto
Total: 12 faltas em 20 referências (60%)					

3.5.3 Ótimo e Segunda Chance com 3 quadros

>_ Ótimo, 3 quadros (primeiras substituições)

```

Ref  Quadros    Status
 7  [7,-,-]    FAULT
 0  [7,0,-]    FAULT
 1  [7,0,1]    FAULT
 2  [2,0,1]    FAULT (sai 7)    # 7 só volta na posição 18
 0  [2,0,1]    HIT
 3  [2,0,3]    FAULT (sai 1)    # 1 só volta na posição 14
 0  [2,0,3]    HIT
 4  [2,4,3]    FAULT (sai 0)    # 0 volta na 11; 2 na 9; 3 na 10
...
Total de page faults: 9  (45%)

```

>_ Segunda Chance, 3 quadros (formato pagina:bit)

```

Ref  Quadros          Status
 7  [7:1,-,-]        FAULT
 0  [7:1,0:1,-]      FAULT
 1  [7:1,0:1,1:1]    FAULT
 2  [2:1,0:0,1:0]    FAULT (sai 7)    # volta completa zerando bits
 0  [2:1,0:1,1:0]    HIT
 3  [2:1,0:0,3:1]    FAULT (sai 1)    # 0 ganhou segunda chance
 0  [2:1,0:1,3:1]    HIT
 4  [4:1,0:0,3:0]    FAULT (sai 2)
 2  [4:1,2:1,3:0]    FAULT (sai 0)
 3  [4:1,2:1,3:1]    HIT
 0  [4:0,2:0,0:1]    FAULT (sai 3)
...
Total de page faults: 14  (70%)

```

3.5.4 Tabela consolidada

Tabela 3.3: Faltas de página por número de quadros, sequência de exemplo (20 referências).

Quadros	FIFO	LRU	Ótimo	Segunda Chance
1	20	20	20	20
2	15	17	13	15
3	15	12	9	14
4	10	8	8	9
5	9	7	7	9
6	6	6	6	6
7	6	6	6	6

Observações interessantes desta tabela, que você deve saber explicar:

- Com 6 ou mais quadros todas as 6 páginas distintas cabem na memória: só ocorrem as 6 faltas compulsórias, em qualquer algoritmo.

- Com 2 quadros, o LRU (17) foi *pior* que o FIFO (15). O LRU não é garantidamente melhor que o FIFO em toda sequência — ele é melhor *em média* quando há localidade temporal.
- O Ótimo é sempre o menor valor de cada linha.

3.5.5 Anomalia de Belady

>_ FIFO na sequência 1 2 3 4 1 2 5 1 2 3 4 5

== FIFO, 3 quadros ==

```
1 [1,-,-] FAULT
2 [1,2,-] FAULT
3 [1,2,3] FAULT
4 [4,2,3] FAULT (sai 1)
1 [4,1,3] FAULT (sai 2)
2 [4,1,2] FAULT (sai 3)
5 [5,1,2] FAULT (sai 4)
1 [5,1,2] HIT
2 [5,1,2] HIT
3 [5,3,2] FAULT (sai 1)
4 [5,3,4] FAULT (sai 2)
5 [5,3,4] HIT
```

Total: 9

== FIFO, 4 quadros ==

```
1 [1,-,-,-] FAULT
2 [1,2,-,-] FAULT
3 [1,2,3,-] FAULT
4 [1,2,3,4] FAULT
1 [1,2,3,4] HIT
2 [1,2,3,4] HIT
5 [5,2,3,4] FAULT (sai 1)
1 [5,1,3,4] FAULT (sai 2)
2 [5,1,2,4] FAULT (sai 3)
3 [5,1,2,3] FAULT (sai 4)
4 [4,1,2,3] FAULT (sai 5)
5 [4,5,2,3] FAULT (sai 1)
```

Total: 10

3.5.6 Casos triviais (testes de sanidade)

Sequência	Quadros	Faltas	O que valida
1 2 3 4 5	5	5 (100%)	tudo cabe: só faltas compulsórias, igual para todos
1 1 1 1 2 2 2 1 1 1	2	2	acessos repetidos não geram falta
1 2 3 4 5 6 7 8 9 10	3	10 (100%)	sem reuso, todos os algoritmos são iguais
qualquer	1	nº de trocas de página	com um quadro, falta sempre que a página muda

3.6 Experimentos

Os experimentos transformam o simulador em um instrumento de análise. Para todos eles: **mesma sequência para todos os algoritmos**, semente fixa, e resultados exportados em CSV.

1. **Curva de faltas × quadros** (experimento principal): para cada sequência, execute todos os algoritmos com 1 a 8 quadros (ou mais, até a curva estabilizar) e gere um gráfico com uma linha por algoritmo.
2. **Anomalia de Belady**: reproduza a anomalia com a sequência 1 2 3 4 1 2 5 1 2 3 4 5 e mostre que LRU e Ótimo não a apresentam. Desafio: tente encontrar (ou gerar aleatoriamente e filtrar) *outra* sequência que cause a anomalia.
3. **Padrões de acesso**: compare os algoritmos com sequências sequenciais, aleatórias e com localidade (por exemplo, 500 acessos, 30 páginas, working set de 4). Em qual padrão o LRU se aproxima mais do Ótimo? Em qual todos ficam parecidos?


```

with open("resultados/faltas.csv") as f:
    for linha in csv.DictReader(f):
        ... # TODO: converter e acrescentar em dados[linha["algoritmo"]]

for alg, pontos in dados.items():
    ... # TODO: ordenar por quadros e plt.plot(x, y, marker="o", label=alg)
plt.xlabel("Número de quadros"); plt.ylabel("Faltas de página")
plt.legend(); plt.grid(alpha=0.3)
plt.savefig("resultados/curva.pdf")

```

3.7 Perguntas para o relatório

Responda às perguntas abaixo na seção de Discussão do relatório, sempre apoiado nos *seus* resultados experimentais:

1. Reproduza a Anomalia de Belady com uma sequência específica. Explique por que ela ocorre no FIFO e por que não ocorre no LRU nem no Ótimo (use o conceito de algoritmo de pilha).
2. Compare o desempenho da Segunda Chance com o FIFO puro. Em que casos há diferença? Em que casos os dois se comportam exatamente igual? Qual convenção de bit você adotou e como ela afeta o resultado?
3. Por que o algoritmo Ótimo não é implementável em um sistema real? Para que ele serve, então?
4. Qual a complexidade (por referência e por falta) da sua implementação do LRU? Como ela poderia ser melhorada?
5. Em qual padrão de acesso (sequencial, aleatório, com localidade) o LRU mais se aproximou do Ótimo? Relacione com o princípio da localidade.
6. Usando a fórmula do tempo efetivo de acesso, estime quanto tempo a execução de uma das suas sequências levaria com cada algoritmo (suponha $t_{mem} = 100\text{ ns}$ e $t_{falta} = 8\text{ ms}$).

3.8 Critérios de avaliação e checklist

Critério	Peso
Corretude de cada algoritmo (FIFO, LRU, Ótimo, Segunda Chance — 15% cada)	60%
Contagem correta de faltas de página e traço de execução	15%
Análise da Anomalia de Belady	15%
Organização do código	10%

Checklist de entrega

Ambiente e compilação

- ☐ Compila e executa em Linux (nativo ou WSL).
- ☐ (C) Compila sem avisos com `-Wall -Wextra`; `make` gera o executável e `make clean` remove os artefatos.

- ☐ (Python) `make run` (ou instrução equivalente no README) executa o simulador sem passos manuais.

Funcionalidade

- ☐ Leitura correta da sequência (um número por linha ou separados por espaço, com comentários).
- ☐ FIFO, LRU, Ótimo e Segunda Chance implementados e conferidos com os exemplos deste capítulo.
- ☐ Traço de execução com estado dos quadros, falta/acerto e página substituída.
- ☐ Total e taxa de faltas; tabela consolidada exportada em CSV.
- ☐ Parâmetros via linha de comando (arquivo, número(s) de quadros, algoritmo).

Qualidade de código

- ☐ Código organizado em arquivos conforme a estrutura sugerida.
- ☐ (C) Sem vazamentos de memória (verificado com `valgrind`).
- ☐ Erros tratados: arquivo inexistente ou malformado, número de quadros ≤ 0 , algoritmo desconhecido.

Testes e experimentos

- ☐ Número de quadros variado em uma faixa ampla.
- ☐ Padrões sequencial, aleatório e com localidade, com semente fixa.
- ☐ Casos extremos: 1 quadro, todas as páginas cabendo, sequência muito longa.
- ☐ Gráfico de faltas $\times$ quadros e reprodução da Anomalia de Belady.

Relatório

- ☐ Escrito no Overleaf, modelo SBC; entregues fontes `.tex` e PDF.
- ☐ Contém introdução, fundamentação, implementação, resultados com gráficos, discussão (perguntas acima) e conclusão.
- ☐ Convenções documentadas: bit R na Segunda Chance, desempates no Ótimo/LFU, estrutura usada no LRU.

3.9 Leituras recomendadas

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. Capítulos de Memória Principal e Memória Virtual (paginação, tabela de páginas, substituição de páginas, Anomalia de Belady, thrashing).
- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. Capítulo 3 — Gerenciamento de Memória (algoritmos de substituição, incluindo o relógio e WSClock).
- STALLINGS, W. *Sistemas Operacionais: Princípios e Projeto*. Capítulo de Memória Virtual.
- ARPACI-DUSSEAU, R.; ARPACI-DUSSEAU, A. *Operating Systems: Three Easy Pieces*. Capítulos *Paging: Introduction*, *Paging: Faster Translations (TLBs)*, *Beyond Physical Memory: Mechanisms e Policies*. Disponível gratuitamente em <https://pages.cs.wisc.edu/~remzi/OSTEP/>.

- BELADY, L. A.; NELSON, R. A.; SHEDLER, G. S. An anomaly in space-time characteristics of certain programs running in a paging machine. *Communications of the ACM*, 12(6), 1969.
- Documentação do Python: `collections.OrderedDict` e `collections.deque` — <https://docs.python.org/3/library/collections.html>.

Capítulo 4

Alocador de Memória Customizado

4.1 Visão geral

Toda vez que um programa em C chama `malloc(100)`, alguém precisa decidir *onde*, dentro da memória do processo, aqueles 100 bytes serão colocados; quando o programa chama `free`, alguém precisa lembrar que aquele espaço voltou a estar disponível e, de preferência, reaproveitá-lo bem. Esse “alguém” é o **alocador de memória dinâmica**, uma camada de software que fica entre o seu programa e o sistema operacional.

Neste trabalho você vai construir o seu próprio alocador: uma versão simplificada de `malloc/free` que opera sobre um *heap simulado* — uma única região contígua de bytes reservada no início do programa. Toda a gerência dessa região (quais pedaços estão livres, quais estão ocupados, como dividir e juntar pedaços) será feita manualmente por você. Em seguida, você vai comparar experimentalmente três estratégias clássicas de escolha de bloco livre (*First-Fit*, *Best-Fit* e *Worst-Fit*) quanto à fragmentação da memória e ao custo computacional.

🕒 Ao final deste trabalho você deverá ser capaz de

- Descrever como o espaço de endereçamento de um processo é organizado (código, dados, heap e pilha) e qual o papel do heap.
- Explicar a diferença entre fragmentação **interna** e **externa** e medi-las numericamente.
- Projetar um cabeçalho (*header*) de bloco armazenado *dentro* da própria memória gerenciada.
- Implementar alocação com alinhamento, divisão de blocos (*splitting*) e liberação com fusão de vizinhos (*coalescência*).
- Implementar e comparar as estratégias First-Fit, Best-Fit e Worst-Fit sob diferentes padrões de uso (*workloads*).
- Detectar e tratar erros clássicos: *double free*, ponteiro inválido, pedido maior que o heap.

⚠️ Atenção

Este trabalho pode ser feito em C ou em Python. Em C você manipulará ponteiros e memória bruta diretamente. Em Python **não** é permitido usar listas de objetos “soltos” como se fossem blocos: o heap deve ser um único `bytearray`, e os cabeçalhos dos blocos devem ser gravados *dentro* dele (ver Seção 4.4). Em ambos os casos, as funções de alocação nativas da linguagem só podem ser usadas **uma única vez**, para reservar o heap simulado.

4.2 Fundamentação teórica

4.2.1 O espaço de endereçamento de um processo

Quando o sistema operacional carrega um programa para executá-lo, ele cria um **processo** e lhe entrega um **espaço de endereçamento**: uma faixa de endereços de memória (virtuais) que o processo pode usar. Esse espaço não é uma “massa” uniforme; ele é dividido em regiões com propósitos diferentes, como mostra a Figura 4.1.

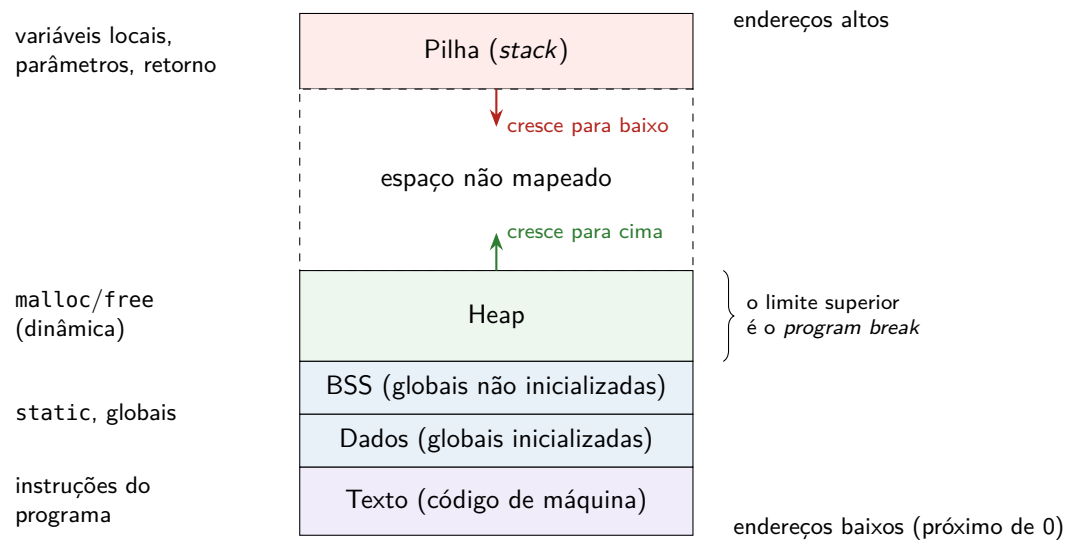


Figura 4.1: Layout típico do espaço de endereçamento de um processo Linux.

- **Texto:** as instruções de máquina do programa. Normalmente é somente leitura.
- **Dados e BSS:** variáveis globais e `static`. Seu tamanho é conhecido em tempo de compilação.
- **Pilha:** guarda as variáveis locais e os endereços de retorno das funções. Cada chamada de função “empilha” um *quadro* (*stack frame*), que é descartado automaticamente quando a função retorna.
- **Heap:** região usada para *alocação dinâmica*, isto é, memória pedida explicitamente pelo programa durante a execução, em quantidades que só são conhecidas em tempo de execução, e que vive até ser explicitamente liberada.

4.2.2 Três formas de alocar memória

	Estática	Automática (pilha)	Dinâmica (heap)
Quando é decidida	compilação	entrada na função	execução, a pedido
Tempo de vida	todo o programa	até a função retornar	até <code>free</code>
Tamanho	fixo	fixo por chamada	qualquer, decidido em tempo de execução
Quem libera	ninguém (fim do programa)	automático	o programador
Exemplo em C	<code>static int v[100];</code>	<code>int x; char buf[64];</code>	<code>p = malloc(n);</code>

A alocação dinâmica é a mais flexível, mas é também a mais difícil: o programa pode pedir e devolver blocos de tamanhos arbitrários, em qualquer ordem. Organizar esses pedidos de forma eficiente é exatamente o problema que um alocador resolve.

4.2.3 De onde o malloc real tira memória?

O `malloc` da biblioteca C (glibc) não é uma chamada de sistema: é uma função comum, em espaço de usuário. Ela pede memória ao núcleo do sistema operacional em *grandes pedaços*, e depois recorta esses pedaços para atender os pedidos pequenos do programa. As chamadas de sistema envolvidas são:

- `brk/sbrk`: movem o *program break*, o limite superior do heap (Figura 4.1). `sbrk(4096)` “estica” o heap em 4096 bytes e devolve o endereço do início da parte nova.
- `mmap`: pede ao núcleo uma região de páginas em qualquer ponto do espaço de endereçamento. A glibc usa `mmap` para pedidos grandes (tipicamente ≥ 128 KiB).

Chamadas de sistema são caras (exigem troca de modo usuário $\rightarrow$ núcleo). Por isso o alocador pede memória ao SO raramente e faz, por conta própria, a gerência fina. No nosso trabalho vamos simplificar: o heap é obtido **uma única vez**, com tamanho fixo, e nunca cresce.

💡 O contrato de malloc e free

- `void *malloc(size_t n)` devolve um ponteiro para uma região de *pelo menos* n bytes, alinhada adequadamente, que não se sobrepõe a nenhuma outra região ainda em uso. Se não for possível, devolve `NULL`.
- `void free(void *p)` recebe *exatamente* um ponteiro devolvido por `malloc` e ainda não liberado, e devolve essa região ao alocador. `free(NULL)` não faz nada.
- Repare: **free não recebe o tamanho!** O alocador precisa descobrir sozinho quantos bytes aquele bloco tem. É por isso que todo bloco carrega um *cabeçalho* com metadados.

4.2.4 O problema do alocador

Imagine o heap como uma faixa de bytes. No início, ela está toda livre. À medida que o programa aloca e libera, a faixa vira um mosaico de pedaços ocupados e livres (Figura 4.2). O alocador precisa, a cada pedido:

1. encontrar um pedaço livre grande o bastante (*política de posicionamento*);
2. decidir o que fazer com a sobra, se o pedaço for maior que o pedido (*divisão*);
3. ao receber um bloco de volta, reintegrá-lo, juntando-o com vizinhos livres (*coalescência*).

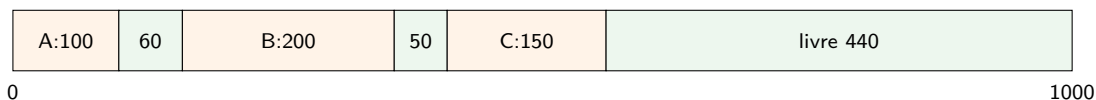


Figura 4.2: Heap de 1000 bytes após algumas alocações e liberações: blocos ocupados (laranja) e livres (verde).

Diferentemente de um coletor de lixo, um alocador C **não pode mover** blocos ocupados: o programa guarda ponteiros para eles, e mover um bloco invalidaria esses ponteiros. Essa restrição é a raiz do problema da fragmentação.

4.2.5 Fragmentação interna e externa

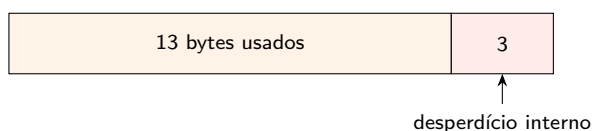
Definição: Fragmentação interna

Espaço desperdiçado *dentro* de um bloco ocupado: o alocador reservou mais bytes do que o programa pediu (por alinhamento, arredondamento ou porque não valia a pena dividir o bloco). Esse espaço está “preso” ao bloco e não pode ser usado por mais ninguém até o bloco ser liberado.

Definição: Fragmentação externa

Espaço livre que existe *fora* dos blocos ocupados, mas está espalhado em pedaços pequenos e não contíguos. A soma dos pedaços pode ser grande, mas nenhum pedaço isolado atende a um pedido maior.

Interna: pedido de 13 bytes, bloco reservado de 16



Externa: 200 bytes livres no total, pedido de 150 falha

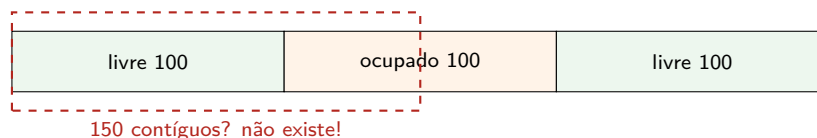


Figura 4.3: Fragmentação interna (em cima) e externa (embaixo).

Para comparar estratégias precisamos *medir* fragmentação. Algumas métricas simples e usuais:

$$\text{Frag. interna total} = \sum_{\text{blocos ocupados}} (\text{tamanho reservado} - \text{tamanho pedido})$$

$$\text{Frag. externa} = 1 - \frac{\text{maior bloco livre}}{\text{total livre}} \quad (0 = \text{sem fragmentação}; \rightarrow 1 = \text{muito fragmentado})$$

$$\text{Overhead de metadados} = \frac{\text{nº de blocos} \times \text{tamanho do cabeçalho}}{\text{tamanho do heap}}$$

$$\text{Taxa de falha} = \frac{\text{pedidos que retornaram NULL}}{\text{total de pedidos}}$$

Exemplo: no heap da Figura 4.2, o total livre é $60 + 50 + 440 = 550$ bytes e o maior bloco livre tem 440. A fragmentação externa é $1 - 440/550 = 0,20$ (20%). Já no exemplo de baixo da Figura 4.3, ela é $1 - 100/200 = 0,5$ (50%).

Dica

É especialmente interessante registrar pedidos que falharam *mesmo havendo espaço livre total suficiente*: são falhas causadas exclusivamente pela fragmentação externa. Separe-as das falhas por falta real de memória.

4.2.6 Cabeçalho de bloco: metadados dentro do heap

Como `free` recebe só o ponteiro, o alocador guarda as informações de cada bloco em um **cabeçalho** (*header*) posicionado imediatamente *antes* da área de dados entregue ao usuário (Figura 4.4). Assim, dado o ponteiro `p` devolvido ao usuário, o cabeçalho está em `p - sizeof(cabeçalho)`.

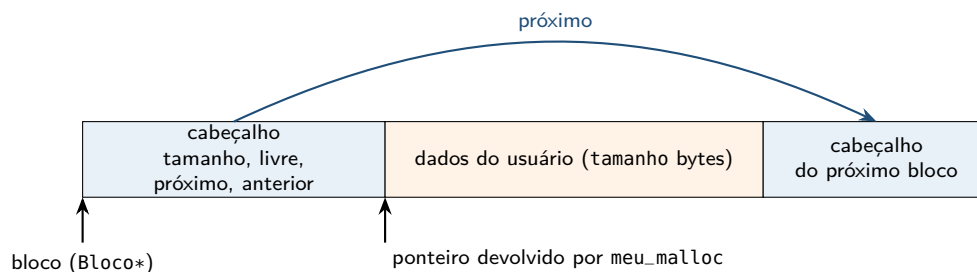


Figura 4.4: Um bloco é formado por cabeçalho + área de dados. O ponteiro do usuário aponta para depois do cabeçalho.

⚠ Atenção

O cabeçalho ocupa espaço *do próprio heap*. Em um sistema de 64 bits, uma estrutura com `size_t`, `int` e dois ponteiros ocupa 32 bytes. Um pedido de 8 bytes custa, portanto, 40 bytes de heap! Isso é o *overhead de metadados*, que você também deve medir.

Lista implícita e lista explícita

Existem duas formas clássicas de organizar os blocos:

- **Lista implícita:** todos os blocos (livres e ocupados) estão encadeados em *ordem de endereço*. Para achar o próximo bloco basta somar o tamanho ao endereço atual (ou seguir um ponteiro `proximo`). A busca percorre também os ocupados, mas a implementação é simples e a coalescência é natural, porque os vizinhos físicos estão lado a lado na lista. **É a organização recomendada para este trabalho.**
- **Lista explícita (*free list*):** apenas os blocos livres são encadeados entre si (os ponteiros ficam dentro da área de dados, que está sem uso). A busca é mais rápida, mas inserir, remover e coalescer exige mais cuidado.

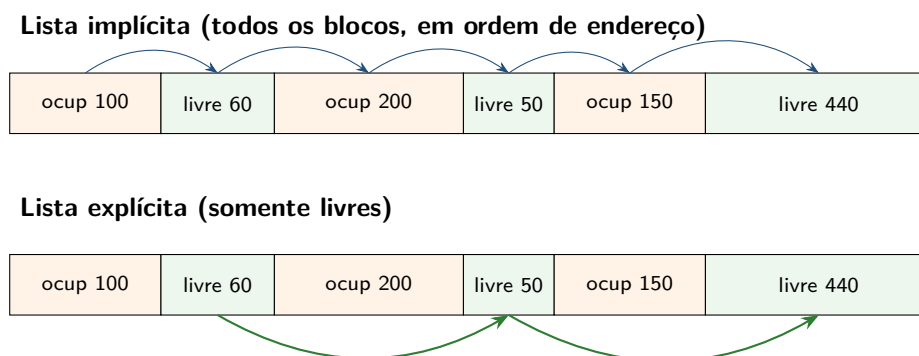


Figura 4.5: Organizações de lista de blocos.

4.2.7 Alinhamento

Processadores acessam memória de forma mais eficiente (e, em algumas arquiteturas, *só conseguem* acessar) quando um dado de k bytes está em um endereço múltiplo de k . Por isso **malloc** garante que todo ponteiro devolvido esteja alinhado a 8 ou 16 bytes. Neste trabalho usaremos **alinhamento de 8 bytes**: todo tamanho pedido é arredondado para cima até o próximo múltiplo de 8.

$$\text{alinhado}(n) = \left\lceil \frac{n}{8} \right\rceil \times 8 \quad \text{ou, com operações de bits,} \quad \text{alinhado}(n) = (n + 7) \& \sim 7$$

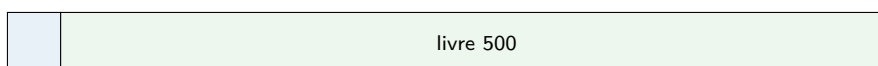
Pedido n	Alinhado	Fragmentação interna
1	8	7
8	8	0
13	16	3
100	104	4

Por que o truque de bits funciona? Somar 7 “empurra” n para o próximo múltiplo de 8 (ou para além dele, se n já for múltiplo); a máscara ~ 7 (que em binário termina em $\dots 11111000$) zera os 3 bits menos significativos, ou seja, arredonda para baixo até um múltiplo de 8. O tamanho do cabeçalho também deve ser múltiplo de 8, para que a área de dados comece alinhada.

4.2.8 Divisão de blocos (*splitting*)

Se o bloco livre escolhido tem 500 bytes e o pedido tem 104, entregar os 500 inteiros geraria 396 bytes de fragmentação interna. Em vez disso, o alocador **divide** o bloco: os primeiros 104 bytes viram o bloco ocupado e o restante vira um *novo* bloco livre, com seu próprio cabeçalho (Figura 4.6).

Antes: um bloco livre de 500 bytes de dados



Depois de `meu_malloc(100)` (alinhado para 104, cabeçalho de 32)



Figura 4.6: Divisão: o novo bloco livre “paga” um cabeçalho extra.

Repare que o novo bloco livre perde o espaço de um cabeçalho. Se a sobra for minúscula (por exemplo, menor que um cabeçalho + 8 bytes), a divisão criaria um fragmento inútil. Por isso define-se um **tamanho mínimo de divisão**: só divide se

$$\text{tamanho_bloco} - \text{tamanho_alinhado} \geq \text{sizeof}(\text{cabeçalho}) + \text{MIN_DADOS}.$$

Caso contrário, o bloco é entregue inteiro e a sobra é contabilizada como fragmentação interna.

4.2.9 Coalescência

Quando um bloco é liberado, seus vizinhos *físicos* (o bloco imediatamente antes e o imediatamente depois, na memória) podem estar livres. Se o alocador apenas marcar o bloco como livre, o heap

vira uma coleção de pedacinhos livres adjacentes que, juntos, atenderiam pedidos grandes, mas separados não atendem (*fragmentação falsa*). A **coalescência** funde blocos livres adjacentes em um só. Há quatro casos (Figura 4.7):

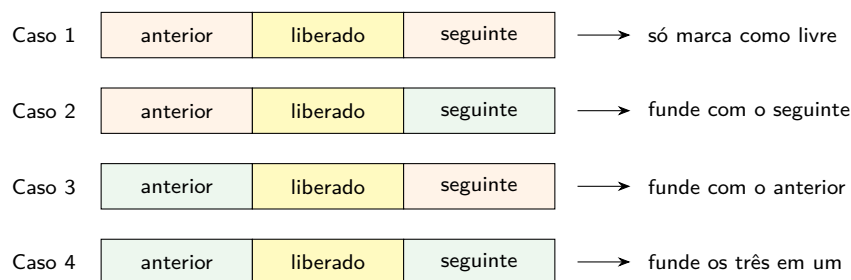


Figura 4.7: Os quatro casos de coalescência ao liberar um bloco (verde = livre, laranja = ocupado).

Ao fundir o bloco X com o seguinte Y , o tamanho de X passa a ser

$$\text{tam}(X) \leftarrow \text{tam}(X) + \text{sizeof}(\text{cabeçalho}) + \text{tam}(Y),$$

pois o cabeçalho de Y deixa de existir e seu espaço volta a ser dado útil. Os encadeamentos (**proximo/anterior**) também precisam ser ajustados.

💡 Boundary tags (Knuth)

Para encontrar o vizinho *anterior* em tempo constante, sem um ponteiro **anterior**, Knuth propôs repetir o tamanho e o estado do bloco também no seu *final* (um “rodapé”). Assim, olhando os bytes imediatamente antes do cabeçalho atual, lê-se o rodapé do bloco anterior. Neste trabalho, a estrutura sugerida já tem o ponteiro **anterior**, que resolve o mesmo problema; boundary tags são leitura complementar.

4.2.10 Estratégias de posicionamento

Dado um pedido de n bytes e vários blocos livres que o comportam, qual escolher? As estratégias clássicas são:

Estratégia	Regra	Vantagem	Desvantagem
First-Fit	o <i>primeiro</i> bloco livre suficiente, a partir do início	rápida; para cedo	acumula fragmentos pequenos no início do heap
Next-Fit	como First-Fit, mas continua de onde a última busca parou	espalha as alocações	costuma fragmentar mais que First-Fit
Best-Fit	o <i>menor</i> bloco livre suficiente	desperdiça pouco no bloco escolhido	percorre a lista toda; deixa sobras minúsculas
Worst-Fit	o <i>maior</i> bloco livre	sobras grandes, reaproveitáveis	percorre tudo; “gasta” os blocos grandes

Exemplo visual. Blocos livres de 100, 50 e 650 bytes, nesta ordem de endereço, e um pedido de 40 bytes:

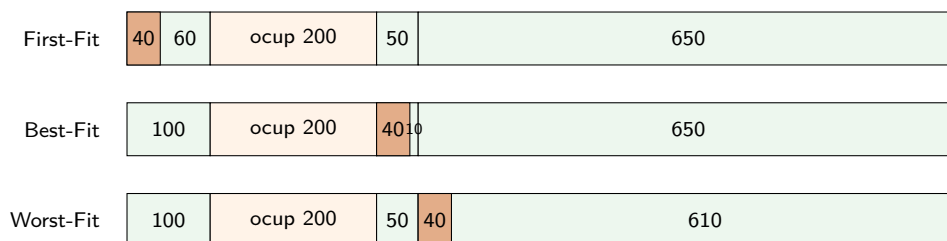


Figura 4.8: Onde cada estratégia coloca um pedido de 40 bytes (em laranja escuro).

- **First-Fit** usa o bloco de 100 (o primeiro que serve) e deixa 60 livres.
- **Best-Fit** usa o de 50 (o mais justo) e deixa uma sobra de 10 bytes — pequena demais para ser útil.
- **Worst-Fit** usa o de 650 e deixa 610, uma sobra grande e reaproveitável, mas “queima” o único bloco capaz de atender um pedido grande.

Nenhuma estratégia é superior em todos os cenários; é justamente isso que seus experimentos vão investigar.

4.2.11 Erros clássicos de uso da memória dinâmica

- **Double free:** chamar `free` duas vezes no mesmo ponteiro. Em alocadores reais corrompe as listas internas. Seu alocador deve detectar: se o bloco já está marcado como livre, reporte erro e não faça nada.
- **Ponteiro inválido:** chamar `free` com um endereço que não veio de `malloc` (fora do heap, ou no meio de um bloco). Detecte verificando se o endereço está dentro dos limites do heap e se corresponde ao início de dados de algum bloco (percorrendo a lista, ou conferindo um “número mágico” no cabeçalho).
- **Uso após liberação (*use after free*):** ler ou escrever em um bloco já liberado. O alocador não consegue impedir, mas o erro pode sobrescrever cabeçalhos. Ferramentas como Valgrind e AddressSanitizer detectam esse tipo de erro em programas C.
- **Estouro de bloco (*overflow*):** escrever além do tamanho alocado, destruindo o cabeçalho do bloco seguinte.
- **Vazamento (*leak*):** nunca liberar um bloco que não será mais usado.

4.2.12 Para ir além (leitura opcional)

- **Buddy system:** o heap é dividido recursivamente em metades (potências de 2). Cada bloco tem um “irmão” (*buddy*); ao liberar, se o irmão também estiver livre, eles se fundem. A coalescência é muito rápida, ao custo de fragmentação interna (pedidos são arredondados para potências de 2). É usado pelo núcleo Linux para gerenciar páginas físicas.
- **Listas segregadas (*segregated free lists*):** várias listas de blocos livres, uma para cada faixa de tamanho (8, 16, 32, ...). A busca começa direto na lista da faixa certa. É a base de alocadores reais como o `dlmalloc`/`ptmalloc` (glibc), `jmalloc` e `tcmalloc`.

4.3 Especificação do trabalho

4.3.1 Interface pública

O alocador deve oferecer as funções abaixo (em Python, métodos equivalentes de uma classe ou funções de um módulo):

C | interface obrigatória (alocador.h)

```
void mem_init(size_t heap_size);
void* meu_malloc(size_t tamanho);
void meu_free(void* ptr);
void mem_stats(void); /* relatório de uso */
void mem_dump(void); /* visualização do heap */
```

Python | interface equivalente

```
class Alocador:
    def __init__(self, heap_size: int, estrategia: str): ...
    def meu_malloc(self, tamanho: int) -> int | None: ... # devolve offset
    def meu_free(self, ptr: int) -> None: ...
    def mem_stats(self) -> dict: ...
    def mem_dump(self) -> None: ...
```

4.3.2 Estrutura de metadados sugerida

C | cabeçalho de bloco sugerido

```
typedef struct bloco {
    size_t tamanho; /* bytes de dados, sem o cabeçalho */
    int livre; /* 1 = livre, 0 = ocupado */
    struct bloco* proximo; /* vizinho físico seguinte */
    struct bloco* anterior; /* vizinho físico anterior */
} Bloco;
```

Você pode acrescentar campos, por exemplo o *tamanho originalmente pedido* (para medir fragmentação interna) ou um *número mágico* (para detectar ponteiros inválidos). Documente no relatório o tamanho final do seu cabeçalho.

4.3.3 Requisitos

Requisito	Descrição
Heap simulado	uma única região contígua, reservada uma vez em <code>mem_init</code>
Três estratégias	First-Fit, Best-Fit e Worst-Fit, selecionáveis por parâmetro (linha de comando, variável global ou compilação)
Splitting	dividir o bloco quando a sobra comportar cabeçalho + mínimo de dados
Coalescência	fundir livres adjacentes em <code>meu_free</code> (e poder desativá-la para experimento)
Alinhamento	tamanhos múltiplos de 8 bytes
Erros	detectar double free, ponteiro inválido, pedido maior que o heap / sem espaço contíguo
Métricas	fragmentação interna e externa, operações por alocação, taxa de falha

4.3.4 Workloads obrigatórios

1. **Alocações sequenciais** de tamanhos variados, sem liberação — mede uso puro de espaço e fragmentação interna.
2. **Alocações e liberações intercaladas** (pseudoaleatórias, com semente fixa) — mede fragmentação externa ao longo do tempo.
3. **Padrão adversarial**: alocar N blocos pequenos, liberar blocos alternados, tentar alocar um bloco grande que caberia na soma dos buracos, mas não em nenhum deles isoladamente.

4.3.5 Estrutura de entrega

Estrutura de diretórios (C)

```
mem-allocator/  
|-- src/  
|   |-- alocador.c / alocador.h  
|   |-- estrategias.c / estrategias.h  
|   |-- estatisticas.c / estatisticas.h  
|   '-- testes.c  
|-- benchmarks/  
|   |-- workload1.c  
|   |-- workload2.c  
|   '-- workload3.c  
|-- resultados/  
|   '-- comparativo.csv  
|-- Makefile  
'-- RELATORIO.pdf (+ fontes .tex)
```

Estrutura de diretórios (Python)

```
mem-allocator/  
|-- alocador/  
|   |-- __init__.py  
|   |-- alocador.py          # classe Alocador, heap em bytearray  
|   |-- estrategias.py  
|   '-- estatisticas.py  
|-- benchmarks/  
|   |-- workload1.py  
|   |-- workload2.py  
|   '-- workload3.py  
|-- resultados/  
|   '-- comparativo.csv  
|-- Makefile                # alvos: run, test, bench  
'-- RELATORIO.pdf (+ fontes .tex)
```

4.4 Como simular memória em Python

Python não expõe ponteiros. A solução é tratar um `bytearray` como a “memória” e usar **deslocamentos inteiros** (*offsets*) no lugar de ponteiros: o “endereço” de um bloco é o índice do primeiro byte do seu cabeçalho dentro do `bytearray`. O cabeçalho é gravado e lido com o módulo `struct`, que converte números em bytes com um formato fixo.

Python | formato do cabeçalho com struct

```
import struct

# tamanho(Q=8 bytes) | livre(I=4) | pad(4) | proximo(q=8) | anterior(q=8)
FMT_CAB = "<QIxxxqq"
TAM_CAB = struct.calcsize(FMT_CAB) # 32 bytes, múltiplo de 8
NULO = -1 # "ponteiro nulo" = offset -1

def escrever_cab(heap, off, tamanho, livre, prox, ant):
    struct.pack_into(FMT_CAB, heap, off, tamanho, livre, prox, ant)

def ler_cab(heap, off):
    return struct.unpack_from(FMT_CAB, heap, off) # tupla de 4 campos
```

Conceito	C	Python
Memória do heap	char heap[N] ou mmap	bytearray(N)
Ponteiro para bloco	Bloco *b	int off
Ponteiro nulo	NULL	-1 (constante NULO)
Ler campo	b->tamanho	ler_cab(heap, off)[0]
Área de dados	(char*)b + sizeof(Bloco)	off + TAM_CAB
Cabeçalho a partir do dado	(Bloco*)((char*)p - sizeof(Bloco))	p - TAM_CAB

⚠ Atenção

Em Python, **não** guarde a lista de blocos em uma `list` de objetos. Toda a informação de cada bloco deve viver dentro do `bytearray`. Um dicionário auxiliar é aceitável apenas para *estatísticas* (ex.: tamanho originalmente pedido), nunca para a lógica de alocação.

4.5 Roteiro de implementação

Siga os passos na ordem. Cada passo termina com uma verificação: não avance enquanto ela não passar. Os trechos de código mostram *estruturas e pontos de partida*; as partes marcadas com `TODO` ou `...` são sua responsabilidade.

4.5.1 Passo 1 — Estrutura de metadados do bloco

Defina o cabeçalho e as constantes do alocador (alinhamento, tamanho mínimo de divisão). Garanta que `sizeof(Bloco)` seja múltiplo de 8.

C | bloco.h

```
#define ALINHAMENTO 8
#define ALINHAR(n) (((n) + (ALINHAMENTO - 1)) & ~(size_t)(ALINHAMENTO - 1))
#define TAM_CAB ALINHAR(sizeof(Bloco))
#define MIN_DADOS 8 /* menor área de dados que vale a pena criar */

typedef struct bloco {
    size_t tamanho; /* dados úteis, já alinhado */
```

```

size_t pedido;           /* quanto o usuário pediu (stats) */
int     livre;
struct bloco *proximo, *anterior;
} Bloco;

```

Python | constantes

```

ALINHAMENTO = 8
MIN_DADOS = 8

```

```

def alinhar(n: int) -> int:
    return (n + ALINHAMENTO - 1) & ~(ALINHAMENTO - 1)

```

✓ Verifique antes de seguir

Imprima `sizeof(Bloco)` (C) ou `TAM_CAB` (Python) e teste `ALINHAR/alinhar` com 1, 8, 13 e 100: os resultados devem ser 8, 8, 16 e 104.

4.5.2 Passo 2 — Inicialização do heap simulado

`mem_init` reserva a região uma única vez e a transforma em **um único bloco livre** cujo tamanho é o total menos um cabeçalho.

📄 C | alocador.c — estado global e início de `mem_init`

```

static char *heap_base = NULL; /* início da região */
static size_t heap_total = 0;
static Bloco *primeiro = NULL; /* cabeça da lista implícita */

void mem_init(size_t heap_size) {
    heap_base = malloc(heap_size); /* ÚNICO uso do malloc real */
    if (heap_base == NULL) { /* TODO: reportar erro */ }
    heap_total = heap_size;
    primeiro = (Bloco *) heap_base;
    /* TODO: preencher tamanho, livre, proximo e anterior */
}

```

Python | construtor

```

class Alocador:
    def __init__(self, heap_size, estrategia="first"):
        self.heap = bytearray(heap_size) # ÚNICA "alocação real"
        self.total = heap_size
        self.primeiro = 0                # offset do 1o bloco
        # TODO: escrever o cabeçalho do bloco livre inicial
        ...

```

👉 Dica

Alternativa em C: `mmap(NULL, n, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)` obtém a região diretamente do SO (lembra de liberar com `munmap` ao final). Um array estático `static char heap[TAM]` também é aceitável.

✓ Verifique antes de seguir

Depois de `mem_init(1000)`, a lista deve ter exatamente um bloco, livre, com tamanho `1000 - TAM_CAB` (968 com cabeçalho de 32 bytes).

4.5.3 Passo 3 — Percurso e exibição do heap (`mem_dump`)

Implemente o `mem_dump` *antes* de alocar qualquer coisa: ele será sua principal ferramenta de depuração. Ele percorre a lista do primeiro ao último bloco e imprime deslocamento (relativo ao início do heap), tamanho e estado.

</> C | esqueleto do percurso

```
void mem_dump(void) {
    printf("%-10s %-8s %s\n", "Endereco", "Tamanho", "Estado");
    for (Bloco *b = primeiro; b != NULL; b = b->proximo) {
        size_t off = (size_t)((char *)b - heap_base);
        /* TODO: imprimir off, b->tamanho e LIVRE/OCUPADO */
    }
}
```

Python | esqueleto do percurso

```
def mem_dump(self):
    print(f"{'Endereco':<10}{ 'Tamanho':<9}Estado")
    off = self.primeiro
    while off != NULO:
        tam, livre, prox, ant = ler_cab(self.heap, off)
        # TODO: imprimir a linha
        off = prox
```

👉 Dica

Acrescente ao `mem_dump` uma *verificação de consistência*: para cada bloco, confira que `b->proximo->anterior == b` e que `off + TAM_CAB + tamanho` é exatamente o deslocamento do próximo bloco. Se algo não bater, imprima um alerta. Isso encontra a maioria dos bugs de splitting e coalescência na hora em que acontecem.

✓ Verifique antes de seguir

Logo após `mem_init`, `mem_dump` mostra uma única linha, e a verificação de consistência não emite alertas.

4.5.4 Passo 4 — Estratégias de busca de bloco livre

Cada estratégia é uma função que recebe o início da lista e o tamanho (já alinhado) e devolve o bloco escolhido, ou nulo. As funções **apenas escolhem**: não alteram a lista. Instrumente-as desde já com um contador de blocos examinados.

</> C | estrategias.h e seleção da estratégia

```
typedef enum { FIRST_FIT, BEST_FIT, WORST_FIT } Estrategia;
```

```
Bloco *buscar_first_fit(Bloco *inicio, size_t tam, size_t *examinados);
Bloco *buscar_best_fit (Bloco *inicio, size_t tam, size_t *examinados);
Bloco *buscar_worst_fit(Bloco *inicio, size_t tam, size_t *examinados);

typedef Bloco *(*FuncaoBusca)(Bloco *, size_t, size_t *);
static FuncaoBusca buscar; /* definida em mem_init conforme a estratégia */
```

🔗 C | ideia do Best-Fit (incompleto)

```
Bloco *buscar_best_fit(Bloco *inicio, size_t tam, size_t *examinados) {
    Bloco *melhor = NULL;
    for (Bloco *b = inicio; b != NULL; b = b->proximo) {
        (*examinados)++;
        /* TODO: se b é candidato e é melhor que 'melhor', atualize */
    }
    return melhor;
}
```

🐍 Python | seleção por dicionário

```
def _first_fit(self, tam): ... # TODO
def _best_fit(self, tam): ... # TODO
def _worst_fit(self, tam): ... # TODO

# no __init__:
self.buscar = {"first": self._first_fit,
               "best": self._best_fit,
               "worst": self._worst_fit}[estrategia]
```

⚠️ Atenção

Critério de desempate: no Best-Fit e no Worst-Fit, se dois blocos tiverem o mesmo tamanho, escolha o de menor endereço (o primeiro encontrado). Documente essa escolha: ela afeta a reprodutibilidade.

✅ Verifique antes de seguir

Monte manualmente um heap com livres de 100, 50 e 650 (use as funções dos próximos passos ou escreva cabeçalhos “na mão”) e confirme que, para um pedido de 40, First-Fit devolve o de 100, Best-Fit o de 50 e Worst-Fit o de 650.

4.5.5 Passo 5 — Alocação sem divisão

Primeira versão de `meu_malloc`: alinha o pedido, busca um bloco, marca como ocupado e devolve o endereço da área de dados. Ainda *sem* dividir.

🔗 C | estrutura de meu_malloc (versão inicial)

```
void *meu_malloc(size_t tamanho) {
    if (tamanho == 0) return NULL;
    size_t tam = ALINHAR(tamanho);
    size_t exam = 0;
    Bloco *b = buscar(primeiro, tam, &exam);
```

```

/* TODO: registrar 'exam' nas estatísticas */
/* TODO: se b == NULL, contabilizar falha e devolver NULL */
/* TODO: marcar ocupado, guardar 'tamanho' em b->pedido */
return (char *)b + TAM_CAB;
}

```

Python | estrutura de meu_malloc (versão inicial)

```

def meu_malloc(self, tamanho):
    if tamanho <= 0:
        return None
    tam = alinhar(tamanho)
    off = self.buscar(tam)
    if off == NULO:
        ... # TODO: contabilizar falha
        return None
    # TODO: marcar ocupado (reescrever o cabeçalho)
    return off + TAM_CAB # "ponteiro" = offset da área de dados

```

Verifique antes de seguir

Faça três alocações seguidas e chame `mem_dump`: sem divisão, a primeira consome o heap inteiro e as outras duas devem falhar (devolver nulo). Isso é o esperado neste passo!

4.5.6 Passo 6 — Divisão de blocos (*splitting*)

Escreva uma função auxiliar `dividir(b, tam)` chamada por `meu_malloc` logo após a busca. Ela cria um novo cabeçalho *dentro* do bloco livre, logo após os `tam` bytes de dados, e o insere na lista.

C | onde fica o novo bloco (fragmento)

```

static void dividir(Bloco *b, size_t tam) {
    if (b->tamanho < tam + TAM_CAB + MIN_DADOS)
        return; /* sobra pequena: não divide */
    Bloco *novo = (Bloco *)((char *)b + TAM_CAB + tam);
    novo->tamanho = /* TODO */;
    novo->livre = 1;
    /* TODO: encadear 'novo' entre b e b->proximo (4 ponteiros!) */
    b->tamanho = tam;
}

```

Python | onde fica o novo bloco (fragmento)

```

def _dividir(self, off, tam):
    tam_b, livre, prox, ant = ler_cab(self.heap, off)
    if tam_b < tam + TAM_CAB + MIN_DADOS:
        return
    novo = off + TAM_CAB + tam
    # TODO: escrever cabeçalho de 'novo' e religar prox/ant
    ...

```

⚠ Atenção

São quatro atualizações de encadeamento: `novo->anterior`, `novo->proximo`, `b->proximo` e, se existir, o `anterior` do antigo seguinte de `b`. Esquecer a última é o bug mais comum deste passo — a verificação de consistência do `mem_dump` vai acusá-lo.

✓ Verifique antes de seguir

Com heap de 1000 bytes e cabeçalho de 32: após `meu_malloc(100)` devem existir dois blocos — ocupado de 104 (offset 0) e livre de $968 - 104 - 32 = 832$ (offset 136).

4.5.7 Passo 7 — Liberação sem coalescência

`meu_free` recupera o cabeçalho a partir do ponteiro, *valida*, e marca como livre.

🔗 C | validações de meu_free (esboço)

```
void meu_free(void *ptr) {
    if (ptr == NULL) return;
    char *p = (char *)ptr;
    if (p < heap_base + TAM_CAB || p >= heap_base + heap_total) {
        fprintf(stderr, "ERR0: ponteiro fora do heap\n");
        return;
    }
    Bloco *b = (Bloco *) (p - TAM_CAB);
    /* TODO: confirmar que b é realmente um bloco da lista */
    /* TODO: se b->livre, reportar double free e retornar */
    /* TODO: marcar como livre */
}
```

🔗 Python | validações de meu_free (esboço)

```
def meu_free(self, ptr):
    if ptr is None:
        return
    off = ptr - TAM_CAB
    if not self._eh_bloco_valido(off):      # TODO: implementar
        print("ERR0: ponteiro invalido")
        return
    ... # TODO: double free? marcar livre
```

👉 Dica

Duas formas de confirmar que um endereço é início de bloco: (a) percorrer a lista procurando o (simples, $O(n)$); (b) gravar um *número mágico* (ex.: `0xB10C0DE`) no cabeçalho em toda criação de bloco e conferi-lo em `meu_free` (rápido, mas probabilístico). Discuta a escolha no relatório.

✓ Verifique antes de seguir

Aloque A, B e C, libere B e confira no `mem_dump` que apenas B mudou de estado. Chame `meu_free` de novo em B: deve aparecer a mensagem de double free, e o heap não deve mudar. Chame `meu_free` com um endereço no meio de A: deve aparecer “ponteiro inválido”.

4.5.8 Passo 8 — Coalescência de blocos livres adjacentes

Após marcar o bloco como livre, verifique o **seguinte** e depois o **anterior**. Escreva uma função `fundir(a, b)` que absorve **b** (livre, imediatamente após **a**) em **a**; os quatro casos da Figura 4.7 se resolvem com no máximo duas chamadas dela.

C | fusão de dois vizinhos (parcial)

```
/* pré-condição: a e b livres e b == a->proximo */
static void fundir(Bloco *a, Bloco *b) {
    a->tamanho += TAM_CAB + b->tamanho;
    /* TODO: religar a->proximo e o 'anterior' do seguinte de b */
}

/* em meu_free, depois de marcar livre: */
if (coalescencia_ativa) {
    /* TODO: caso seguinte livre -> fundir(b, b->proximo) */
    /* TODO: caso anterior livre -> fundir(b->anterior, b) */
}
```

Python | fusão de dois vizinhos (parcial)

```
def _fundir(self, a, b):
    tam_a, _, _, ant_a = ler_cab(self.heap, a)
    tam_b, _, prox_b, _ = ler_cab(self.heap, b)
    # TODO: novo tamanho de a, novo proximo, e ajustar prox_b.anterior
    ...
```

Atenção

Mantenha uma variável/flag `coalescencia_ativa` (ou opção de linha de comando): o experimento da Seção 4.7 exige rodar o mesmo workload com e sem coalescência.

Verifique antes de seguir

Aloque três blocos consecutivos (A, B, C) e libere A e depois B: o `mem_dump` deve mostrar *um único* livre no lugar dos dois. Libere C: o heap inteiro volta a ser um único bloco livre, idêntico ao estado logo após `mem_init`.

4.5.9 Passo 9 — Estatísticas e métricas

`mem_stats` percorre o heap e calcula as métricas da fundamentação.

C | estrutura de estatísticas

```
typedef struct {
    size_t total_livre, total_ocupado, maior_livre;
    size_t n_livres, n_ocupados;
    size_t frag_interna; /* soma de tamanho - pedido */
    size_t n_malloc, n_falhas, n_falhas_frag;
    size_t blocos_examinados; /* soma das buscas */
} Estatisticas;
```

Python | cálculo da fragmentação externa

```
def frag_externa(total_livre, maior_livre):
    if total_livre == 0:
        return 0.0
    return 1 - maior_livre / total_livre
```

Métrica	Como obter
Fragmentação interna	somar tamanho - pedido dos ocupados (ou somar também sobras não divididas)
Fragmentação externa	$1 - \text{maior_livre} / \text{total_livre}$
Custo médio por alocação	$\text{blocos_examinados} / \text{n_malloc}$
Taxa de falha	$\text{n_falhas} / \text{n_malloc}$
Falha por fragmentação	falha em que $\text{total_livre} \geq \text{tamanho pedido}$

Verifique antes de seguir

No cenário do Passo 8, antes de liberar C, `mem_stats` deve informar 2 blocos livres; após liberar C, 1 bloco livre e fragmentação externa 0.

4.5.10 Passo 10 — Tratamento de erros e robustez

Revise todos os caminhos de erro:

- `meu_malloc(0)` e tamanhos negativos (Python) → nulo, sem mexer no heap.
- Pedido maior que o heap inteiro → nulo com mensagem clara.
- Pedido que não cabe em nenhum bloco, embora `total_livre` seja suficiente → nulo, contabilizado como falha por fragmentação externa.
- `meu_free(NULL)` → nada; ponteiro fora do heap, no meio de bloco ou já livre → mensagem, heap intacto.
- Chamadas antes de `mem_init` → erro, não *segfault*.

C | compilando com detectores de erro

```
/* No Makefile:
 *   CFLAGS = -Wall -Wextra -g -fsanitize=address,undefined
 * E rode também: valgrind --leak-check=full ./testes */
```

Verifique antes de seguir

Rode seus testes sob AddressSanitizer ou Valgrind: nenhum acesso fora da região reservada, e nenhum vazamento do heap real ao final (libere-o em uma função `mem_destroy`).

4.6 Exemplos de entrada e saída

Para facilitar os testes, sugere-se um programa *driver* que lê comandos de um arquivo:

 comandos.txt

```
# comando  argumento
ALLOC      100
ALLOC      200
ALLOC      50
FREE       2
ALLOC      120
DUMP
```

ALLOC <tamanho> solicita um bloco; FREE <id> libera o bloco identificado pela ordem de alocação (1, 2, 3, ...); DUMP imprime o heap.

 Atenção

Simplificação didática nos exemplos abaixo: para deixar os números fáceis de acompanhar, os exemplos consideram o cabeçalho com tamanho **zero** e tamanhos já múltiplos de 8 ou ignoram o alinhamento. Na sua implementação real, cada bloco ocupará **TAM_CAB** bytes a mais, os endereços serão deslocados e os tamanhos serão alinhados; portanto os números exatos da sua saída serão diferentes. O que deve coincidir é o *comportamento*: qual bloco é escolhido, quando há divisão, quando há fusão e quando há falha.

Cenário 2 — estado inicial

>_ Heap de 1000 bytes, First-Fit, DUMP sem alocações

Endereco	Tamanho	Estado
0	1000	LIVRE

Valida: inicialização como um único bloco livre.

Cenário 3 — alocações sequenciais (First-Fit)

>_ ALLOC 100, ALLOC 200, ALLOC 50, DUMP

```
Bloco 1 alocado em endereco 0, tamanho 100
Bloco 2 alocado em endereco 100, tamanho 200
Bloco 3 alocado em endereco 300, tamanho 50
```

Endereco	Tamanho	Estado
0	100	OCUPADO (bloco 1)
100	200	OCUPADO (bloco 2)
300	50	OCUPADO (bloco 3)
350	650	LIVRE

Valida: em heap vazio, todas as estratégias produzem o mesmo resultado; cada alocação divide o bloco livre final.

Cenário 4 — liberação e reuso (First-Fit)

>_ Continuação: FREE 1, ALLOC 80, DUMP

Bloco 1 (tamanho 100, endereço 0) liberado
Bloco 4 alocado em endereço 0, tamanho 80

Endereço	Tamanho	Estado
0	80	OCUPADO (bloco 4)
80	20	LIVRE
100	200	OCUPADO (bloco 2)
300	50	OCUPADO (bloco 3)
350	650	LIVRE

Valida: First-Fit escolhe o primeiro livre suficiente (o de 100), e a sobra de 20 vira um novo livre. Observe que, com cabeçalho real de 32 bytes, uma sobra de 20 *não* comportaria cabeçalho + MIN_DADOS: sua implementação entregaria o bloco inteiro e os 20 bytes seriam contabilizados como fragmentação interna.

Cenário 5 — Best-Fit

>_ Heap 1000, Best-Fit: ALLOC 100, ALLOC 200, ALLOC 50, FREE 1, FREE 3, ALLOC 40

Endereço	Tamanho	Estado	(antes do ALLOC 40)
0	100	LIVRE	
100	200	OCUPADO	(bloco 2)
300	50	LIVRE	
350	650	LIVRE	

Bloco 4 alocado em endereço 300, tamanho 40

Endereço	Tamanho	Estado
0	100	LIVRE
100	200	OCUPADO (bloco 2)
300	40	OCUPADO (bloco 4)
340	10	LIVRE
350	650	LIVRE

Valida: Best-Fit escolhe o menor livre suficiente (50). Note dois detalhes: (i) os livres em 300 (50 bytes) e 350 (650 bytes) *são adjacentes* — com coalescência ativa, ao executar **FREE 3** eles seriam fundidos em um livre de 700 bytes, e o Best-Fit escolheria então o bloco de 100 (endereço 0). O resultado acima corresponde, portanto, à execução **sem coalescência**; rode o cenário nos dois modos e explique a diferença no relatório. (ii) A sobra de 10 bytes é o “fragmento inútil” típico do Best-Fit.

Cenário 6 — Worst-Fit

>_ Mesma sequência, Worst-Fit (sem coalescência)

Bloco 4 alocado em endereço 350, tamanho 40

Endereço	Tamanho	Estado
0	100	LIVRE

100	200	OCUPADO (bloco 2)
300	50	LIVRE
350	40	OCUPADO (bloco 4)
390	610	LIVRE

Valida: Worst-Fit escolhe o maior livre (650), deixando uma sobra grande. Compare com o Cenário 5.

Cenário 7 — coalescência

```
>_ ALLOC 100 (1), ALLOC 100 (2), ALLOC 100 (3), FREE 1, FREE 2, DUMP
```

```
Bloco 1 liberado
Bloco 2 liberado -- mesclado com bloco 1 (blocos livres adjacentes)
```

Endereco	Tamanho	Estado
0	200	LIVRE
200	100	OCUPADO (bloco 3)
300	700	LIVRE

Valida: caso 3 da Figura 4.7 (anterior livre). Na implementação real o livre fundido terá $100 + \text{TAM_CAB} + 100$ bytes de dados, pois o cabeçalho do bloco 2 é reaproveitado.

Cenário 8 — falha por fragmentação externa

```
>_ Heap de 300 bytes: ALLOC 100 x3, FREE 1, FREE 3, ALLOC 150
```

```
Bloco 1 liberado (100 bytes livres em 0)
Bloco 3 liberado (100 bytes livres em 200)
ERRO: ALLOC 150 falhou -- sem bloco contiguo suficiente
      (memoria livre total: 200 bytes, mas fragmentada em 2 blocos de 100)
```

Valida: há memória livre total suficiente, mas não contígua; o bloco 2 separa os dois buracos. Fragmentação externa = $1 - 100/200 = 50\%$.

Cenário 9 — liberação inválida

```
>_ ALLOC 100, FREE 99
```

```
Bloco 1 alocado em endereco 0, tamanho 100
ERRO: tentativa de liberar bloco inexistente (id 99)
```

Valida: entradas inválidas geram mensagem e não corrompem o heap. Teste também `FREE 1` duas vezes (double free).

4.7 Experimentos

Execute cada workload sob as três estratégias. Use sempre a **mesma semente** do gerador pseudoaleatório para as três, de modo que todas recebam exatamente a mesma sequência de pedidos.

C | reprodutibilidade

```
srand(42); /* mesma semente p/ as 3 estratégias */
size_t t = 10 + rand() % 141; /* tamanho entre 10 e 150 */
```

Python | reprodutibilidade

```
import random
rng = random.Random(42) # gerador próprio, semente fixa
t = rng.randint(10, 150)
```

1. **Workload 1 — sequencial sem liberação.** Aloque tamanhos variados até o heap esgotar. Meça: número de alocações bem-sucedidas, fragmentação interna total, overhead de metadados.
2. **Workload 2 — intercalado aleatório.** Por exemplo, heap de 2000 bytes e 100 operações alternando **ALLOC** (10 a 150 bytes) e **FREE** de blocos aleatórios ainda ativos. Registre a fragmentação externa *ao longo do tempo* (a cada operação) e faça um gráfico linha (operação $\times$ fragmentação), uma curva por estratégia.
3. **Workload 3 — adversarial.** Aloque N blocos pequenos, libere os de índice par, peça um bloco com tamanho igual à soma dos liberados. Registre se cada estratégia falha e por quê.
4. **Coalescência ligada $\times$ desligada.** Repita o workload 3 (e o cenário 7) com a coalescência desativada.
5. **Escala.** Repita o workload 2 com heaps de 1000, 5000 e 20000 bytes e com 100, 1000 e 10000 operações; observe o custo médio por alocação (blocos examinados).
6. **(Opcional) Referência:** rode o mesmo benchmark usando o **malloc/free** da biblioteca padrão (em C) e compare o tempo.

Consolide os resultados em uma tabela como a abaixo. **Ela é apenas um modelo:** os valores são os que *você* medir.

Estratégia	Aloc. OK	Falhas	Frag. ext. (%)	Maior livre	Exam./aloc.
First-Fit	?	?	?	?	?
Best-Fit	?	?	?	?	?
Worst-Fit	?	?	?	?	?

resultados/comparativo.csv (formato sugerido)

```
workload,estrategia,coalescencia,heap,ops,aloc_ok,falhas,falhas_frag,frag_int,
frag_ext,maior_livre,exam_medio
w2,first,1,2000,100,,,,,,
w2,best,1,2000,100,,,,,,
w2,worst,1,2000,100,,,,,,
```

Dica

Um gráfico de “barra do heap” (blocos coloridos lado a lado, livre em verde e ocupado em laranja) ao final de cada workload, uma barra por estratégia, é a forma mais convincente de mostrar a fragmentação no relatório. Você pode gerá-lo em Python (`matplotlib.pyplot.barh`)

a partir de um `mem_dump` exportado em CSV.

4.8 Perguntas para o relatório

1. Qual estratégia apresentou menor fragmentação externa no padrão adversarial? Isso confirma a teoria (Best-Fit tende a gerar muitos fragmentos pequenos)?
2. Qual o custo, em número de blocos examinados, de cada estratégia à medida que o heap fica mais fragmentado? Relacione com a complexidade ($O(n)$ no número de blocos) e com o fato de First-Fit poder parar cedo.
3. Como a coalescência afeta os resultados? Mostre a comparação com ela desativada.
4. Qual é o tamanho do seu cabeçalho e qual o overhead de metadados no workload de muitos blocos pequenos? Como ele poderia ser reduzido?
5. Que valor de tamanho mínimo de divisão você escolheu e qual o seu efeito sobre a fragmentação interna?
6. Como seu alocador detecta double free e ponteiros inválidos? Quais os limites dessa detecção?

4.9 Critérios de avaliação

Critério	Peso
Corretude da alocação/liberação (sem corromper o heap)	30%
Implementação das 3 estratégias	30%
Splitting e coalescência funcionando	15%
Análise comparativa de fragmentação	15%
Tratamento de erros	10%

Checklist de entrega

- ☐ Compila e executa em Linux (nativo ou WSL); em C, sem warnings com `-Wall -Wextra`.
- ☐ `Makefile` na raiz; `make` gera o executável (ou, em Python, `make run/make bench` funcionam); `make clean` limpa.
- ☐ Heap reservado uma única vez; metadados dentro do heap (`bytearray` em Python).
- ☐ First-Fit, Best-Fit e Worst-Fit implementados e selecionáveis.
- ☐ Splitting com tamanho mínimo; coalescência com opção de desativar.
- ☐ Alinhamento a 8 bytes.
- ☐ `mem_dump` e `mem_stats` funcionando.
- ☐ Pedido sem espaço devolve nulo; `meu_free(NULL)` não falha; double free e ponteiro inválido detectados.
- ☐ Sem acessos fora do heap nem vazamentos (Valgrind/AddressSanitizer, em C).
- ☐ Três workloads executados nas três estratégias, com semente fixa; experimento de coalescência on/off.

- ☐ Resultados em CSV e gráficos no relatório.
- ☐ Relatório no Overleaf, modelo SBC, com fontes `.tex` e PDF.
- ☐ Sem plágio e sem código gerado por IA.

4.10 Leituras recomendadas

- ARPACI-DUSSEAU, R.; ARPACI-DUSSEAU, A. *Operating Systems: Three Easy Pieces*, capítulo *Free-Space Management*. <https://pages.cs.wisc.edu/~remzi/OSTEP/vm-freespace.pdf>
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*, capítulo de Gerência de Memória (alocação contígua, fragmentação, *dynamic storage-allocation problem*).
- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*, capítulo 3 (gerência de memória com listas livres).
- KERNIGHAN, B. W.; RITCHIE, D. M. *The C Programming Language*, seção 8.7 — um alocador de memória simples, como estudo de caso.
- KERRISK, M. *The Linux Programming Interface*, capítulo 7 (Memory Allocation).
- KNUTH, D. E. *The Art of Computer Programming*, vol. 1, seção 2.5 (alocação dinâmica, *boundary tags*).
- Páginas de manual: `man 2 brk`, `man 2 mmap`, `man 3 malloc`.
- Documentação do Python: módulo `struct` (<https://docs.python.org/3/library/struct.html>).

Capítulo 5

Problemas Clássicos de Sincronização

5.1 Visão geral

Até aqui, todos os programas que você escreveu provavelmente executavam *uma coisa de cada vez*: uma instrução depois da outra, em uma única linha de execução. Os sistemas reais, porém, são **concorrentes**: um navegador baixa imagens enquanto desenha a página; um servidor web atende milhares de clientes “ao mesmo tempo”; o próprio núcleo do sistema operacional trata interrupções enquanto executa processos. Sempre que duas ou mais linhas de execução compartilham dados, surge uma classe de erros completamente nova — erros que *não aparecem sempre*, que dependem da ordem exata em que as instruções foram intercaladas e que, muitas vezes, somem quando você tenta depurá-los.

Neste trabalho você vai implementar, com **threads**, três problemas clássicos da literatura de Sistemas Operacionais — o **Produtor-Consumidor**, o **Jantar dos Filósofos** e os **Leitores-Escritores** — usando as primitivas de sincronização oferecidas pelo sistema: *mutexes*, *semáforos* e *variáveis de condição*. Além das versões corretas, você vai construir, de propósito, versões **com defeito** e documentar como um engenheiro diagnostica deadlocks, condições de corrida e inanição.

🕒 Ao final deste trabalho você deverá ser capaz de

- explicar a diferença entre concorrência e paralelismo, e entre processos e threads;
- reconhecer uma **condição de corrida** e identificar a **seção crítica** de um trecho de código;
- usar corretamente mutexes, semáforos e variáveis de condição em C (`pthread`, `semaphore.h`) ou em Python (`threading`);
- enunciar as quatro condições necessárias para **deadlock** e aplicar uma estratégia que quebre ao menos uma delas;
- distinguir deadlock, *livelock* e *starvation* (inanição);
- resolver o Produtor-Consumidor, o Jantar dos Filósofos e o Leitores-Escritores com prioridade para escritores;
- provocar, observar e diagnosticar bugs de concorrência usando logs e ferramentas (ThreadSanitizer, Helgrind, `gdb`, `py-spy`).

5.2 Fundamentação teórica

Esta seção apresenta, do zero, os conceitos necessários para o trabalho. Leia com calma e *antes* de escrever qualquer código: em programação concorrente, entender o problema é mais da metade da solução.

5.2.1 Concorrência e paralelismo

Definição: Concorrência

Duas atividades são **concorrentes** quando seus intervalos de execução se sobrepõem no tempo: uma começa antes de a outra terminar. Não importa se elas realmente executam no mesmo instante físico.

Definição: Paralelismo

Há **paralelismo** quando duas atividades executam *literalmente ao mesmo tempo*, em unidades de processamento (núcleos) diferentes.

Em uma máquina com um único núcleo, o sistema operacional cria a *ilusão* de simultaneidade alternando rapidamente entre as tarefas (isso é estudado no capítulo de escalonamento de CPU). Há concorrência, mas não paralelismo. Em uma máquina com vários núcleos, há as duas coisas. A Figura 5.1 ilustra a diferença.

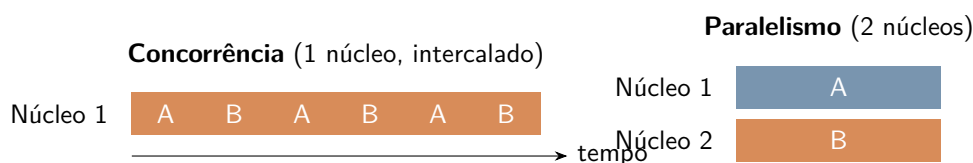


Figura 5.1: Concorrência versus paralelismo.

Atenção

Os bugs estudados neste capítulo aparecem **mesmo em um único núcleo**. Basta que o sistema operacional interrompa uma thread no meio de uma operação e passe a CPU para outra. Por isso, “minha máquina tem um núcleo só” ou “em Python há o GIL” *não* tornam o código seguro.

5.2.2 Processos e threads

Um **processo** é um programa em execução, com seu próprio espaço de endereçamento (memória isolada dos outros processos), seus arquivos abertos e pelo menos uma linha de execução. Uma **thread** (*fio de execução*) é uma linha de execução *dentro* de um processo. Um processo pode ter várias threads; todas compartilham:

- o código do programa;
- as variáveis globais e estáticas;
- o *heap* (tudo o que foi alocado com `malloc` ou, em Python, todos os objetos);
- os arquivos abertos.

Cada thread, porém, possui **privativamente**: seus registradores (incluindo o contador de programa, que diz qual instrução executar a seguir) e sua **pilha** (onde ficam as variáveis locais e os parâmetros das funções).

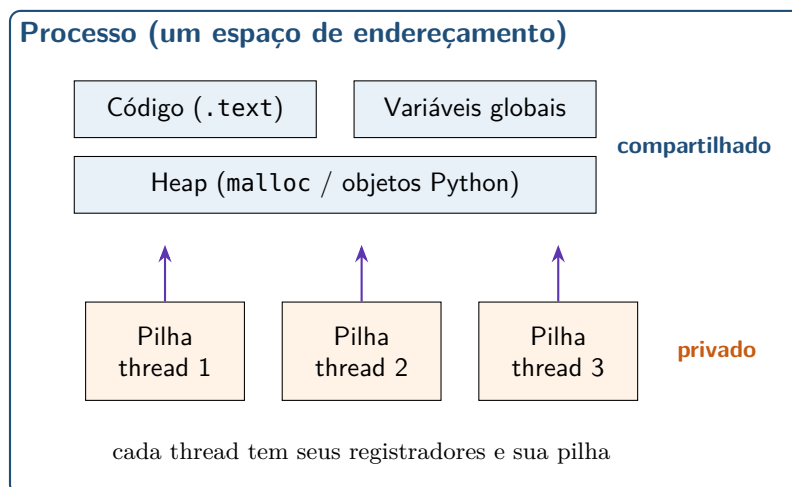


Figura 5.2: Threads de um mesmo processo compartilham código, globais e heap, mas têm pilhas e registradores próprios.

Criar uma thread é bem mais barato que criar um processo, e a comunicação entre threads é trivial: basta ler e escrever a mesma variável. É exatamente essa facilidade que torna a sincronização necessária.

💡 Threads em C e em Python

Em C, no Linux, usamos a biblioteca **POSIX Threads** (pthread): `pthread_create` cria uma thread que executa uma função e `pthread_join` espera seu término. Compile e ligue com a opção `-pthread`. Em Python, usamos o módulo `threading`: a classe `threading.Thread(target=funcao, args=(...))`, com os métodos `start()` e `join()`.

📄 C | criando e esperando threads (fragmento)

```
#include <pthread.h>

void *trabalho(void *arg) {
    int id = *(int *)arg;
    /* ... o que a thread faz ... */
    return NULL;
}

/* em main(): */
pthread_t t[N];
int ids[N];
for (int i = 0; i < N; i++) {
    ids[i] = i;
    pthread_create(&t[i], NULL, trabalho, &ids[i]);
}
/* TODO: esperar todas com pthread_join */
```

Python | criando e esperando threads (fragmento)

```
import threading

def trabalho(id_thread):
    ... # o que a thread faz

threads = [threading.Thread(target=trabalho, args=(i,)) for i in range(N)]
for t in threads:
    t.start()
# TODO: esperar todas com join()
```

⚠ Atenção

Em C, **não** passe `&i` (o endereço da variável do laço) para `pthread_create`: quando a thread for ler o valor, o laço pode já ter alterado `i`. Isso, por si só, já é uma condição de corrida! Use um vetor de argumentos, um por thread, como no fragmento acima.

5.2.3 Condição de corrida

Considere duas threads que executam, cada uma, um milhão de vezes a instrução `contador++` sobre uma variável global. O resultado esperado é 2 000 000. Na prática, o valor impresso costuma ser menor e *muda a cada execução*. Por quê?

Porque `contador++` **não é uma operação indivisível**. O compilador a traduz em (pelo menos) três instruções de máquina:

📄 contador++ em linguagem de montagem (simplificado)

```
LOAD R1, [contador]    ; copia o valor da memória para o registrador
ADD  R1, R1, 1          ; soma 1 no registrador
STORE [contador], R1    ; grava o registrador de volta na memória
```

Se o sistema operacional interromper a thread A entre o `LOAD` e o `STORE`, a thread B pode ler o valor *antigo*. A Tabela 5.1 mostra uma intercalação possível partindo de `contador = 5`.

Tabela 5.1: Uma intercalação que perde um incremento.

Passo	Thread A	Thread B	Registradores	contador
1	LOAD R1		A.R1 = 5	5
2		LOAD R1	B.R1 = 5	5
3	ADD R1		A.R1 = 6	5
4		ADD R1	B.R1 = 6	5
5	STORE			6
6		STORE		6 (deveria ser 7)

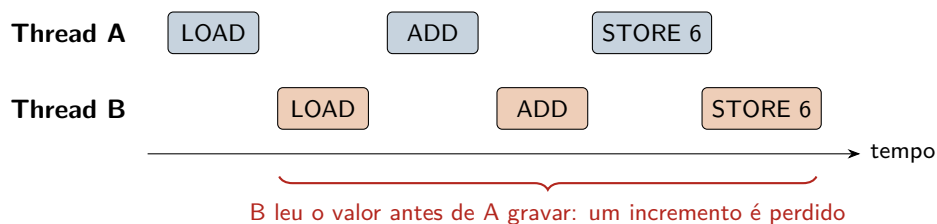


Figura 5.3: Linha do tempo da intercalação da Tabela 5.1.

Definição: Condição de corrida (*race condition*)

Situação em que o resultado de um programa depende da ordem (ou do *timing*) em que threads concorrentes acessam dados compartilhados, e pelo menos um desses acessos é uma escrita. Um caso particular é a **corrida de dados** (*data race*): dois acessos à mesma posição de memória, por threads diferentes, sem sincronização entre eles, sendo ao menos um deles de escrita.

Outro padrão perigoso muito comum é o **verificar-e-agir** (*check-then-act*): a thread verifica uma condição e, com base nela, age — mas entre a verificação e a ação outra thread muda o estado.

Python | check-then-act: parece correto, mas não é

```
if contador_itens < TAM_BUFFER:      # (1) verifica: "há espaço"
    # <- outra thread pode inserir aqui e lotar o buffer!
    buffer[entrada] = item           # (2) age com base em (1), já obsoleta
    contador_itens += 1
```

5.2.4 Seção crítica

Definição: Seção crítica

Trecho de código que acessa um recurso compartilhado (variável, estrutura de dados, arquivo) e que, portanto, **não pode** ser executado por mais de uma thread ao mesmo tempo.

O **problema da seção crítica** é projetar um protocolo que cada thread executa antes (*protocolo de entrada*) e depois (*protocolo de saída*) da seção crítica:

estrutura geral de uma thread

```
repita:
    protocolo de ENTRADA      <- pede permissão
    SEÇÃO CRÍTICA            <- acessa dados compartilhados
    protocolo de SAÍDA       <- devolve a permissão
    seção restante           <- código que não toca dados compartilhados
```

Uma solução correta deve satisfazer três requisitos:

1. **Exclusão mútua:** se uma thread está executando sua seção crítica, nenhuma outra pode estar executando a mesma seção crítica.
2. **Progresso:** se nenhuma thread está na seção crítica e algumas querem entrar, a decisão de quem entra não pode ser adiada indefinidamente (e só participam dela as threads que querem entrar).

3. **Espera limitada** (*bounded waiting*): existe um limite para o número de vezes que outras threads podem entrar na seção crítica depois que uma thread pediu para entrar e antes de ela ser atendida. Em outras palavras: ninguém espera para sempre.

Dica

Deixe a seção crítica **tão curta quanto possível**. Tudo o que não precisa de exclusão mútua (gerar um número aleatório, dormir simulando trabalho, imprimir estatísticas locais) deve ficar *fora* dela. Uma seção crítica longa serializa as threads e destrói o ganho de concorrência.

5.2.5 Espera ativa versus bloqueio

Uma forma ingênua de esperar é ficar num laço testando uma variável: `while (!pode_entrar) { }`. Isso é **espera ativa** (*busy waiting*): a thread consome CPU sem fazer nada útil, roubando tempo justamente da thread que poderia liberar o recurso. A alternativa é o **bloqueio**: a thread pede ao sistema operacional para ser *suspensa* (sai da fila de prontos) e só é acordada quando a condição pode ter mudado. Todas as primitivas usadas neste trabalho (mutex, semáforo, variável de condição) bloqueiam.

Atenção

Espera ativa (inclusive com `usleep` ou `time.sleep` dentro do laço, “para não gastar tanta CPU”) **não é aceita** como mecanismo de sincronização neste trabalho.

5.2.6 Operações atômicas e travas (*locks*)

Como construir um protocolo de entrada correto se até `contador++` pode ser interrompido? O hardware ajuda oferecendo instruções **atômicas** — que leem e escrevem uma posição de memória em um único passo indivisível. A clássica é o **test-and-set**:

semântica de test-and-set (executada atômicamente pelo hardware)

```
função test_and_set(ref alvo):
    antigo = alvo
    alvo = VERDADEIRO
    retorne antigo
```

Com ela, uma trava simples é: “repita `test_and_set(trava)` enquanto o retorno for verdadeiro”. Quem recebe *falso* foi o primeiro a travar e entra. Isso é um *spinlock* (trava com espera ativa), usado dentro do núcleo para seções críticas curtíssimas. Você **não** precisa implementá-lo: as bibliotecas já oferecem travas prontas, que combinam instruções atômicas com bloqueio no sistema operacional.

Definição: Mutex (*mutual exclusion lock*)

Objeto com dois estados, *livre* e *travado*, e duas operações: **lock** (adquirir: se livre, trava e segue; se travado, bloqueia até ser liberado) e **unlock** (liberar). Apenas a thread que travou deve destravar.

C | mutex POSIX (fragmento)

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;    /* ou pthread_mutex_init */

pthread_mutex_lock(&m);
/* seção crítica: ex. contador++ */
pthread_mutex_unlock(&m);
```

Python | Lock em Python (fragmento)

```
m = threading.Lock()

with m:          # adquiere; libera automaticamente ao sair do bloco
    contador += 1 # seção crítica
```

💡 O GIL do Python não resolve o problema

O interpretador CPython possui o **GIL** (*Global Interpreter Lock*): apenas uma thread executa *bytecode* Python por vez. Isso elimina o paralelismo real em código Python puro, mas **não** elimina condições de corrida: o interpretador troca de thread entre instruções de *bytecode*, e `x += 1` corresponde a várias delas (carregar `x`, somar, armazenar). Um verificar-e-agir também atravessa várias instruções. Portanto, código Python concorrente também precisa de **Lock**, **Semaphore** e **Condition**.

Para tornar corridas *mais visíveis* durante os testes em Python, diminua o intervalo de troca de thread com `sys.setswitchinterval(1e-6)` ou insira um `time.sleep(0)` (que cede a CPU) *entre* a leitura e a escrita de uma variável compartilhada na versão com bug.

5.2.7 Semáforos

Em 1965, Edsger W. Dijkstra propôs o **semáforo**: uma variável inteira não negativa acessada apenas por duas operações atômicas.

📄 Definição: Semáforo

Um semáforo `S` guarda um contador inteiro e uma fila de threads bloqueadas.

- **P(S)** (também `wait`, `down`, `sem_wait`, `acquire`): se o contador for maior que zero, decrementa-o e segue; se for zero, a thread *bloqueia* até que possa decrementá-lo.
- **V(S)** (também `signal`, `up`, `sem_post`, `release`): incrementa o contador e, se houver threads bloqueadas, acorda uma delas.

(As letras vêm do holandês: *proberen*, “testar”, e *verhogen*, “incrementar”).

📄 semântica conceitual (cada operação é atômica)

P(S): enquanto `S.valor == 0`: bloqueie a thread na fila de `S`
`S.valor = S.valor - 1`

V(S): `S.valor = S.valor + 1`
 se há threads bloqueadas em `S`: acorde uma delas

Uma boa intuição: o valor do semáforo é o **número de “fichas” disponíveis**. `P` pega uma ficha (esperando se não houver); `V` devolve uma.

- **Semáforo binário** (valor inicial 1, só assume 0 ou 1): funciona como um mutex — uma única ficha dá direito à seção crítica.
- **Semáforo contador** (valor inicial N): controla um recurso com N unidades idênticas (por exemplo, N posições livres num buffer, ou N cadeiras numa mesa).
- **Semáforo de sinalização** (valor inicial 0): uma thread faz P e espera até que outra faça V — serve para avisar “aconteceu algo”.

C | semáforos POSIX (fragmento)

```
#include <semaphore.h>

sem_t vagas;
sem_init(&vagas, 0, N);    /* 0 = compartilhado entre threads; valor inicial N */
sem_wait(&vagas);          /* P */
sem_post(&vagas);          /* V */
sem_destroy(&vagas);
```

Python | semáforos em Python (fragmento)

```
vagas = threading.Semaphore(N)    # valor inicial N
vagas.acquire()                   # P
vagas.release()                   # V
```

Atenção

Diferença importante entre semáforo e mutex: um semáforo **não tem dono**. Qualquer thread pode fazer V, mesmo sem ter feito P. Isso é o que permite usá-lo para sinalização entre threads diferentes (o produtor faz V no semáforo que o consumidor espera). Em compensação, um V a mais por engano “cria fichas do nada” e quebra a exclusão mútua silenciosamente.

5.2.8 Variáveis de condição e monitores

Muitas vezes a thread precisa esperar não por um recurso, mas por uma *condição* sobre dados compartilhados: “o buffer não está vazio”, “nenhum escritor está esperando”. A **variável de condição** serve para isso e é sempre usada **junto com um mutex**, que protege os dados dos quais a condição depende.

Definição: Variável de condição

Fila de threads esperando que uma condição se torne verdadeira. Operações:

- **wait(c, m)**: *atomicamente* libera o mutex m e bloqueia a thread em c ; ao ser acordada, a thread *readquire* m antes de retornar;
- **signal(c)** (*notify*): acorda uma thread bloqueada em c (se houver; se não houver, o sinal é perdido);
- **broadcast(c)** (*notify_all*): acorda todas.

Por que o *wait* precisa liberar o mutex **atomicamente**? Se a thread liberasse o mutex e só depois bloqueasse, outra thread poderia, nesse intervalo, mudar a condição e sinalizar — o sinal seria perdido e a primeira dormiria para sempre (*lost wake-up*).

O padrão de uso é sempre o mesmo:

padrão de uso de uma variável de condição

```
lock(m)
enquanto NÃO condição:      <- SEMPRE "enquanto", nunca "se"
    wait(c, m)
... usa/altera os dados protegidos por m ...
unlock(m)

(em outra thread, após tornar a condição verdadeira, ainda segurando m:)
signal(c) ou broadcast(c)
```

Atenção

Por que `while` e não `if`? Por três razões: (1) entre o *signal* e o momento em que a thread acordada readquire o mutex, outra thread pode ter entrado e consumido a condição; (2) com *broadcast*, várias acordam mas só uma pode ser atendida; (3) o padrão POSIX permite **despertares espúrios** (*spurious wakeups*): `wait` pode retornar sem que ninguém tenha sinalizado. Reverificar a condição num laço resolve os três casos.

C | variável de condição POSIX (fragmento)

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  c = PTHREAD_COND_INITIALIZER;

pthread_mutex_lock(&m);
while (!condicao)
    pthread_cond_wait(&c, &m);
/* ... */
pthread_mutex_unlock(&m);
```

Python | Condition em Python (fragmento)

```
m = threading.Lock()
c = threading.Condition(m)

with c:
    while not condicao():          # adquiere m
        c.wait()                  # libera m, dorme, readquire m
    ...
# em outra thread, dentro de "with c": c.notify() ou c.notify_all()
```

Um **monitor** é a ideia de agrupar, num só módulo (ou classe), os dados compartilhados, o mutex que os protege, as variáveis de condição e as operações que os manipulam, de modo que *toda* operação pública adquira o mutex ao começar e o libere ao terminar. Um buffer circular com `inserir()` e `remover()` é um exemplo natural de monitor.

5.2.9 Deadlock

Definição: Deadlock (impasse)

Um conjunto de threads está em **deadlock** quando cada uma delas está bloqueada esperando um evento (tipicamente, a liberação de um recurso) que só pode ser causado por outra thread do mesmo conjunto. Ninguém progride, para sempre.

O exemplo mínimo tem duas threads e dois mutexes: T1 trava A e depois tenta B; T2 trava B e depois tenta A. Se ambas fizerem o primeiro lock antes do segundo, travam para sempre.

Coffman e colegas (1971) mostraram que um deadlock só pode ocorrer se **as quatro condições** abaixo valerem simultaneamente:

1. **Exclusão mútua:** ao menos um recurso só pode ser usado por uma thread de cada vez.
2. **Posse e espera** (*hold and wait*): uma thread segura pelo menos um recurso enquanto espera por outro.
3. **Não preempção:** recursos não podem ser tomados à força; só quem os possui pode liberá-los.
4. **Espera circular:** existe um ciclo $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$ em que cada thread espera um recurso segurado pela próxima.

Essas situações são visualizadas no **grafo de alocação de recursos**: círculos são threads, quadrados são recursos; uma seta recurso $\rightarrow$ thread significa “está alocado a”; uma seta thread $\rightarrow$ recurso significa “está esperando por”. Se cada recurso tem uma única instância, **um ciclo no grafo significa deadlock**.

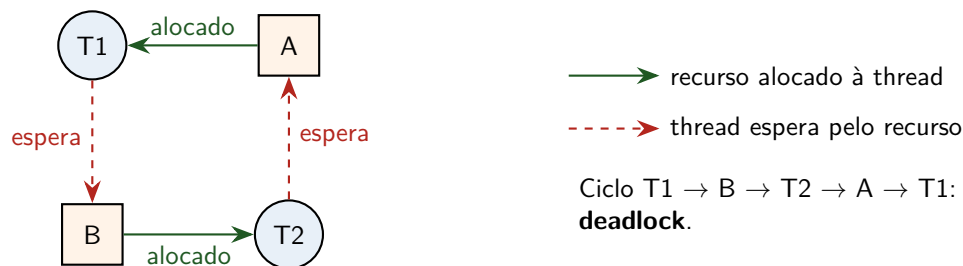


Figura 5.4: Grafo de alocação de recursos com ciclo.

Como evitar deadlock

Como as quatro condições são *necessárias*, basta garantir que **uma delas nunca ocorra** (*prevenção*):

- **Quebrar a espera circular — ordenação de recursos.** Numere todos os recursos e exija que toda thread os adquira *em ordem crescente*. Um ciclo exigiria que alguma thread, segurando um recurso de número alto, esperasse por um de número mais baixo — o que a regra proíbe. É a técnica mais usada na prática.
- **Quebrar a posse e espera.** Adquirir todos os recursos de uma só vez (ou nenhum); ou, se não conseguir o segundo, devolver o primeiro e tentar de novo mais tarde (`pthread_mutex_trylock`, `lock.acquire(timeout=...)`).
- **Limitar a concorrência.** Se N threads disputam N recursos em anel, permitir que no máximo $N - 1$ tentem ao mesmo tempo garante que ao menos uma consiga tudo (é a ideia do “garçom” no Jantar dos Filósofos).

(Há também estratégias de *evitar* deadlock dinamicamente, como o algoritmo do banqueiro, e de *detectar e recuperar*; elas não são necessárias neste trabalho.)

Livelock, starvation e justiça

- **Livelock:** as threads não estão bloqueadas — continuam executando — mas não progridem. Exemplo: dois filósofos que pegam um garfo, percebem que o outro está ocupado, devolvem, e repetem exatamente o mesmo movimento ao mesmo tempo, para sempre (como duas pessoas que tentam desviar uma da outra num corredor). Resolve-se tipicamente com esperas aleatórias.
- **Starvation (inanição):** o sistema como um todo progride, mas uma thread específica é preterida indefinidamente. Exemplo: um escritor que nunca consegue escrever porque sempre há algum leitor lendo.
- **Justiça (fairness):** propriedade de uma solução de distribuir o acesso de forma razoavelmente equilibrada. Ausência de deadlock não implica justiça!

5.2.10 Problema 1: Produtor-Consumidor (buffer limitado)

Enunciado. Threads *produtoras* geram itens e os colocam num buffer de capacidade fixa N ; threads *consumidoras* retiram itens do buffer. Requisitos: (i) um produtor não pode inserir com o buffer cheio — deve bloquear; (ii) um consumidor não pode remover com o buffer vazio — deve bloquear; (iii) inserções e remoções concorrentes não podem corromper o buffer (nenhum item perdido ou duplicado).

O buffer é tipicamente um **vetor circular**: o índice **entrada** indica onde o próximo item será inserido e **saída** de onde o próximo será retirado; ambos avançam “dando a volta” com a operação módulo: $\text{entrada} = (\text{entrada} + 1) \% N$.

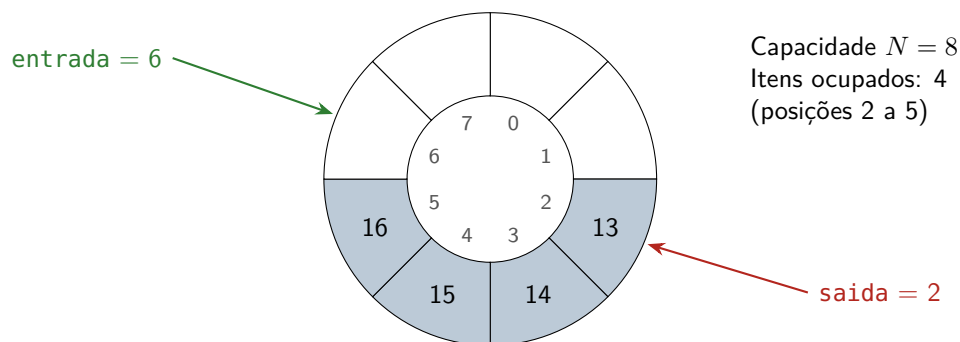


Figura 5.5: Buffer circular com quatro itens. O consumidor retira da posição **saída**; o produtor insere na posição **entrada**.

Ideia da solução com semáforos. Usam-se três objetos de sincronização, cada um com um papel bem definido:

- **vazios:** semáforo contador de *posições livres*, valor inicial N . O produtor faz P antes de inserir (bloqueia se não há vaga) e o consumidor faz V depois de remover (devolve uma vaga).
- **cheios:** semáforo contador de *itens disponíveis*, valor inicial 0. Papel simétrico: o consumidor faz P , o produtor faz V .
- **mutex:** exclusão mútua sobre o vetor e os índices (necessária quando há vários produtores ou vários consumidores).

⚠ Atenção

A **ordem** das operações importa, e descobri-la faz parte do trabalho. Pense no que acontece se um produtor adquirir o **mutex** e só *depois* fizer P em **vazios** com o buffer cheio: ele dorme *segurando* o mutex, e nenhum consumidor consegue entrar para abrir espaço...

Uma solução equivalente usa um mutex e *duas variáveis de condição* (“não cheio” e “não vazio”) com um contador de itens — é a abordagem de monitor. O enunciado pede a versão com semáforos; a versão com variáveis de condição pode ser usada como comparação no relatório.

5.2.11 Problema 2: Jantar dos Filósofos

Enunciado (Dijkstra, 1965). Cinco filósofos sentam-se ao redor de uma mesa redonda. Entre cada par de vizinhos há **um** garfo (cinco garfos ao todo). Cada filósofo alterna entre *pensar* e *comer*; para comer, precisa dos **dois** garfos adjacentes — o da esquerda e o da direita. Cada garfo só pode estar com um filósofo por vez.

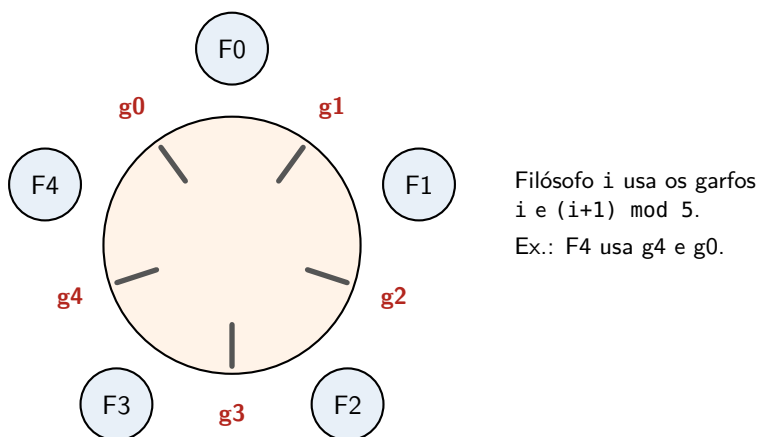


Figura 5.6: A mesa do Jantar dos Filósofos (g_i = garfo i).

A armadilha. A solução ingênua é: “pegue o garfo da esquerda, depois o da direita, coma, devolva os dois”. Se os cinco ficarem com fome ao mesmo tempo e cada um pegar o garfo da esquerda, todos esperarão pelo da direita — que está na mão do vizinho. As quatro condições de Coffman estão presentes e há uma espera circular perfeita: **deadlock**.

Estratégias que funcionam (você deve implementar ao menos uma):

1. **Ordenação de recursos:** cada filósofo pega primeiro o garfo de *menor índice* e depois o de maior. Para F0 a F3 isso é “esquerda, depois direita”; para F4 (garfos 4 e 0) é “0, depois 4”. Essa única assimetria quebra o ciclo.
2. **Garçom (árbitro):** um semáforo contador inicializado com 4 ($N - 1$); o filósofo precisa de uma “ficha” do garçom antes de tentar pegar garfos. Com no máximo 4 disputando 5 garfos, pelo princípio da casa dos pombos ao menos um consegue os dois.
3. **Tentar e devolver:** pega o primeiro garfo e tenta o segundo sem bloquear (ou com tempo limite); se falhar, devolve o primeiro e espera um tempo aleatório (quebra a posse e espera; cuidado com *livelock*).

5.2.12 Problema 3: Leitores-Escritores

Enunciado (Courtois, Heymans e Parnas, 1971). Um recurso compartilhado (por exemplo, uma base de dados) é acessado por *leitores*, que apenas consultam, e *escritores*, que modificam. Como leituras não interferem entre si, **vários leitores podem ler simultaneamente**. Um escritor, porém, precisa de **acesso exclusivo**: nem leitores nem outros escritores durante a escrita.

Primeira variante — prioridade para leitores. Mantém-se um contador de leitores ativos, protegido por um mutex. O *primeiro* leitor que entra trava o recurso (bloqueando escritores); o *último* que sai o libera. Funciona, mas tem um defeito: se leitores chegam continuamente, sempre há ao menos um lendo, o contador nunca volta a zero e o escritor **morre de fome**.

Segunda variante — prioridade para escritores (a exigida neste trabalho). Quando um escritor manifesta a intenção de escrever, *novos* leitores passam a ser barrados na porta, embora os que já estão lendo possam terminar. Assim que o último leitor ativo sai, o escritor entra. Conceitualmente, o estado a controlar é:

estado conceitual (não é código pronto)

```
leitores_ativos      quantos estão lendo agora
escritor_ativo       há um escritor escrevendo agora? (sim/não)
escritores_esperando quantos escritores querem escrever

Leitor pode entrar <=> não há escritor ativo E não há escritor esperando
Escritor pode entrar <=> não há escritor ativo E leitores_ativos == 0
```

Essas regras podem ser implementadas com um mutex e variáveis de condição (abordagem de monitor), ou só com semáforos (a solução clássica usa um semáforo extra, que os escritores seguram enquanto houver algum escritor interessado, para “fechar a porta” aos novos leitores). A escolha é sua e deve ser justificada no relatório.

Dica

Note que a prioridade para escritores pode, por sua vez, causar inanição de *leitores* se escritores chegarem sem parar. Existe ainda uma terceira variante, “justa” (fila FIFO de chegada), que você pode citar na discussão.

5.2.13 Não determinismo e depuração de programas concorrentes

Bugs de concorrência são notoriamente difíceis porque dependem do *escalonamento* das threads, que varia a cada execução conforme a carga da máquina, o número de núcleos, interrupções, o tempo gasto em `printf...` Um programa com uma condição de corrida pode passar em 99 execuções e falhar na centésima. Pior: adicionar um `printf` ou rodar no depurador muda o *timing* e pode esconder o bug — o chamado *heisenbug*.

Por isso, a validação não pode ser apenas “rodei e funcionou”. Use:

- **Logs estruturados** com timestamp de relógio monotônico e identificação da thread, protegidos por um mutex próprio, para reconstruir a ordem real dos eventos.
- **Assertões de invariantes** (ex.: “o número de itens no buffer está entre 0 e N ”, “nunca há leitor e escritor ativos ao mesmo tempo”), verificadas dentro da seção crítica.
- **Execuções repetidas sob estresse** (muitas threads, muitas iterações, pausas aleatórias) para aumentar a probabilidade de intercalações raras.
- **Ferramentas automáticas:**

- C: **ThreadSanitizer** (compilar com `-fsanitize=thread -g`), que aponta corridas de dados e inversões de ordem de locks com as linhas de código envolvidas; **Helgrind** e **DRD** (`valgrind --tool=helgrind` ou `--tool=drd`), que funcionam sobre o binário sem recompilar; **gdb**, para inspecionar um programa travado (`gdb -p PID` e `thread apply all bt` mostram onde cada thread está parada).
- Python: `faulthandler.dump_traceback_later(seg)` imprime a pilha de todas as threads se o programa não terminar a tempo; `py-spy dump --pid PID` mostra onde cada thread de um processo travado está bloqueada.

5.3 Especificação do trabalho

5.3.1 Problemas a implementar

Tabela 5.2: Resumo dos três problemas.

Problema	Requisitos
Produtor-Consumidor	Buffer circular de tamanho finito (ex.: 10). N produtores e M consumidores configuráveis pela linha de comando. Produtor bloqueia com buffer cheio; consumidor bloqueia com buffer vazio. Implementação com semáforos (vazios , cheios) e mutex .
Jantar dos Filósofos	5 filósofos e 5 garfos. Ciclo pensar $\rightarrow$ fome $\rightarrow$ pegar dois garfos $\rightarrow$ comer $\rightarrow$ devolver. Pelo menos uma estratégia que evite deadlock (ordenação de recursos, garçom com semáforo $N - 1$, ou tentativa com tempo limite).
Leitores-Escritores	Múltiplos leitores simultâneos; escritor com acesso exclusivo. Variante com prioridade para escritores (escritor esperando impede a entrada de novos leitores).

5.3.2 Requisito adicional: diagnóstico de bugs

Para **cada um** dos três problemas, entregue:

1. uma versão **correta** e funcional;
2. uma versão **com bug proposital** (ordem de lock trocada, semáforo com valor inicial errado, *unlock* esquecido num caminho de erro, remoção da regra anti-deadlock, etc.) que produza condição de corrida, deadlock ou inanição *observável*;
3. um texto curto explicando **como o bug foi identificado** (logs, **gdb**, **strace**, TSan, Helgrind, **py-spy**, inspeção da ordem de aquisição) e qual correção foi aplicada.

5.3.3 Requisitos técnicos gerais

- Logs com timestamp e identificação da thread, impressos de forma atômica (mutex de log dedicado — `printf` de threads diferentes pode se misturar).
- Parâmetros pela linha de comando: número de threads, tamanho do buffer, número de itens/refeições/operações ou tempo de simulação.
- Em C, compilar com `-Wall -Wextra -pthread`; recomenda-se também um alvo `make tsan` com `-fsanitize=thread`.

- Em Python, usar apenas `threading.Thread`, `Lock`, `Semaphore` e `Condition`.

⚠ Atenção

Em Python é proibido usar **queue.Queue** (ou `SimpleQueue`, `multiprocessing.Queue`) como buffer do Produtor-Consumidor: essa classe já é um buffer limitado sincronizado e esconderia exatamente o que o trabalho pede para você construir. O buffer deve ser uma lista de tamanho fixo com índices **entrada/saida**. Do mesmo modo, em C não use bibliotecas de filas concorrentes prontas.

5.3.4 Estrutura de entrega

📄 estrutura sugerida (C)

```
sincronizacao/
|-- comum/
|   |-- log.c log.h           (log sincronizado com timestamp)
|-- produtor_consumidor/
|   |-- correto.c
|   |-- com_bug.c
|-- filosofos/
|   |-- correto.c
|   |-- com_bug.c
|-- leitores_escritores/
|   |-- correto.c
|   |-- com_bug.c
|-- Makefile                 (alvos: all, tsan, clean)
'-- RELATORIO.pdf
```

📄 estrutura sugerida (Python)

```
sincronizacao/
|-- comum/
|   |-- log.py
|-- produtor_consumidor/
|   |-- correto.py
|   |-- com_bug.py
|-- filosofos/
|   |-- correto.py
|   |-- com_bug.py
|-- leitores_escritores/
|   |-- correto.py
|   |-- com_bug.py
|-- Makefile   (ou run.sh: alvos para executar cada versão com parâmetros padrão)
'-- RELATORIO.pdf
```

5.4 Roteiro de implementação

O roteiro abaixo segue uma ordem pensada para que cada etapa possa ser testada antes da seguinte. Os trechos de código são **parciais**: mostram declarações, chamadas de biblioteca e o esqueleto das funções, mas a lógica de sincronização — a parte que realmente importa — fica marcada com **TUDO** para você completar. Escolha **uma** linguagem (C ou Python) e siga apenas os

trechos correspondentes.

5.4.1 Passo 0 — Utilitário de log sincronizado

Antes de qualquer problema, construa uma função `log_evento(entidade, mensagem)` que imprima uma linha no formato

formato de log recomendado

```
timestamp;entidade;evento;detalhe
0.0012;filosofo_0;tentando_pegar_garfo;garfo_0
0.0018;filosofo_0;pegou_garfo;garfo_0
```

com três cuidados:

1. o timestamp vem de um **relógio monotônico** (que nunca volta para trás), expresso em segundos desde o início do programa;
2. a impressão é protegida por um **mutex exclusivo do log**, separado dos mutexes dos problemas (senão o log interfere na lógica que você quer observar);
3. a saída é descarregada imediatamente (`fflush / flush=True`), para que, se o programa travar, as últimas linhas não fiquem presas no buffer de saída.

C | log.c (fragmento)

```
#include <stdio.h>
#include <stdarg.h>
#include <time.h>
#include <pthread.h>

static pthread_mutex_t mutex_log = PTHREAD_MUTEX_INITIALIZER;
static struct timespec t0;          /* preenchido em log_iniciar() */

static double agora(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return (t.tv_sec - t0.tv_sec) + (t.tv_nsec - t0.tv_nsec) / 1e9;
}

void log_evento(const char *entidade, const char *fmt, ...) {
    /* TODO: travar mutex_log, imprimir "%.4f;%s;" + mensagem formatada
       (use va_list / vprintf), '\n', fflush(stdout), destravar. */
}
```

Python | log.py (fragmento)

```
import threading, time

_mutex_log = threading.Lock()
_t0 = time.monotonic()

def log_evento(entidade, evento, detalhe=""):
    t = time.monotonic() - _t0
    # TODO: dentro do _mutex_log, imprimir f"{t:.4f};{entidade};{evento};{detalhe}"
    # com print(..., flush=True)
```

✓ Verifique antes de seguir

Crie 10 threads que chamam `log_evento` 1000 vezes cada. Nenhuma linha pode sair “misturada” com outra, e o total de linhas deve ser exatamente 10 000 (`./teste | wc -l`).

5.4.2 Passo 1 — Produtor-Consumidor: buffer compartilhado

Defina o buffer circular e os objetos de sincronização. O tamanho do buffer vem da linha de comando, portanto aloque o vetor dinamicamente (C) ou crie uma lista de tamanho fixo (Python).

✎ C | estado compartilhado do Produtor-Consumidor (fragmento)

```
#include <semaphore.h>

typedef struct {
    int *itens;           /* vetor de tamanho 'capacidade' */
    int capacidade;
    int entrada;         /* próxima posição de inserção */
    int saida;           /* próxima posição de remoção */
    sem_t vazios;         /* conta posições livres */
    sem_t cheios;         /* conta itens disponíveis */
    pthread_mutex_t mutex; /* protege itens/entrada/saida */
} Buffer;

void buffer_iniciar(Buffer *b, int capacidade) {
    b->itens = malloc(capacidade * sizeof(int));
    /* TODO: zerar índices; sem_init(&b->vazios, 0, ???);
       sem_init(&b->cheios, 0, ???); pthread_mutex_init(...) */
}
```

🐍 Python | estado compartilhado do Produtor-Consumidor (fragmento)

```
class Buffer:
    def __init__(self, capacidade):
        self.itens = [None] * capacidade # lista de tamanho FIXO (nada de Queue!)
        self.capacidade = capacidade
        self.entrada = 0
        self.saida = 0
        self.vazios = threading.Semaphore(...) # TODO: valor inicial?
        self.cheios = threading.Semaphore(...) # TODO: valor inicial?
        self.mutex = threading.Lock()
```

👉 Dica

Os valores iniciais dos semáforos respondem à pergunta “quantas fichas existem no começo?”. No início, quantas posições livres há? E quantos itens disponíveis? Um valor errado aqui é, inclusive, um ótimo candidato a bug proposital (Passo 7).

5.4.3 Passo 2 — Produtor-Consumidor: produção e consumo

Cada produtor executa um laço: gera um item, espera uma vaga, entra na seção crítica, insere na posição `entrada`, avança o índice circularmente, sai da seção crítica, avisa que há um item novo e registra o evento. O consumidor é simétrico. Descubra a *ordem correta* das operações P/V e lock/unlock — releia a caixa de atenção da Seção 5.2.10 sobre dormir segurando o mutex.

</> C | funções das threads (esqueleto)

```
typedef struct { Buffer *buf; int id; int n_itens; } ArgsPC;

void *produtor(void *arg) {
    ArgsPC *a = arg;
    for (int k = 0; k < a->n_itens; k++) {
        int item = /* gerar item (ex.: a->id * 1000 + k) */;
        /* TODO: sincronização de entrada */
        /* TODO: inserir em buf->itens[buf->entrada] e avançar o índice */
        /* TODO: sincronização de saída */
        log_evento(...); /* fora da seção crítica, se possível */
    }
    return NULL;
}

void *consumidor(void *arg) {
    /* TODO: simétrico ao produtor */
}
```

Python | funções das threads (esqueleto)

```
def produtor(buf, id_prod, n_itens):
    for k in range(n_itens):
        item = id_prod * 1000 + k
        # TODO: sincronização de entrada
        # TODO: inserir em buf.itens[buf.entrada] e avançar o índice
        # TODO: sincronização de saída
        log_evento(f"produtor-{id_prod}", "produziu", f"item-{item}")

def consumidor(buf, id_cons, n_itens):
    ... # TODO: simétrico ao produtor
```

Como terminar o programa? Se cada produtor produz P itens e há n_p produtores, o total é $n_p \cdot P$; distribua esse total entre os n_c consumidores (cuidado com a divisão não exata). Ao final, a `main` imprime o total produzido e o total consumido — e, se quiser uma verificação forte, a **soma** dos valores produzidos e consumidos, que também devem coincidir.

</> C | leitura de parâmetros na main (fragmento)

```
if (argc != 5) {
    fprintf(stderr, "uso: %s <produtores> <consumidores> <tam_buffer> <itens_por_produtores>\n", argv[0]);
    return 1;
}
int np = atoi(argv[1]);
int nc = atoi(argv[2]);
/* ... validar valores positivos ... */
```

Python | leitura de parâmetros (fragmento)

```
import argparse
p = argparse.ArgumentParser()
p.add_argument("--produtores", type=int, default=2)
p.add_argument("--consumidores", type=int, default=2)
```

```
p.add_argument("--buffer", type=int, default=5)
p.add_argument("--itens", type=int, default=10)
args = p.parse_args()
```

✓ Verifique antes de seguir

- 1 produtor e 1 consumidor: total produzido = total consumido.
- Consumidor lento (`usleep(500000)` / `time.sleep(0.5)` após cada consumo): o log deve mostrar o buffer chegando a N/N e o produtor *parando* até haver espaço — sem uso de CPU em espera (confira com `top`).
- Vários produtores e consumidores, buffer de tamanho 1: sem travar, sem perder itens.
- Invariante: o número de itens no buffer nunca é negativo nem maior que a capacidade.

5.4.4 Passo 3 — Jantar dos Filósofos: garfos e estratégia

Represente cada garfo como um mutex (ou um semáforo binário). O filósofo *i* usa os garfos *i* e $(i+1) \% N$. **Antes** de escrever a função do filósofo, decida a estratégia anti-deadlock, pois ela muda a lógica de aquisição.

✎ C | estado do Jantar dos Filósofos (fragmento)

```
#define N 5
pthread_mutex_t garfo[N];
sem_t garcom; /* só se usar a estratégia do garçom */
volatile int executando = 1; /* sinaliza fim da simulação (ver dica) */
int refeicoes[N]; /* estatística por filósofo */

/* em main(): */
for (int i = 0; i < N; i++) pthread_mutex_init(&garfo[i], NULL);
/* sem_init(&garcom, 0, ???); -- estratégia do garçom */
```

🐍 Python | estado do Jantar dos Filósofos (fragmento)

```
N = 5
garfos = [threading.Lock() for _ in range(N)]
garcom = threading.Semaphore(...) # só na estratégia do garçom; valor?
parar = threading.Event() # sinaliza fim da simulação
refeicoes = [0] * N
```

👉 Dica

Para simulações com duração fixa (ex.: 10 segundos), a `main` dorme pelo tempo desejado, sinaliza o fim (em C, uma flag protegida por mutex ou `atomic_int`; em Python, `threading.Event`) e faz `join` nas threads. Cada filósofo verifica a flag no início de cada ciclo.

5.4.5 Passo 4 — Jantar dos Filósofos: função do filósofo

Laço de cada filósofo: pensar (pausa aleatória) → registrar “com fome” → adquirir os dois garfos *segundo a estratégia* → registrar “comendo” → comer (pausa aleatória) → liberar os garfos (ordem inversa da aquisição) e incrementar o contador de refeições → registrar “pensando”.

</> C | função do filósofo (esqueleto)

```

void *filosofo(void *arg) {
    int i = *(int *)arg;
    int esq = i, dir = (i + 1) % N;
    unsigned int semente = time(NULL) ^ i; /* rand_r é seguro entre threads */
    while (executando) {
        log_evento(..., "pensando");
        usleep(rand_r(&semente) % 100000);
        log_evento(..., "com fome, garfos %d e %d", esq, dir);
        /* TODO: adquirir os dois garfos conforme a estratégia escolhida */
        /* TODO: comer (log + pausa) e contar a refeição */
        /* TODO: liberar os garfos */
    }
    return NULL;
}

```

Python | função do filósofo (esqueleto)

```

def filosofo(i):
    esq, dir = i, (i + 1) % N
    while not parar.is_set():
        log_evento(f"filosofo-{i}", "pensando")
        time.sleep(random.uniform(0, 0.1))
        log_evento(f"filosofo-{i}", "com fome", f"garfos_{esq}_{dir}")
        # TODO: adquirir os dois garfos conforme a estratégia escolhida
        # TODO: comer (log + pausa) e contar a refeição
        # TODO: liberar os garfos

```

✓ Verifique antes de seguir

- Rode por 10–30 segundos: todos os filósofos devem comer várias vezes (imprima `refeicoes[]` ao final).
- Nunca dois vizinhos comendo ao mesmo tempo (confira no log: entre “comendo” e “liberou” de F_i , não pode haver “comendo” de $F(i \pm 1)$).
- Reduza as pausas a zero e aumente a duração: continua sem travar?

5.4.6 Passo 5 — Leitores-Escritores: estado com prioridade para escritores

O recurso compartilhado pode ser simples — por exemplo, um inteiro que o escritor incrementa e o leitor lê — mas o controle de acesso deve seguir as regras conceituais da Seção de fundamentação. A abordagem mais legível é a de **monitor** (um mutex + variáveis de condição + contadores); a abordagem só com semáforos também é aceita.

</> C | estado do Leitores-Escritores, abordagem de monitor (fragmento)

```

typedef struct {
    pthread_mutex_t mutex;
    pthread_cond_t pode_ler;
    pthread_cond_t pode_escrever;
    int leitores_ativos;
    int escritores_esperando;
    int escritor_ativo; /* 0 ou 1 */
}

```

```
int dado;                /* o "recurso" compartilhado */
} ControleLE;
```

Python | estado do Leitores-Escritores, abordagem de monitor (fragmento)

```
class ControleLE:
    def __init__(self):
        self.mutex = threading.Lock()
        self.pode_ler = threading.Condition(self.mutex)
        self.pode_escrever = threading.Condition(self.mutex)
        self.leitores_ativos = 0
        self.escritores_esperando = 0
        self.escriptor_ativo = False
        self.dado = 0
```

👉 Dica

Separe cada lado em duas funções de protocolo — `entrar_leitura()`, `sair_leitura()`, `entrar_escrita()`, `sair_escrita()` — e deixe a leitura/escrita propriamente dita (a pausa simulando trabalho) *fora* do mutex. Isso é o que permite vários leitores simultâneos.

5.4.7 Passo 6 — Leitores-Escritores: leitura e escrita

Pense em cada protocolo respondendo a duas perguntas: *sob qual condição eu espero?* e *quem eu preciso acordar quando saio?*

- **Entrar na leitura:** esperar enquanto houver escritor ativo *ou esperando*; depois, contar-se como leitor ativo.
- **Sair da leitura:** descontar-se; se foi o último leitor, alguém precisa ser avisado.
- **Entrar na escrita:** registrar a intenção (é isso que barra novos leitores), esperar até que o recurso esteja totalmente livre, e então marcar-se como ativo.
- **Sair da escrita:** desmarcar-se e decidir quem acordar — outro escritor esperando (prioridade) ou todos os leitores.

📄 C | protocolos de entrada/saída (esqueleto)

```
void entrar_leitura(ControleLE *c) {
    pthread_mutex_lock(&c->mutex);
    /* TODO: while (condição de espera do leitor) pthread_cond_wait(...); */
    /* TODO: atualizar contadores */
    pthread_mutex_unlock(&c->mutex);
}

void sair_leitura(ControleLE *c) { /* TODO */ }
void entrar_escrita(ControleLE *c) { /* TODO */ }
void sair_escrita(ControleLE *c) { /* TODO: signal ou broadcast? para quem? */ }

void *leitor(void *arg) {
    /* laço: entrar_leitura; log "iniciou leitura (ativos: X)"; pausa;
       log "terminou"; sair_leitura; pausa fora do recurso */
}
```

Python | protocolos de entrada/saída (esqueleto)

```
def entrar_leitura(c):
    with c.mutex:
        # TODO: while <condição de espera do leitor>: c.pode_ler.wait()
        # TODO: atualizar contadores
        ...

def sair_leitura(c): ... # TODO
def entrar_escrita(c): ... # TODO
def sair_escrita(c): ... # TODO: notify() ou notify_all()? em qual Condition?
```

⚠ Atenção

Para imprimir “leitores ativos: X” corretamente, leia o contador *enquanto ainda segura o mutex* (guarde numa variável local e registre depois). Ler o contador fora do mutex para montar o log é, por si só, uma corrida de dados que o ThreadSanitizer vai apontar.

✓ Verifique antes de seguir

- Com 4 leitores e 1 escritor, o log deve mostrar “leitores ativos” maior que 1 em algum momento (leituras são simultâneas).
- Nunca aparece escrita com leitores ativos, nem duas escritas sobrepostas (adicione um `assert` dentro de `entrar_escrita`).
- Com 8 leitores relendo sem parar e 1 escritor, o escritor consegue escrever em tempo limitado (meça o tempo entre “aguardando” e “iniciou escrita”).

5.4.8 Passo 7 — Versões com bug proposital e diagnóstico

Parta de cada versão *correta e já validada* e introduza **um único defeito** realista. Sugestões:

Tabela 5.3: Sugestões de defeitos propositais e sintomas esperados.

Problema	Defeito	Sintoma esperado
Produtor-Consumidor	Remover o mutex da inserção/remoção (com vários produtores e consumidores).	Itens perdidos ou duplicados; totais ou somas não batem; resultado muda a cada execução.
Produtor-Consumidor	Inverter a ordem: adquirir o mutex antes do P no semáforo.	Deadlock quando o buffer enche (ou esvazia).
Produtor-Consumidor	Valor inicial errado de um semáforo (ex.: <code>vazios = capacidade + 1</code>).	Sobrescrita de item não consumido; invariante do buffer violado.
Filósofos	Todos pegam esquerda e depois direita, sem estratégia.	Deadlock circular: todos seguram um garfo e ninguém imprime mais nada.
Leitores-Escritores	Remover a regra que barra leitores quando há escritor esperando.	Inanição do escritor sob fluxo contínuo de leitores.
Qualquer um	Esquecer o <code>unlock</code> num caminho de retorno antecipado.	Travamento após certo número de iterações.

Tornando o bug reproduzível

Bugs de concorrência se escondem. Para aumentar a probabilidade de manifestação:

- insira uma pausa *na janela perigosa* — por exemplo, entre pegar o primeiro e o segundo garfo, ou entre ler e escrever o índice do buffer (`usleep(1000)`; em Python, `time.sleep(0)` ou `time.sleep(0.001)`);
- aumente o número de threads e de iterações;
- em Python, reduza o intervalo de troca com `sys.setswitchinterval(1e-6)`;
- execute muitas vezes seguidas com um laço no terminal e conte as falhas.

>_ executando 20 vezes e contando falhas (exemplo)

```
$ for i in $(seq 20); do timeout 10 ./pc_bug 4 4 5 1000 | tail -1; done | sort |
  uniq -c
  13 [FIM] Total produzido: 4000 | Total consumido: 4000
   7 [FIM] Total produzido: 4000 | Total consumido: 3991
```

Detectando travamentos: o “vigia”

Uma técnica simples e muito didática é uma thread **vigia**: cada thread de trabalho atualiza um registro de “último progresso” (timestamp), e o vigia, periodicamente, verifica se *todas* estão sem progredir há mais de, por exemplo, 2 segundos. Nesse caso, imprime “possível deadlock” e o que cada uma esperava.

Python | vigia e dump de pilhas em Python (fragmento)

```
import faulthandler, sys
# se o programa não terminar em 15 s, imprime a pilha de TODAS as threads:
faulthandler.dump_traceback_later(15, exit=True, file=sys.stderr)

def vigia(ultimo_progresso, limite=2.0):
    while not parar.is_set():
        time.sleep(0.5)
        # TODO: se agora - max(ultimo_progresso) > limite: log "possível deadlock"
```

Ferramentas de diagnóstico

>_ C: ThreadSanitizer

```
$ gcc -g -O1 -fsanitize=thread -pthread com_bug.c ../comum/log.c -o pc_tsan
$ ./pc_tsan 2 2 5 100
=====
WARNING: ThreadSanitizer: data race (pid=4242)
  Write of size 4 at 0x7b0c00000010 by thread T2:
    #0 produtor com_bug.c:41
  Previous read of size 4 at 0x7b0c00000010 by thread T3:
    #0 consumidor com_bug.c:63
```

>_ C: Helgrind e DRD (sem recompilar)

```
$ valgrind --tool=helgrind ./filosofos_bug 5 10
==9311== Thread #3: lock order "0x4C8E0C0 before 0x4C8E0E8" violated
$ valgrind --tool=drd ./leitores_bug 8 1
```

>_ C: inspecionando um programa travado com gdb

```
$ ./filosofos_bug 5 30 & # travou
$ gdb -p $(pgrep filosofos_bug)
(gdb) thread apply all bt # onde cada thread está parada?
#0 __lll_lock_wait ... #2 pthread_mutex_lock ... #3 filosofo (arg=...) at com_bug.
  c:52
(gdb) frame 3
(gdb) print i
```

>_ Python: py-spy

```
$ pip install py-spy
$ python3 filosofos/com_bug.py &
$ py-spy dump --pid $(pgrep -f com_bug.py)
Thread 0x7F... (idle): "Thread-3 (filosofo)"
    filosofo (com_bug.py:41)
```

⚠ Atenção

ThreadSanitizer e Helgrind detectam **corridas de dados** e **inversões de ordem de locks**. Eles *não* detectam diretamente inanição nem todos os deadlocks com semáforos. Para esses, a evidência vem do log (quem esperou quanto tempo) e da inspeção do programa travado.

Para cada versão com bug, documente no relatório: (1) o defeito introduzido; (2) o sintoma observado; (3) a taxa de reprodução; (4) a ferramenta ou técnica que levou ao diagnóstico, com trecho da saída; (5) a correção.

</> C | Makefile (fragmento)

```
CFLAGS = -Wall -Wextra -g -pthread
all: pc_correto pc_bug filosofos_correto filosofos_bug le_correto le_bug
pc_correto: produtor_consumidor/correto.c comum/log.c
    $(CC) $(CFLAGS) $^ -o $@
# ... demais alvos ...
tsan: CFLAGS += -fsanitize=thread -O1
tsan: clean all
```

5.5 Exemplos de saída esperada

Como a execução é concorrente, as linhas *variam* a cada execução. O que se descreve aqui são os **padrões** esperados e os **sintomas** de erro. (Os logs abaixo estão em formato legível; o formato `timestamp;entidade;evento;detalhe` do Passo 0 é o recomendado para processamento automático.)

5.5.1 Produtor-Consumidor

>_ 2 produtores, 2 consumidores, buffer 5, 10 itens por produtor (correto)

```
[Produtor-1] produziu item 1000 (buffer: 1/5)
[Produtor-2] produziu item 2000 (buffer: 2/5)
[Consumidor-1] consumiu item 1000 (buffer: 1/5)
[Produtor-1] produziu item 1001 (buffer: 2/5)
...
[FIM] Total produzido: 20 | Total consumido: 20
```

A contagem final deve coincidir e o nível do buffer nunca ultrapassa 5 nem fica negativo.

>_ 1 produtor rápido, 1 consumidor lento (500 ms), buffer 3

```
[Produtor] produziu item 1 (buffer: 1/3)
[Produtor] produziu item 2 (buffer: 2/3)
[Produtor] produziu item 3 (buffer: 3/3)
[Produtor] buffer cheio, aguardando espaço...
[Consumidor] consumiu item 1 (buffer: 2/3)
[Produtor] produziu item 4 (buffer: 3/3)
```

O produtor **bloqueia** (sem espera ativa) e só retoma quando há espaço.

>_ versão com bug (seção crítica removida) – duas execuções

```
[FIM] Total produzido: 20 | Total consumido: 17
ERR0: contagem final não corresponde -- itens perdidos ou sobrescritos

[FIM] Total produzido: 20 | Total consumido: 20
(execução "sortuda": o bug não se manifestou, mas continua lá)
```

5.5.2 Jantar dos Filósofos

>_ 5 filósofos, ordenação de recursos, 10 s (correto)

```
[Filosofo-2] com fome, tentando pegar garfos 2 e 3
[Filosofo-2] pegou garfo 2
[Filosofo-2] pegou garfo 3
[Filosofo-2] comendo
[Filosofo-2] devolveu garfos 2 e 3
[Filosofo-2] pensando
...
[FIM] Refeições: F0=8, F1=7, F2=9, F3=8, F4=7
```

>_ versão com deadlock (esquerda, depois direita)

```
[Filosofo-0] pegou garfo esquerdo (0)
[Filosofo-1] pegou garfo esquerdo (1)
[Filosofo-2] pegou garfo esquerdo (2)
[Filosofo-3] pegou garfo esquerdo (3)
[Filosofo-4] pegou garfo esquerdo (4)
[Filosofo-0] aguardando garfo direito (1)...
[Filosofo-1] aguardando garfo direito (2)...
```

```
[Filosofo-2] aguardando garfo direito (3)...  
[Filosofo-3] aguardando garfo direito (4)...  
[Filosofo-4] aguardando garfo direito (0)...  
(nenhuma linha nova: todas as threads bloqueadas para sempre)
```

>_ estratégia do garçom (semáforo com 4 fichas)

```
[Filosofo-3] aguardando permissão do garçom (4 já à mesa)  
[Filosofo-0] devolveu garfos, liberando lugar  
[Filosofo-3] pegou garfo esquerdo (3)  
...  
[FIM] Refeições: F0=6, F1=7, F2=6, F3=8, F4=7
```

5.5.3 Leitores-Escritores

>_ 4 leitores e 1 escritor: leituras simultâneas

```
[Leitor-1] iniciou leitura (leitores ativos: 1)  
[Leitor-2] iniciou leitura (leitores ativos: 2)  
[Leitor-3] iniciou leitura (leitores ativos: 3)  
[Leitor-1] terminou leitura (leitores ativos: 2)  
[Leitor-4] iniciou leitura (leitores ativos: 3)
```

>_ prioridade para escritores (correto)

```
[Escritor-1] aguardando (leitores ativos: 2)  
[Leitor-6] aguardando (há escritor na fila)  
[Leitor-2] terminou leitura (leitores ativos: 1)  
[Leitor-5] terminou leitura (leitores ativos: 0)  
[Escritor-1] iniciou escrita (acesso exclusivo)  
[Escritor-1] terminou escrita  
[Leitor-6] iniciou leitura (leitores ativos: 1)
```

>_ versão com bug: inanição do escritor (8 leitores contínuos)

```
[Escritor-1] aguardando (leitores ativos: 1)  
[Leitor-5] iniciou leitura (leitores ativos: 2)  
[Escritor-1] aguardando (leitores ativos: 2)  
[Leitor-2] iniciou leitura (leitores ativos: 3)  
...  
[t=30s] [Escritor-1] ainda aguardando -- nunca conseguiu escrever
```

5.6 Experimentos

Como o comportamento varia entre execuções, rode cada cenário **pelo menos 10 vezes** e reporte estatísticas (média, mínimo, máximo, frequência do bug), não uma execução isolada.

Tabela 5.4: Experimentos mínimos.

Experimento	Objetivo
Produtor-Consumidor com bug, 20 execuções, registrando os totais	Mostrar o não determinismo da condição de corrida (taxa de falha).
Produtor-Consumidor correto variando o tamanho do buffer (1, 5, 50)	Throughput (itens/s) e tempo médio de espera por vaga.
Filósofos com bug até o deadlock, medindo o tempo até travar	Documentar a reprodutibilidade do deadlock (com e sem pausa entre os garfos).
Filósofos: ordenação versus garçom	Comparar refeições por filósofo e total (throughput e justiça).
Leitores-Escritores com e sem prioridade para escritores	Tempo médio (e máximo) de espera do escritor; número de escritas em 30 s.
Escalabilidade: variar o número de threads (ex.: 2, 8, 32)	Discutir contenção sob carga crescente.
Execução sob TSan/Helgrind (C) ou com <code>setswitchinterval</code> reduzido (Python)	Evidência de ausência de corridas na versão correta e detecção na versão com bug.

Dica

Métricas como “tempo de espera por recurso” saem diretamente do log: é a diferença entre os timestamps de “tentando pegar” e “pegou”. Um script Python curto, lendo o arquivo `;-separado`, calcula médias por entidade e pode desenhar uma **linha do tempo** (uma faixa por filósofo, colorida por estado) — uma figura excelente para o relatório. Gere-a a partir de uma execução *real*, nunca “de memória”.

5.7 Perguntas para o relatório

Na seção de Discussão do relatório (modelo SBC), responda:

1. Qual estratégia você escolheu para evitar deadlock no Jantar dos Filósofos? Justifique informalmente por que ela garante ausência de deadlock (por exemplo, por que a ordenação de recursos impede a formação de um ciclo de espera — use as condições de Coffman).
2. No Leitores-Escritores, como sua solução garante que escritores não sofram inanição quando há um fluxo contínuo de leitores? Ela pode causar inanição de leitores? Mostre evidência experimental.
3. Descreva um caso em que o bug proposital foi difícil de reproduzir de forma determinística. Por que bugs de concorrência costumam ser não determinísticos?
4. (Produtor-Consumidor) O que acontece se a ordem entre o P no semáforo e o lock do mutex for invertida? Explique com uma intercalação concreta.
5. (Python) O GIL impediu a manifestação de alguma corrida? Por que ele não torna o código seguro?

5.8 Critérios de avaliação e checklist

Tabela 5.5: Critérios de avaliação.

Critério	Peso
Produtor-Consumidor correto	20%
Jantar dos Filósofos correto (sem deadlock)	20%
Leitores-Escritores correto (com prioridade de escritor)	20%
Versões com bug + diagnóstico documentado (3 problemas)	25%
Uso de ferramentas de análise (TSan, Helgrind, gdb, py-spy etc.)	15%

Checklist de entrega

Ambiente e compilação

- ☐ Compila e executa em Linux (nativo ou WSL).
- ☐ (C) Sem *warnings* com `-Wall -Wextra; -pthread` na compilação e na ligação; `make` e `make clean` funcionam.
- ☐ (Python) Executa com `python3` sem dependências externas (exceto ferramentas de diagnóstico).

Funcionalidade

- ☐ Produtor-Consumidor: buffer circular próprio (sem `queue.Queue`), sem perda nem duplicação de itens; bloqueio correto com buffer cheio/vazio.
- ☐ Jantar dos Filósofos: livre de deadlock, comprovado em múltiplas execuções; nenhum filósofo fica indefinidamente sem comer.
- ☐ Leitores-Escritores: leitores simultâneos, escritor exclusivo, prioridade para escritores, contador de leitores sem corrida.
- ☐ Logs com timestamp e identificação da thread nos três problemas.
- ☐ Parâmetros configuráveis pela linha de comando.

Qualidade

- ☐ Mutexes, semáforos e variáveis de condição inicializados e destruídos corretamente.
- ☐ Sem espera ativa.
- ☐ (C) Sem vazamentos (`valgrind`); sem corridas na versão correta (TSan/Helgrind).

Versões com bug e relatório

- ☐ Três versões com bug, cada uma com defeito único e sintoma observável.
- ☐ Diagnóstico documentado (sintoma, taxa de reprodução, ferramenta, correção).
- ☐ Relatório no Overleaf, modelo SBC, fontes `.tex` e PDF entregues; perguntas respondidas.
- ☐ Código produzido pelo próprio grupo (sem plágio e sem geração por IA).

5.9 Leituras recomendadas

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. Capítulos sobre Sincronização de Processos, Problemas Clássicos de Sincronização e Deadlocks.
- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. Capítulo 2 (comunicação entre processos, problemas clássicos) e capítulo 6 (impasses).
- ARPACI-DUSSEAU, R.; ARPACI-DUSSEAU, A. *Operating Systems: Three Easy Pieces*. Capítulos *Locks*, *Condition Variables*, *Semaphores* e *Common Concurrency Problems* (gratuitos em <https://pages.cs.wisc.edu/~remzi/OSTEP/>).
- KERRISK, M. *The Linux Programming Interface*. Capítulos 29 a 33 (threads, sincronização) e 53 (semáforos POSIX).
- DOWNEY, A. *The Little Book of Semaphores* (gratuito), com dezenas de variações dos problemas clássicos.
- Páginas de manual: `pthread_create(3)`, `pthread_mutex_lock(3p)`, `pthread_cond_wait(3p)`, `sem_init(3)`, `sem_wait(3)`.
- Documentação do Python: módulo `threading` (<https://docs.python.org/3/library/threading.html>) e `faulthandler`.

Apêndice A

Equivalências entre C e Python

Esta tabela resume as funções e os tipos usados nos cinco trabalhos, lado a lado nas duas linguagens. Use-a como referência rápida; os detalhes de cada chamada estão nos capítulos correspondentes e nas páginas de manual (`man 2 fork`, `man 3 sem_wait`, ...) ou na documentação oficial do Python.

A.1 Processos, arquivos e sinais (Capítulo 1)

C	Python	Observação
<code>fork()</code>	<code>os.fork()</code>	0 no filho, PID do filho no pai
<code>execvp(arq, argv)</code>	<code>os.execvp(arq, args)</code>	em Python, erro vira exceção <code>OSError</code>
<code>waitpid(pid, &st, 0)</code>	<code>os.waitpid(pid, 0)</code>	Python devolve a tupla <code>(pid, status)</code>
<code>WIFEXITED / WEXITSTATUS</code>	<code>os.WIFEXITED / os.WEXITSTATUS</code>	interpretam o status
<code>pipe(fd)</code>	<code>r, w = os.pipe()</code>	<code>fd[0]</code> /r leitura; <code>fd[1]</code> /w escrita
<code>dup2(velho, novo)</code>	<code>os.dup2(velho, novo)</code>	
<code>open(caminho, flags, 0644)</code>	<code>os.open(caminho, flags, 0o644)</code>	flags <code>O_WRONLY</code> , <code>O_CREAT</code> , ...
<code>close(fd)</code>	<code>os.close(fd)</code>	
<code>chdir(caminho)</code>	<code>os.chdir(caminho)</code>	builtin <code>cd</code>
<code>getcwd(buf, n)</code>	<code>os.getcwd()</code>	builtin <code>pwd</code>
<code>setenv(nome, valor, 1)</code>	<code>os.environ[nome] = valor</code>	builtin <code>export</code>
<code>sigaction(SIGINT, ...)</code>	<code>signal.signal(signal.SIGINT, h)</code>	
<code>_exit(1)</code>	<code>os._exit(1)</code>	encerrar o filho sem executar limpezas do pai

A.2 Threads e sincronização (Capítulo 5)

C (pthreads)	Python (threading)	Observação
<code>pthread_create(&t, NULL, f, arg)</code>	<code>t = Thread(target=f, args=(a,)); t.start()</code>	
<code>pthread_join(t, NULL)</code>	<code>t.join()</code>	
<code>pthread_mutex_lock / unlock</code>	<code>lock.acquire() / release()</code>	ou with lock:
<code>sem_init(&s, 0, n)</code>	<code>s = Semaphore(n)</code>	
<code>sem_wait(&s) / sem_post(&s)</code>	<code>s.acquire() / s.release()</code>	operações P e V
<code>pthread_cond_wait(&c, &m)</code>	<code>cond.wait()</code>	sempre dentro de um laço while
<code>pthread_cond_signal / broadcast</code>	<code>cond.notify() / notify_all()</code>	
<code>usleep(us)</code>	<code>time.sleep(s)</code>	Python recebe segundos (float)

A.3 Estruturas de dados (Capítulos 2, 3 e 4)

C	Python	Observação
<code>typedef struct {...} Processo;</code>	<code>@dataclass class Processo:</code>	registro de campos
<code>vetor + índices início/fim</code>	<code>collections.deque</code>	fila (FIFO, Round Robin)
<code>malloc / free</code>	(gerência automática)	no alocador, Python usa bytearray
<code>ponteiro Bloco *</code>	<code>deslocamento int no bytearray</code>	
<code>leitura de campo b->tamanho</code>	<code>struct.unpack_from(fmt, heap, off)</code>	
<code>escrita de campo</code>	<code>struct.pack_into(fmt, heap, off, ...)</code>	
<code>fscanf / fgets + sscanf</code>	<code>open(...), linha.split()</code>	leitura de arquivos de entrada
<code>qsort</code>	<code>lista.sort(key=...)</code>	ordenação com critério

Apêndice B

Bibliografia geral

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. LTC.
- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. Pearson.
- STALLINGS, W. *Sistemas Operacionais: Princípios e Projeto*. Pearson.
- ARPACI-DUSSEAU, R. H.; ARPACI-DUSSEAU, A. C. *Operating Systems: Three Easy Pieces*. Disponível gratuitamente em <https://pages.cs.wisc.edu/~remzi/OSTEP/>.
- KERRISK, M. *The Linux Programming Interface*. No Starch Press.
- KERNIGHAN, B. W.; RITCHIE, D. M. *The C Programming Language*. 2. ed. Prentice Hall.
- Páginas de manual do Linux: <https://man7.org/linux/man-pages/>.
- Documentação oficial do Python: <https://docs.python.org/3/>.